

角色动画与运动仿真 课程笔记

AutoPku

2026-05-20

Contents

课程概览	4
课程脉络	5
知识图谱（全课程）	7
各讲速览	8
全局术语中英对照	9
Lecture 01 —Introduction to 3D Character Animation	12
0. 课程定位与目标	12
1. 什么是 Character Animation	12
2. Pipeline 与生物学动机	13
3. 方法学坐标系：high-level goals low-level control physics	13
4. Kinematic 方法谱系	14
5. Physics-based / Dynamic 方法谱系	15
6. 全图：把谱系并排	15
7. 课程时间线（讲次概览）	16
Lecture 02 —数学基础：旋转的四种表示	17
0. 大纲与动机 / Motivation	17
1. 线性代数复习	17
2. 刚体变换：缩放、平移、旋转	18
3. Rodrigues 公式：从轴角到旋转矩阵	19
4. 旋转表示 1：旋转矩阵	20
5. 旋转表示 2：Euler 角	20
6. 旋转表示 3：旋转向量 / 轴角	21
7. 旋转表示 4：四元数	21
8. 四种表示对比与转换	23
9. 其他表示（一笔带过）	24
记号速查	24
Lecture 03 —Character Kinematics	25
0. 角色运动学全景 / Why Kinematics	25
1. 骨骼与单链 FK / Skeleton & Chain Forward Kinematics	27
2. 角色 FK（树状骨架） / Character FK	28
3. Retargeting：在不同 reference pose 间迁移动作	29
4. Inverse Kinematics：问题与启发式解法	29
5. IK 作为优化问题 / IK as Optimization	31
6. Jacobian 方法 / Gradient and Newton-style Updates	32
7. 全身 IK / Full-body IK	35
8. 全局总结	36
记号速查	36

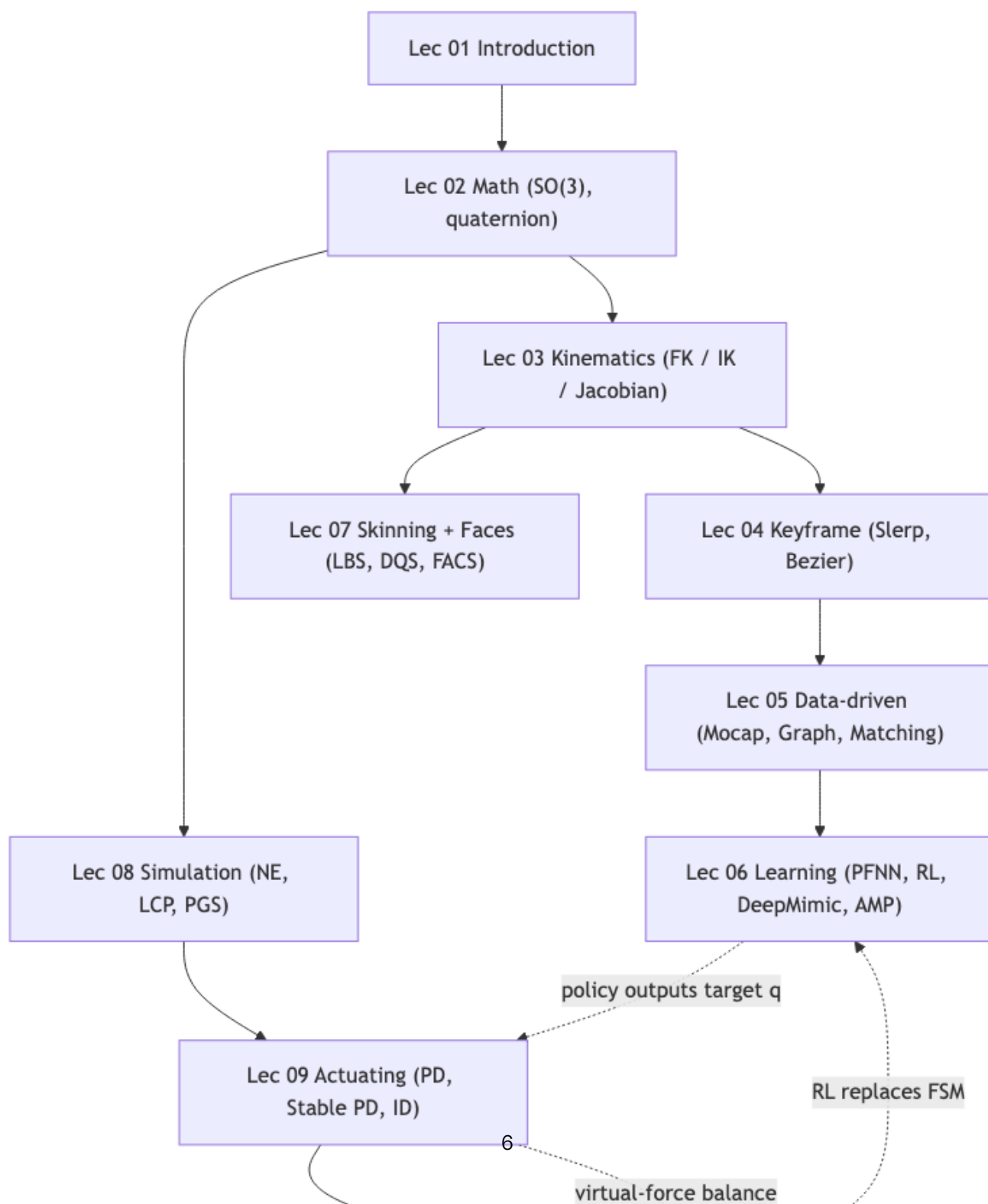
Lecture 04 —Keyframe Animation	38
0. 从 Zoetrope 到 Keyframe	38
1. 插值问题的形式化	38
2. 多项式插值与 Runge 现象	39
3. 样条插值 (Spline Interpolation)	40
4. 旋转的插值 (Interpolation of Rotations)	42
5. 多维 / 散点插值	44
6. 总览: 从关键帧到曲线的选择树	46
7. 速查表	46
记号速查	46
Lecture 05 —Data-driven Animation	48
0. 课程脉络 / Outline	48
1. Motion Capture 动作捕捉	48
2. Motion Data 动作数据	49
3. Motion Retargeting 动作重定向	50
4. Motion Editing 动作编辑	51
5. Motion Transition 动作过渡	51
6. Motion Composition 动作组合	52
7. Motion Graph 动作图	53
8. Motion Fields 与 Motion Matching	54
9. 各方法横向对比	55
10. 一帧 Motion Matching 伪代码	55
记号速查	56
Lecture 06 —Learning-based Animation	57
0. 核心问题 / Motivation	57
1. 人体运动的低维性 / Low-dimensionality	58
2. PCA 与运动先验 / Principal Component Analysis	59
3. 高斯模型族 / Gaussian Family	60
4. 学习运动模型的两视角 / Two Perspectives	61
5. 自回归模型 / Autoregressive Model	61
6. PFNN: Phase-Functioned Neural Networks (Holden et al. SIGGRAPH 2017)	62
7. 从 PCA 到 Autoencoder	63
8. Variational Autoencoder (VAE)	64
9. VQ-VAE: Vector-Quantized VAE	66
10. Diffusion Models	66
11. 生成模型大全 / Generative Model Zoo	68
12. 其他应用 / Other Applications	68
13. 总结 / Summary	69
Lecture 07 —Skinning & Facial Animation	70
0. Pipeline 总览 / Character Animation Pipeline	70
1. Skinning Deformation 的几何推导	70
2. Linear Blend Skinning (LBS / 线性混合蒙皮)	71
3. Dual Quaternion Skinning (DQS / 对偶四元数蒙皮)	72
4. LBS / DQS / PSD 方法对比	74
5. Pose Space Deformation (PSD)	75
6. SMPL / 一个工业级人体模型	76
7. Facial Animation / 面部动画	77
8. 总结一页	78
记号速查	78
Lecture 08 —基于物理的仿真与铰接刚体	79
0. 核心问题 / Motivation	79
1. 质点动力学与时间离散	79
2. 刚体运动学: 位姿、速度、积分	81

3. 刚体动力学：动量、惯量、Newton—Euler	82
4. 铰接刚体与约束	83
5. 接触	85
6. 把所有东西拼起来：一步刚体系统仿真	88
记号速查	89
Lecture 09 一角色驱动：关节力矩、PD 控制与平衡	90
0. 核心问题 / Motivation	90
1. 从仿真到驱动：极大坐标与广义坐标	90
2. 力 vs. 力矩：施加位置的等价变换	92
3. 正动力学 vs. 逆动力学	93
4. PD 控制：最简的反馈律	94
5. Jacobian Transpose Control (虚拟力控制)	96
6. 跟踪控制：让角色”跳舞”	97
7. 静态平衡 (Static Balance)	98
8. 概念结构图	99
9. 工程要点速记	99
记号速查	99
10. 与上下游课的衔接	100
Lecture 10 —Learning to Walk：动态平衡与简化模型	101
0. 核心问题 / Motivation	101
1. 零力矩点 ZMP：脚不翻转的判据	101
2. 线性倒立摆模型 LIPM	103
3. ZMP-Based Walking Pattern Generation (Cart-Table 模型)	104
4. SIMBICON：4 状态 FSM 走路控制	105
5. 概念结构图	107
6. 三种简化模型对比	107
7. 走路控制律的层级	107
8. 与 Lec 06 学习派的衔接	108
9. 工程要点速记	108
记号速查	108
10. 关键文献	109
Lecture 11 —Optimal Control & Reinforcement Learning：从 Pontryagin 到 LLM	110
0. 章节地图	111
1. 问题设定：从 Trajectory Tracking 到 Optimal Control	111
2. Trajectory Optimization: Lagrange / KKT / Pontryagin	112
3. Dynamic Programming 与 Bellman 方程	115
4. Linear Quadratic Regulator (LQR)	117
5. Sampling-based Policy Optimization (Terrain Runner 案例)	118
6. 从最优控制到 Reinforcement Learning	119
7. Value-Based Methods	120
8. Policy Gradient Methods	121
9. Model-Based Reinforcement Learning	123
10. 生成式控制：从 VAE 到 LLM	124
11. 结论与全局对照	125
12. 记号速查	125
13. 参考文献	126

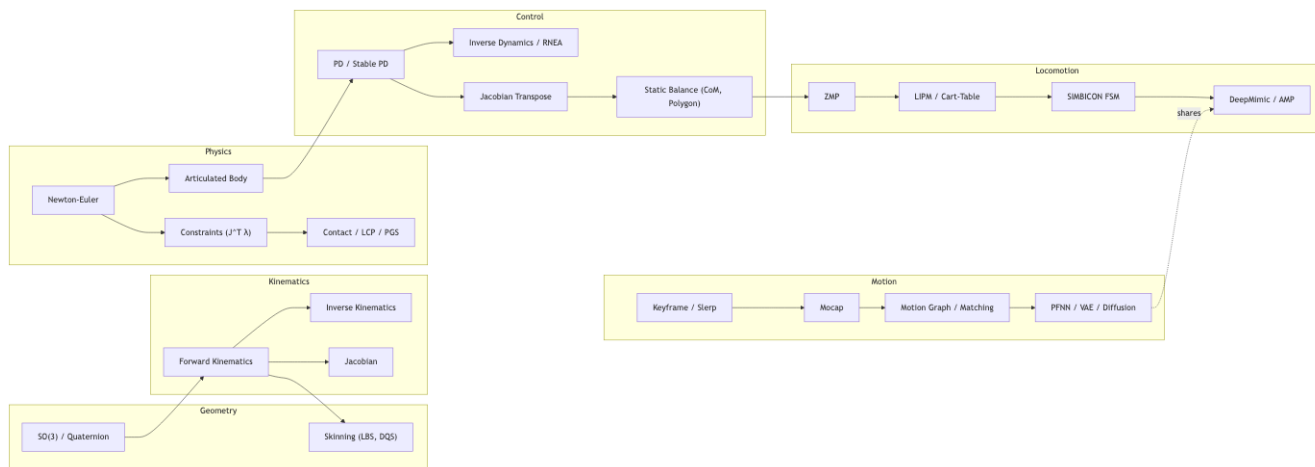
课程概览

北京大学《角色动画与运动仿真》(刘利斌, MOCCA 2026) 2025–2026 学年第 2 学期。课程从几何与运动学起步, 依次介绍关键帧动画、数据驱动方法、学习方法, 再到蒙皮与表情动画、基于物理的仿真, 再到角色驱动与走路控制, 最后统一到最优控制与强化学习框架 (Trajectory Optimization、Bellman/LQR、RL、生成式控制)。本笔记覆盖至今全部 11 讲。

课程脉络



知识图谱（全课程）



各讲速览

Lec	主题	关键概念	文件
01	Introduction	课程定位、动画产业链、计算机动画的研究问题	lec01_introduction.md
02	Math Background	旋转表示、 $SO(3)$ 、四元数、指数映射、 $\dot{R} = [\omega]_{\times} R$	lec02_math.md
03	Character Kinematics	骨骼层级、Forward / Inverse Kinematics、Jacobian、CCD、FABRIK	lec03_kinematics.md
04	Keyframe Animation	插值（线性/Hermite/Bezier/Catmull-Rom）、Slerp/Squad、动画曲线编辑	lec04_keyframe.md
05	Data-driven Animation	Motion capture、Motion graph、Motion matching、Motion blending	lec05_data_driven.md
06	Learning-based Animation	PFNN/MANN、Motion VAE/Diffusion、DeepMimic/AMP/ControlVAE	lec06_learning.md
07	Skinning & Facial Animation	Linear blend skinning、Dual quaternion、Blendshapes、FACS	lec07_skinning.md
08	Simulation	数值积分、Newton-Euler、铰接刚体、约束、Penalty/LCP 接触、PGS	lec08_simulation.md
09	Actuating Characters	极大/广义坐标、关节力矩、正/逆动力学、RNEA、PD/Stable PD、Jacobian Transpose、静态平衡	lec09_actuating.md
10	Learning to Walk	动态平衡、ZMP、LIPM、轨道能量、Cart-Table、Preview Control、SIMBICON	lec10_learning_to_walk.md
11	Optimal Control & RL	Trajectory Opt、Lagrange/KKT、Pontryagin、Bellman、LQR/iLQR、MDP、Q-Learning/DQN、Policy Gradient/PPO、Model-based RL、ControlVAE/VQ-VAE/MoConVQ	lec11_optimal_control_rl.md

全局术语中英对照

中文	English	首次出现
自由度	Degrees of Freedom (DoF)	Lec 03
前向运动学	Forward Kinematics (FK)	Lec 03
逆运动学	Inverse Kinematics (IK)	Lec 03
雅可比矩阵	Jacobian matrix	Lec 03
阻尼最小二乘	Damped Least Squares (DLS)	Lec 03
四元数	Quaternion	Lec 02
球面线性插值	Spherical Linear Interpolation (Slerp)	Lec 04
动作捕捉	Motion Capture (MoCap)	Lec 05
动作图	Motion Graph	Lec 05
动作匹配	Motion Matching	Lec 05
相位函数神经网络	Phase-Functioned Neural Network (PFNN)	Lec 06
强化学习	Reinforcement Learning (RL)	Lec 06
对抗动作先验	Adversarial Motion Prior (AMP)	Lec 06
线性混合蒙皮	Linear Blend Skinning (LBS)	Lec 07
双四元数蒙皮	Dual Quaternion Skinning (DQS)	Lec 07
形状混合	Blendshape	Lec 07
显式/隐式/半隐式 Euler	Explicit / Implicit / Semi-implicit Euler	Lec 08
角速度	Angular velocity	Lec 02/08
反对称矩阵	Skew-symmetric matrix	Lec 02/08
角动量	Angular momentum	Lec 08
惯量张量	Moment of inertia tensor	Lec 08
牛顿-欧拉方程	Newton-Euler equations	Lec 08
铰接刚体	Articulated rigid body	Lec 08
拉格朗日乘子	Lagrange multiplier	Lec 08
库仑摩擦律	Coulomb's law of friction	Lec 08
线性互补问题	Linear Complementary Problem (LCP)	Lec 08
摩擦锥	Friction cone	Lec 08
投影高斯-赛德尔法	Projected Gauss-Seidel	Lec 08
极大坐标	Maximized Coordinates	Lec 09
广义坐标	Generalized Coordinates	Lec 09
关节力矩	Joint Torque	Lec 09
正动力学	Forward Dynamics (FD)	Lec 09
逆动力学	Inverse Dynamics (ID)	Lec 09
递归牛顿欧拉算法	Recursive Newton-Euler Algorithm (RNEA)	Lec 09
比例-微分控制	Proportional-Derivative Control (PD)	Lec 09
比例-积分-微分控制	PID Control	Lec 09
稳定 PD 控制	Stable PD Control	Lec 09
重力补偿	Gravity Compensation	Lec 09

中文	English	首次出现
雅可比转置控制	Jacobian Transpose Control	Lec 09
虚拟力控制	Virtual Model Control / Virtual Force	Lec 09
残差力/力矩	Residual Force / Torque	Lec 09
欠驱动系统	Underactuated System	Lec 09
全驱动系统	Fully-actuated System	Lec 09
质心	Center of Mass (CoM)	Lec 09
支撑多边形	Support Polygon	Lec 09
踝/髌平衡策略	Ankle / Hip Strategy	Lec 09
动量控制	Momentum Control	Lec 09
单/双支撑相	Single / Double Stance Phase	Lec 10
飞行相	Flight Phase	Lec 10
零力矩点	Zero-Moment Point (ZMP)	Lec 10
压力中心	Center of Pressure (CoP)	Lec 10
地面反作用力	Ground Reaction Force (GRF)	Lec 10
倒立摆模型	Inverted Pendulum Model (IPM)	Lec 10
线性倒立摆模型	Linear Inverted Pendulum Model (LIPM)	Lec 10
弹簧负载倒立摆	Spring-Loaded Inverted Pendulum (SLIP)	Lec 10
轨道能量	Orbital Energy	Lec 10
桌-车模型	Cart-Table Model	Lec 10
预览控制	Preview Control	Lec 10
被动动态行走	Passive Dynamic Walking	Lec 10
SIMBICON	SIMple Blped LOCOMOTION CONTROL	Lec 10
有限状态机	Finite State Machine (FSM)	Lec 10
跨步策略	Stepping Strategy	Lec 10
轨迹优化	Trajectory Optimization	Lec 11
时空约束	Spacetime Constraints	Lec 11
开环/闭环控制	Open-loop / Closed-loop Control	Lec 11
前馈/反馈控制	Feedforward / Feedback Control	Lec 11
拉格朗日乘子	Lagrange Multiplier	Lec 11
KKT 条件	Karush-Kuhn-Tucker Conditions	Lec 11
协态/伴随变量	Co-state / Adjoint Variable	Lec 11
庞特里亚金极小值原理	Pontryagin's Minimum Principle (PMP)	Lec 11
打靶法	Shooting Method	Lec 11
无导数优化	Derivative-Free Optimization	Lec 11
协方差矩阵自适应进化策略	CMA-ES	Lec 11
基于采样的运动控制	SAMCON	Lec 11
动态规划	Dynamic Programming (DP)	Lec 11
贝尔曼最优性原理	Bellman's Principle of Optimality	Lec 11
值函数	Value Function	Lec 11
状态-动作值函数	Q-function / State-Action Value	Lec 11
线性二次调节器	Linear Quadratic Regulator (LQR)	Lec 11
黎卡提递推	Riccati Recursion	Lec 11
迭代 LQR / 微分动态规划	iLQR / DDP	Lec 11
马尔可夫决策过程	Markov Decision Process (MDP)	Lec 11
折扣因子	Discount Factor	Lec 11
回报	Return	Lec 11
时序差分学习	Temporal Difference (TD) Learning	Lec 11
Q 学习	Q-Learning	Lec 11
深度 Q 网络	Deep Q-Network (DQN)	Lec 11
经验回放	Replay Buffer	Lec 11

中文	English	首次出现
目标网络	Target Network	Lec 11
策略梯度	Policy Gradient	Lec 11
行动者-评论家	Actor-Critic	Lec 11
近端策略优化	Proximal Policy Optimization (PPO)	Lec 11
基于模型的强化学习	Model-Based Reinforcement Learning	Lec 11
变分自编码器	Variational Autoencoder (VAE)	Lec 11
向量量化	Vector Quantization (VQ-VAE)	Lec 11
码本	Codebook	Lec 11

Lecture 01 —Introduction to 3D Character Animation

[TIP] 学习指南

本节是导论：把”角色动画 (character animation)”放进 3D 计算机动画的整体地图里，澄清它要解决的问题（让有 20+ 关节、智能行为的角色动起来），并给出贯穿全课的方法学分类——**Kinematic (运动学) vs. Dynamic (基于物理)**。前置知识：线性代数、微积分、Python 编程、概率、力学；ML / RL 会用到，但课程基本自洽。需要从本节带走的概念：① Animation pipeline 的两段 (Modeling/Skinning + Skeletal Animation) ② 控制层级 high-level goals → low-level control → physics ③ 各家方法挂在这棵树的哪一支。

0. 课程定位与目标

不教什么：Maya / MotionBuilder / Houdini / Unity / Unreal 操作，也不教如何成为一名 animator。

教什么：动作背后的方法、理论与技术—— Kinematics of characters (FK / IK / 插值 / Motion Graphs / 数据驱动) – Physics-based simulation (刚体、关节、接触) – Motion control (tracking、轨迹优化、DRL、分层控制) – 让”机器狗 / 人形机器人”动起来的能力。

考核：Class participation 15% + Programming exercises 60% (FK/IK、动作数据、ragdoll 仿真、机器人控制) + Project 25%。禁止抄袭、禁止一稿多投。

1. 什么是 Character Animation

[NOTE] 定位

3D 计算机图形学关心 Geometry / Animation / Rendering；其中 **Animation** 又分：– **Simulation animation**：模拟自然现象 / phenomena——刚体、可变形体、绳、布、流体、烟、声……– **Character animation**：模拟生物 / 智能体的 behavior——人、动物、虚拟生物、手、机器人、人群……
关键差异：character 自带“+ Intelligence”，行为是被驱动的，不是被动演化的。

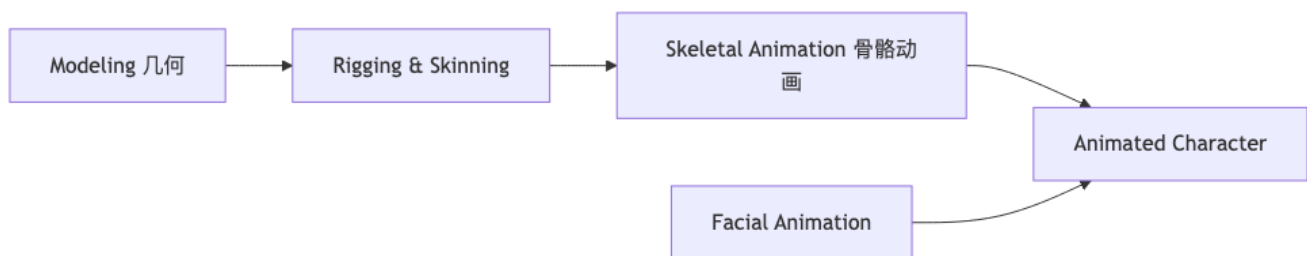
为什么值得专门研究？一个角色 20+ 关节、50–100+ 参数，并不算超高维，理论上手 K 帧就能做。但：– 手 K 劳动密集，不适合交互式应用；– 想要复用 / 编辑 / 生成新动作；– 想要迁移到新身体（数字分身、机器人）。

抽象地，character animation 就是构造一个 **Motion Model**：

$$\text{Motion Model} : (\text{current state}, \text{control signal}) \mapsto \text{next state}$$

control signal 可以是交互输入、音频/文本、抽象任务等。

2. Pipeline 与生物学动机



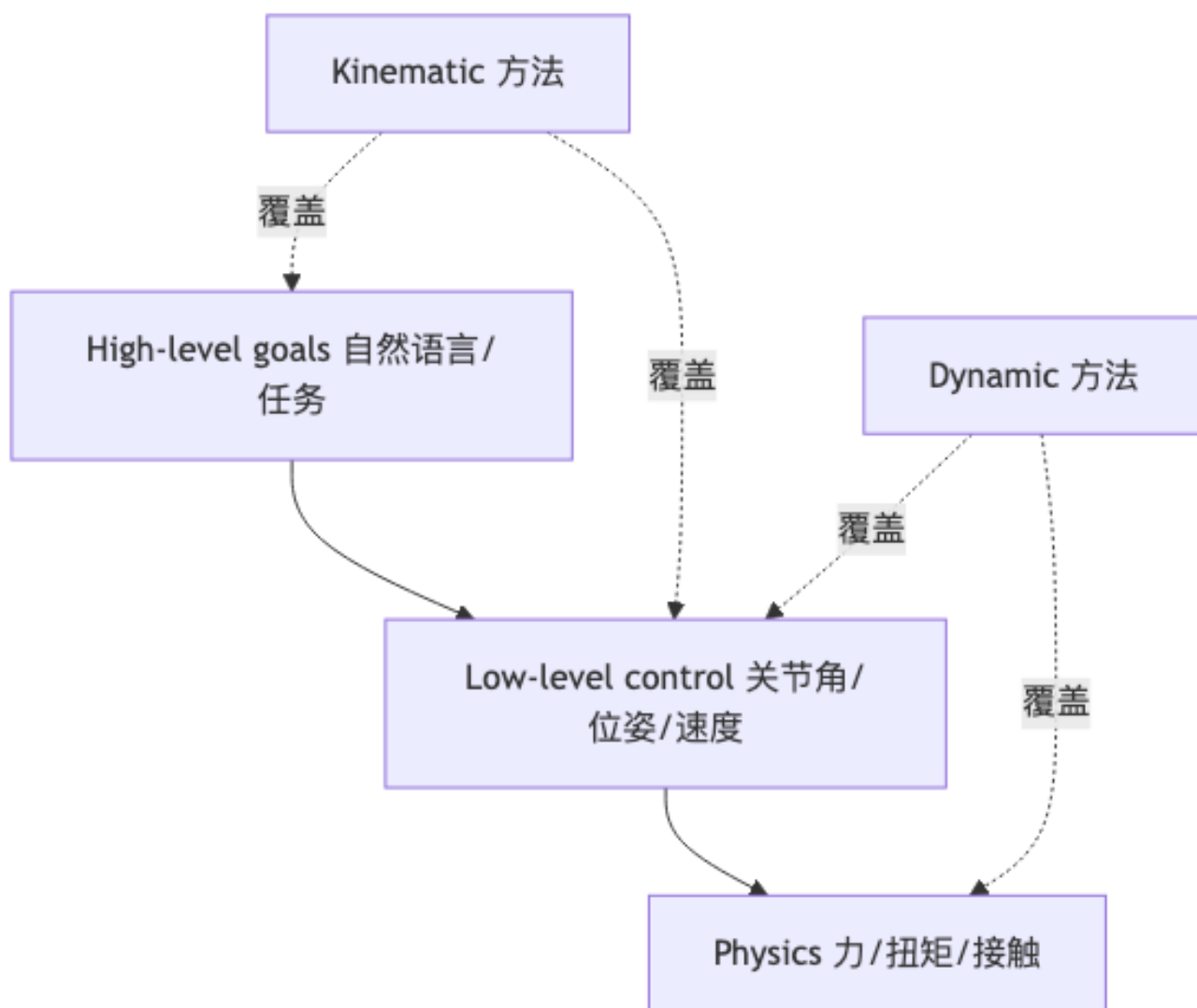
人为什么会动？生理链条：**Neuronal excitation** → **Muscle activation** → **Forces/torques on musculoskeletal system** → **Physics** → **Body pose**。课程里的两大方法学正是从这条链上“切一刀”：

- **Kinematic 方法**：直接更新 pose / velocity / acceleration，跳过力与肌肉。
- **Dynamic 方法**：保留 Physics 这一段，控制器输出 forces/torques 作用到（简化的）骨架上。

3. 方法学坐标系：high-level goals low-level control physics

[NOTE] 直觉理解

把所有方法摆到同一根坐标轴上：靠近 physics 的方法控制力/扭矩；靠近 high-level goals 的方法接受“go there and bring me a glass of water”这种语义命令；中间是 low-level control (pose、轨迹、关节角)。Kinematic 方法主要覆盖 high-level → low-level 段，Dynamic 方法把 low-level 接到 physics。



4. Kinematic 方法谱系

按出现顺序自下而上堆叠：

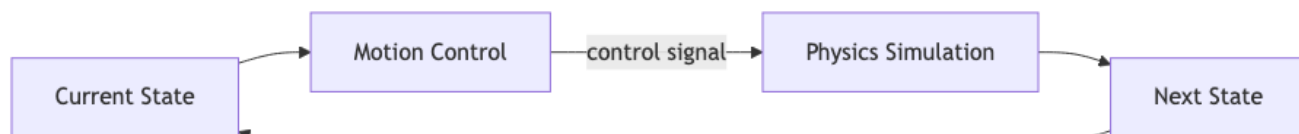
1. **Keyframe Animation**: 动画师摆 pose, 工具做 in-between。Disney 12 principles、Rotoscoping (Max Fleischer, ~1914) 是其源头。
2. **Forward / Inverse Kinematics**:
 - FK: 给关节旋转 → 求末端位置。
 - IK: 给末端位置 → 求关节旋转 (多解、欠定)。
3. **Interpolation / In-betweening / In-painting**: 在 keyframe 之间补帧 (如 RSMT 2023)。
4. **Motion Capture**: 光学/IMU/视频姿态估计 (OpenPose 等) 采集真人动作。
5. **Motion Retargeting**: 把源角色动作迁到不同骨骼大小、骨数、拓扑的目标角色。
6. **Motion Graphs / State Machines**: 把动作片段串成图/状态机, 按转移条件播放; Parametric motion graphs 可参数化插值。复杂工程里容易变成“spaghetti”。
7. **Motion Matching**: 用 query feature 在动作库里检索最近邻继续播放 (GDC 2016 起流行)。
8. **Generative Models (Learning-based)**: MLP / VAE / Diffusion 等学习 $(s_t, \text{ctrl}) \mapsto s_{t+1}$; 代表: Local Motion Phases、Motion VAE、MotionDiffuse。
9. **Cross-Modal Synthesis**: 音频 → 舞蹈/手势、文本 → 动作 (MotionDiffuse、Rhythmic Gesticulator)。
10. **LLM 驱动的高层规划**: 把“开门 → 坐下”分解成一串带索引的动作片段。

[WARN] Kinematic 方法的两大软肋

- **Physical plausibility**: 脚滑、穿模、惯性不对——因为根本没解物理。
- **Environment interaction**: 碰撞、推拉、被动反应难以用纯数据生成。这正是 Physics-based 方法的入口。

5. Physics-based / Dynamic 方法谱系

核心结构：在每一步把 motion control 输出的控制信号交给物理仿真器，让 physics 决定下一帧。



最简单的极端是 **Ragdoll Simulation**: 完全无控制，只有重力 + 关节 + 接触，角色”瘫”下去。要让它”动”得像活物，需要一层 controller:

1. **Tracking Controllers** (Hodgins & Wooten 1995): 让仿真角色跟随参考动作 (PD 控制 + 平衡修正)。
2. **“Keyframe Control” / Trajectory Crafting**: 动画师手 K 力/扭矩轨迹 (NaturalMotion Endorphin)。
3. **Spacetime / Trajectory Optimization**: 把轨迹 + 控制写成大优化问题 (SAMCON 2010、Wampler & Popovic 2009)。
4. **Abstract Models**: 用 Inverted Pendulum、SIMBICON (FSM + linear feedback) 等简化模型解释步态; 现代人形机器人 (Unitree H1 等) 仍在用。
5. **Reinforcement Learning / Deep RL**: 环境-状态-动作-奖励循环, 神经网络做策略。DeepMimic (Peng 2018) 是分水岭——把参考动作纳入 reward, 可学会高度风格化的 athletic motion。
6. **Hierarchical Controllers**: 一层 tracking controllers + 上层调度器 (Liu 2017 Schedule Control Fragments), 扩展到多技能角色。
7. **Generative Control Policies**: 把”生成模型”和”物理仿真”组合——生成模型给出下一个 pose 作为 tracking 目标, 仿真保证物理可行。
8. + **LLM**: 高层语义任务由语言模型分解, 落到底层 controller。

[NOTE] 力 / 力矩 (基本式)

多个力 F_i 作用于位置 r_i : 合力 $F = \sum_i F_i$, 对参考点的合力矩

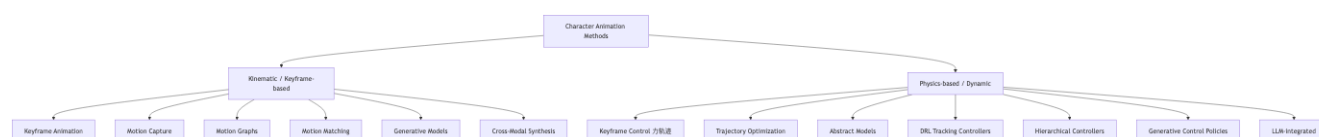
$$\tau = \sum_i r_i \times F_i$$

这是 Lec 07-08 真正展开 Newton-Euler 的起点。

[WARN] 控制为什么难 (QWOP intuition)

仿真双足角色就像玩 QWOP——状态高维、接触不连续、误差指数级累积。kinematic 方法不会摔倒, dynamic 方法默认就会摔倒; 课程后半段都在回答”如何不摔”。

6. 全图：把谱系并排



7. 课程时间线（讲次概览）

Lec	主题
02	Rotation, Transformation, Forward Kinematics
03	Inverse Kinematics
04	Keyframe Character Animation
05	Data-driven / Learning-based Animation
06	Skinning & Facial Animation
07	Physics-based Simulation & Articulated Rigid Bodies
08	Actuating Character & Feedback Control
09	Learning to Walk with Simplified Models
10	Optimal Control & Trajectory Optimization
11	Reinforcement Learning & Multi-skilled Characters
12	Human Simulation (optional)
13	Crowd Simulation (optional)

练习：FK/IK → 动作数据 → 仿真 +ragdoll → “让你的机器人动起来”→ 交互式仿真 character project。

Lecture 02 —数学基础：旋转的四种表示

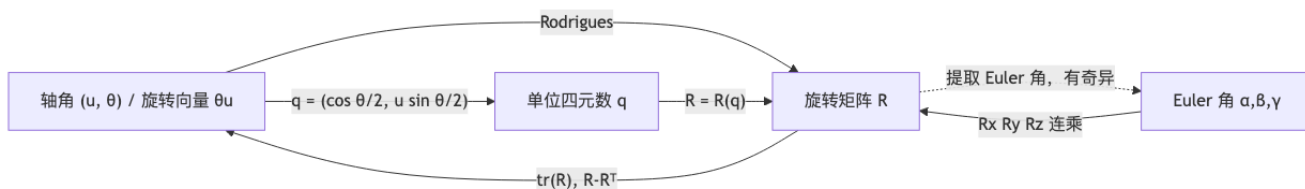
[TIP] 学习指南

本节是后续所有讲的“工具箱”，必须把每条公式都“用得动”而不是“看得懂”。核心问题：3D 旋转怎么写、怎么算、怎么合成、怎么插值？四种表示——**旋转矩阵 / Euler 角 / 轴角 (rotation vector) / 四元数**——各自的优缺点是什么？重点关注：① $\dot{R} = [\omega]_{\times} R$ 来自 $RR^T = I$ ；② Rodrigues 公式的几何推导；③ 四元数乘法与 $p' = qpq^*$ ；④ SLERP 的推导；⑤ Gimbal Lock 与“插值不是恒速”等坑。

0. 大纲与动机 / Motivation

角色动画的“姿态”本质是 $SE(3)$ 元素：平移 $\mathbf{t} \in \mathbb{R}^3$ + 旋转 $R \in SO(3)$ 。平移是线性空间，简单；**旋转所在的流形非线性、非交换**，必须挑表示。本讲依次回答：

1. 向量、矩阵、点积、叉积的代数 / 几何含义；
2. 怎样把“绕轴 $\mathbf{u}$ 转 θ ”变成显式公式 (**Rodrigues**)；
3. 四种旋转表示的等价、相互转换、各自的“易做 / 难做”；
4. 旋转怎么插值，为什么线性插旋转矩阵会爆。



1. 线性代数复习

1.1 向量与运算

向量 $\mathbf{a} = (a_x, a_y, a_z)^T \in \mathbb{R}^3$ ；模 $\|\mathbf{a}\|_2 = \sqrt{a_x^2 + a_y^2 + a_z^2}$ ；单位化 $\hat{\mathbf{a}} = \mathbf{a}/\|\mathbf{a}\|$ 。加减 / 数乘逐分量。

1.2 点积 (Dot Product)

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta = \mathbf{a}^T \mathbf{b}$$

- 用途 1 (夹角): $\theta = \arccos \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$
- 用途 2 (投影): $\mathbf{b}$ 在 $\mathbf{a}$ 方向的标量分量 $b_a = \frac{\mathbf{b} \cdot \mathbf{a}}{\|\mathbf{a}\|}$
- 用途 3 (垂直判定): $\mathbf{a} \cdot \mathbf{b} = 0 \Leftrightarrow \mathbf{a} \perp \mathbf{b}$

1.3 叉积 (Cross Product)

$$\mathbf{c} = \mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix} = \|\mathbf{a}\| \|\mathbf{b}\| \sin \theta \mathbf{n}$$

记忆口诀: $x:yz, y:zx, z:xy$ 。性质:

- $\mathbf{c} \perp \mathbf{a}, \mathbf{c} \perp \mathbf{b}$;
- 反交换 $\mathbf{a} \times \mathbf{b} = -\mathbf{b} \times \mathbf{a}$, 分配 $\mathbf{a} \times (\mathbf{b} + \mathbf{d}) = \mathbf{a} \times \mathbf{b} + \mathbf{a} \times \mathbf{d}$;
- 不结合: $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) \neq (\mathbf{a} \times \mathbf{b}) \times \mathbf{c}$;
- 平行判定: $\mathbf{a} \times \mathbf{b} = \mathbf{0} \Leftrightarrow \mathbf{a} \parallel \mathbf{b}$ 。

[NOTE] 叉积的“两向量间最小旋转”

对非平行 $\mathbf{a}, \mathbf{b}$, 把 $\mathbf{a}$ 旋到 $\mathbf{b}$ 的最小旋转: 轴 $\mathbf{u} = \frac{\mathbf{a} \times \mathbf{b}}{\|\mathbf{a} \times \mathbf{b}\|}$, 角 $\theta = \arccos \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$ 。

1.4 叉积的矩阵形式: 反对称矩阵

$$\mathbf{a} \times \mathbf{b} = [\mathbf{a}]_{\times} \mathbf{b}, \quad [\mathbf{a}]_{\times} = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix}$$

满足 $[\mathbf{a}]_{\times}^T = -[\mathbf{a}]_{\times}$ (反对称 / skew-symmetric)。常用恒等式:

$$[\mathbf{a}]_{\times} [\mathbf{a}]_{\times} \mathbf{b} = \mathbf{a} \times (\mathbf{a} \times \mathbf{b}), \quad [\mathbf{a}]_{\times} \mathbf{b} = -[\mathbf{b}]_{\times} \mathbf{a}$$

[WARN] 叉积没有结合律, 但写成矩阵后矩阵乘法有结合律——所以 $([\mathbf{a}]_{\times} [\mathbf{b}]_{\times} [\mathbf{c}]) = ([\mathbf{a}]_{\times} [\mathbf{b}]_{\times}) [\mathbf{c}]$, 但不等于 $[\mathbf{a}]_{\times} ([\mathbf{b}]_{\times} [\mathbf{c}])$ 的另一种括号方式 $([\mathbf{a}]_{\times} [\mathbf{b}]_{\times}) [\mathbf{c}]$ 。它们是 BAC-CAB 恒等式相差一个 $-(\mathbf{a} \cdot \mathbf{b}) \mathbf{c} + (\mathbf{b} \cdot \mathbf{c}) \mathbf{a}$ 。

1.5 矩阵基础

特殊矩阵: 对角、单位 I 、对称 $A^T = A$ 、反对称 $A^T = -A$ 。乘法 $AB \neq BA$, 但有结合律和分配律; $(AB)^T = B^T A^T$, $(AB)^{-1} = B^{-1} A^{-1}$ 。

正交矩阵: 列(行)相互单位正交 $\Leftrightarrow A^T A = A A^T = I \Leftrightarrow A^{-1} = A^T$ 。 $\det A = \pm 1$ 。

特征值/特征向量: $A\mathbf{x} = \lambda\mathbf{x}$ 。一个 3×3 正交阵至少有一个实特征值 $\lambda = \det A = \pm 1$ ——这是后面“旋转一定有不动轴”的根据。

2. 刚体变换: 缩放、平移、旋转

整体 $\mathbf{a}' = R\mathbf{a} + \mathbf{t}$ (暂不讨论缩放)。平移可叠加 ($\mathbf{t} = \mathbf{t}' + \mathbf{t}''$, 可交换); 旋转不交换 $R_2 R_1 \neq R_1 R_2$ 。

2.1 旋转矩阵的性质

$$R^T R = I, \det R = +1, \|R\mathbf{x}\| = \|\mathbf{x}\|$$

- 保长度（等距变换）；
- $\det = +1$ 而非 -1 ，排除反射；
- 集合 $SO(3) = \{R \in \mathbb{R}^{3 \times 3} : R^T R = I, \det R = 1\}$ 是 3 维李群。

2.2 三个基本旋转（绕坐标轴）

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix}, R_y(\beta) = \begin{pmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{pmatrix}, R_z(\gamma) = \begin{pmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

2.3 坐标变换 (object global)

设物体局部基 $\mathbf{e}'_x, \mathbf{e}'_y, \mathbf{e}'_z$ 在世界系下排成矩阵 $R = [\mathbf{e}'_x | \mathbf{e}'_y | \mathbf{e}'_z]$ ，物体上一点局部坐标 (x', y', z') ，世界坐标 (x, y, z) ：

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = R \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} + \mathbf{t}, \quad \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = R^T \left(\begin{pmatrix} x \\ y \\ z \end{pmatrix} - \mathbf{t} \right)$$

[NOTE] 一句话记

“世界 = R · 局部 + t”，反向用 R^T 。后续讲刚体仿真、骨骼蒙皮、IK 全部基于此。

3. Rodrigues 公式：从轴角到旋转矩阵

3.1 几何推导

设单位轴 $\mathbf{u}$ 、待转向量 $\mathbf{a}$ 、目标 $\mathbf{b}$ 。把 $\mathbf{b} - \mathbf{a}$ 拆成两个方向：

- 沿 $\mathbf{u} \times \mathbf{a}$ 的分量 $\mathbf{v}$ （与 $\mathbf{u}$ 垂直、与 $\mathbf{a}$ 在 $\mathbf{u} \perp$ 平面内的圆周切向）；
- 沿 $\mathbf{u} \times (\mathbf{u} \times \mathbf{a})$ 的分量 $\mathbf{t}$ （指向圆心）。

由几何观察：

$$\mathbf{v} = \sin \theta (\mathbf{u} \times \mathbf{a}), \quad \mathbf{t} = (1 - \cos \theta) \mathbf{u} \times (\mathbf{u} \times \mathbf{a})$$

故

$$\mathbf{b} = \mathbf{a} + \sin \theta (\mathbf{u} \times \mathbf{a}) + (1 - \cos \theta) \mathbf{u} \times (\mathbf{u} \times \mathbf{a})$$

3.2 写成矩阵形式

把 $\mathbf{u} \times = [\mathbf{u}]_{\times}$ 提出： $\mathbf{b} = (I + \sin \theta [\mathbf{u}]_{\times} + (1 - \cos \theta) [\mathbf{u}]_{\times}^2) \mathbf{a}$ ，于是：

[NOTE] Rodrigues' rotation formula

$$R(\mathbf{u}, \theta) = I + \sin \theta [\mathbf{u}]_{\times} + (1 - \cos \theta) [\mathbf{u}]_{\times}^2$$

把“绕 $\mathbf{u}$ 转 θ ”显式写成 3×3 矩阵的关键公式，后续 $\dot{R} = [\omega]_{\times} R$ 、四元数 矩阵转换、指数映射 $\exp([\omega]_{\times})$ 都建立在此之上。

3.3 反向：从 R 提取 $(\mathbf{u}, \theta)$

- 角度：迹公式 $\text{tr}(R) = 1 + 2 \cos \theta$ ，故 $\theta = \arccos \frac{\text{tr}(R)-1}{2}$ 。
- 轴： $R\mathbf{u} = \mathbf{u}$ (特征值 1 的特征向量)。等价地 $(R - R^T)\mathbf{u} = 0$ ，注意 $R - R^T$ 是反对称：

$$R - R^T = \begin{pmatrix} 0 & r_{12} - r_{21} & r_{13} - r_{31} \\ \cdot & 0 & r_{23} - r_{32} \\ \cdot & \cdot & 0 \end{pmatrix}^*$$

直接读出 $\mathbf{u}' = (r_{32} - r_{23}, r_{13} - r_{31}, r_{21} - r_{12})^T$ ，且 $\|\mathbf{u}'\| = 2 \sin \theta$ 。归一化后即得轴。

[WARN] 退化情形

$\theta = 0$: $R = I$, 轴任意; $\theta = \pi$: $\sin \theta = 0$, 上述方法失效, 此时 $R + R^T = 2(I + 2\mathbf{u}\mathbf{u}^T - 2I) = 2(2\mathbf{u}\mathbf{u}^T - I)$, 从对角元抽根号取轴。

4. 旋转表示 1: 旋转矩阵

操作	实现	难易
应用到向量	一次 3×3 矩阵乘 $\mathbf{b} = R\mathbf{a}$	容易
合成	矩阵乘 $R = R_2 R_1$	容易
求逆	转置 $R^{-1} = R^T$	容易
参数化	9 个数 + 6 个正交约束 $\rightarrow \text{DoF} = 3$	不直观
插值	$(1-t)R_0 + tR_1$ 不再是旋转	不能直接插

[WARN] 旋转矩阵插值的反例

$R_0 = R_y(-90^\circ)$, $R_1 = R_y(+90^\circ)$, $0.5(R_0 + R_1) = \text{diag}(0, 1, 0)$ ——零行列式，不是旋转。说明 $SO(3)$ 在 $\mathbb{R}^9$ 中是弯曲流形。

5. 旋转表示 2: Euler 角

任意旋转 = 三个绕坐标轴的基本旋转的乘积。例： $R = R_z(\gamma)R_y(\beta)R_x(\alpha)$ 。

- 12 种顺序 (XYZ、XZY、...ZYX)：6 种 Tait-Bryan (三个轴互不重复) + 6 种 proper Euler (首尾轴重合)。
- **Intrinsic vs Extrinsic**：绕物体局部轴 / 绕世界固定轴。代数关系： $R_x R_y R_z$ (外旋顺序 $X \rightarrow Y \rightarrow Z$) = $R_z R_y R_x$ (内旋顺序 $Z \rightarrow Y \rightarrow X$, 反序)。

[WARN] Gimbal Lock (万向锁)

当中间轴的角度让另两个轴对齐时，3 个 DoF 退化为 2 个——例如 ZYX 顺序中 $\beta = \pm 90^\circ$ 时 $R_z(\gamma)R_y(\pm 90^\circ)R_x(\alpha)$ 变成只依赖 γ 和 α 一个参数。后果：连续插值时角速度不恒定、可能出现“猛抽一下”的奇异。

优缺点	说明
直观	3 个角，DoF 等于参数
难合成	需要先转矩阵再乘
难指定旋转	给定目标姿态难直接写出三个角
奇异	Gimbal Lock
插值非恒速	角分量线性插不等价于姿态匀速变

6. 旋转表示 3：旋转向量 / 轴角

Axis-Angle: $(\mathbf{u}, \theta)$, $\|\mathbf{u}\| = 1$ 。Rotation Vector (紧凑形式): $\theta = \theta\mathbf{u} \in \mathbb{R}^3$, $\theta = \|\theta\|$, $\mathbf{u} = \theta/\|\theta\|$ 。

- 应用：通过 Rodrigues 转矩阵后再乘向量。
- 合成：没有简单公式（一般不能 $\theta_1 + \theta_2$ ）。
- 插值：
 1. 直接线性: $\theta_t = (1-t)\theta_0 + t\theta_1$ ——值合法，但角速度非恒定。
 2. 偏移旋转法 (更好): $R_{\delta\theta} = R^T(\theta_0)R(\theta_1)$ ，提取其旋转向量 $\delta\theta$ ，再 $\delta\theta_t = t\delta\theta$ ，最后 $R(\theta_t) = R(\theta_0)R(\delta\theta_t)$ 。这相当于“球面”上的最短路径插值。

[WARN] 表示不唯一

$(\mathbf{u}, \theta) \sim (-\mathbf{u}, -\theta) \sim (\mathbf{u}, \theta + 2n\pi)$, $\theta = 0$ 时轴无定义。求逆变换、查询时要小心做归一化（一般约定 $\theta \in [0, \pi]$ ）。

7. 旋转表示 4：四元数

7.1 从复数到四元数

2D 旋转可写成单位复数: $z = e^{i\theta} = \cos \theta + i \sin \theta$ ，旋转 $z' = e^{i\alpha}z$ 。Hamilton 把这个推广到 3D: 引入两个新虚单位 $\mathbf{j}, \mathbf{k}$,

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1, \quad \mathbf{ij} = \mathbf{k}, \mathbf{jk} = \mathbf{i}, \mathbf{ki} = \mathbf{j}, \mathbf{ji} = -\mathbf{k}, \dots$$

注意虚单位间的乘法像叉积。一般四元数: $\mathbf{q} = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k} = (w, \mathbf{v})^T \in \mathbb{H}$, w 为实部, $\mathbf{v} = (x, y, z)^T$ 为虚部。

7.2 四元数代数

逐分量：加法、数乘、点积、模长一致。

$$\mathbf{q}_1 + \mathbf{q}_2 = (w_1 + w_2, \mathbf{v}_1 + \mathbf{v}_2), \quad \mathbf{q}_1 \cdot \mathbf{q}_2 = w_1 w_2 + \mathbf{v}_1 \cdot \mathbf{v}_2, \quad \|\mathbf{q}\| = \sqrt{\mathbf{q} \cdot \mathbf{q}}$$

共轭: $q^* = (w, -\mathbf{v})$ 。乘法 (关键): 把 $q_1 q_2$ 按虚单位规则展开整理得

$$q_1 q_2 = \left(w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2 \right)$$

[NOTE] 推导草图

用分配律展开 $(w_1 + \mathbf{v}_1)(w_2 + \mathbf{v}_2)$, 标量 $\times$ 标量 $= w_1 w_2$; 标量 $\times$ 矢量 $= w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1$; 矢量 $\times$ 矢量 $= \mathbf{v}_1 \mathbf{v}_2 = -\mathbf{v}_1 \cdot \mathbf{v}_2 + \mathbf{v}_1 \times \mathbf{v}_2$ (用 $\mathbf{i}\mathbf{j} = \mathbf{k}$ 等导出)。合并即得。

性质: $q_1 q_2 \neq q_2 q_1$ (非交换), 但有结合律; $(q_1 q_2)^* = q_2^* q_1^*$; $\|q\|^2 = q q^* = q^* q$; $q^{-1} = q^* / \|q\|^2$ 。

7.3 单位四元数与旋转

单位四元数 $\|q\| = 1$: $q^{-1} = q^*$ 。集合是 4D 单位超球 S^3 。约定:

$$q = \left(\cos \frac{\theta}{2}, \mathbf{u} \sin \frac{\theta}{2} \right), \quad \|\mathbf{u}\| = 1$$

它编码”绕 $\mathbf{u}$ 转 θ “的同样信息, 但形式不同。从 q 反推: $\theta = 2 \arccos w$, $\mathbf{u} = \mathbf{v} / \|\mathbf{v}\|$ 。

7.4 用四元数旋转向量

把 3D 向量 $\mathbf{p}$ 嵌入为纯四元数 $(0, \mathbf{p})$ 。旋转公式:

$$(0, \mathbf{p}') = q (0, \mathbf{p}) q^*$$

[NOTE] 为什么是”夹心”而不是 $\mathbf{q} \mathbf{p} \mathbf{q}$?

直接 $q p q^{-1}$ 是 4D 旋转的”双覆盖”事实: 单位四元数群 S^3 是 $SO(3)$ 的双重覆盖 $\Rightarrow q$ 与 $-q$ 表示同一个 3D 旋转。代入纯四元数验证: $q(0, \mathbf{p})q^*$ 的标量部仍是 0, 矢量部正好是 Rodrigues 公式给出的 $\mathbf{p}'$ 。

合成: 先 q_1 后 q_2 $q = q_2 q_1$ (注意顺序)。证明:

$$(0, \mathbf{p}'') = q_2 [q_1 (0, \mathbf{p}) q_1^*] q_2^* = (q_2 q_1) (0, \mathbf{p}) (q_2 q_1)^*$$

7.5 四元数插值

朴素 LERP: $\tilde{q}_t = (1-t)q_0 + tq_1$ 不再是单位四元数 $\rightarrow$ 投影回球面 $q_t = \tilde{q}_t / \|\tilde{q}_t\|$ 。问题: 角速度不恒定 (弦不是弧)。

SLERP — Spherical Linear Interpolation: 在大圆弧上等弧长插值。

$$q_t = \frac{\sin[(1-t)\theta]}{\sin \theta} q_0 + \frac{\sin[t\theta]}{\sin \theta} q_1, \quad \cos \theta = q_0 \cdot q_1$$

[NOTE] SLERP 推导

设 $\mathbf{q}_t = a(t)\mathbf{q}_0 + b(t)\mathbf{q}_1$ 。两侧分别点乘 $\mathbf{q}_0, \mathbf{q}_1$ ，并要求 $\mathbf{q}_t$ 与 $\mathbf{q}_0$ 夹角 $t\theta$ 、与 $\mathbf{q}_1$ 夹角 $(1-t)\theta$ ：

$$\cos(t\theta) = a + b \cos \theta, \quad \cos((1-t)\theta) = a \cos \theta + b$$

解出 $a = \frac{\sin[(1-t)\theta]}{\sin \theta}$, $b = \frac{\sin t\theta}{\sin \theta}$ 。 $t = 0, 1$ 时退化为 $\mathbf{q}_0, \mathbf{q}_1$ ； $\theta \rightarrow 0$ 时退化为 LERP (用 $\sin \theta \rightarrow \theta$)。

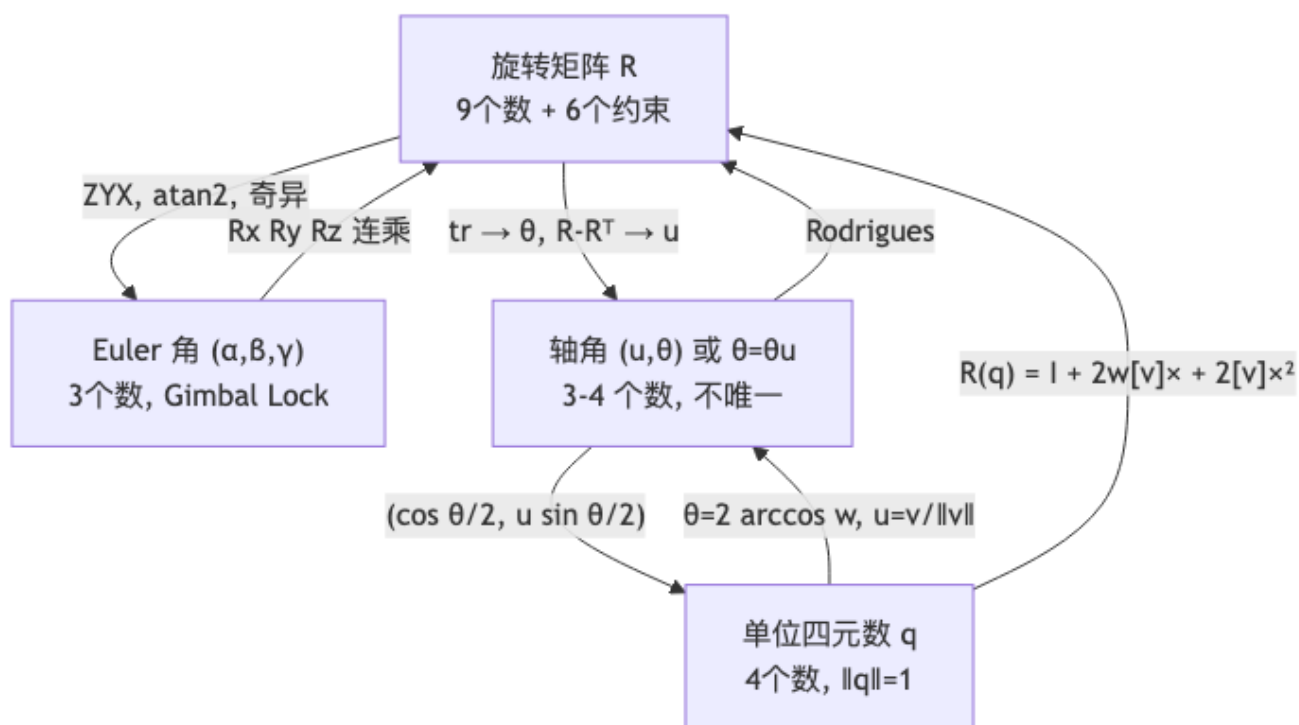
[WARN] 双覆盖陷阱

因 $\mathbf{q}, -\mathbf{q}$ 表示同一旋转，做 SLERP 前需要保证 $\mathbf{q}_0 \cdot \mathbf{q}_1 \geq 0$ ；若小于 0，把 $\mathbf{q}_1$ 翻号 $\mathbf{q}_1 \leftarrow -\mathbf{q}_1$ ，否则会沿”长弧”绕远路。

7.6 四元数评分

优缺点	说明
紧凑	4 个数 (vs 矩阵 9 个)
易合成	一次四元数乘 (16 次乘 + 12 次加)
易插值	SLERP 球面恒速
无 Gimbal Lock	S^3 没有全局奇异
数值漂移	积分后需要归一化 $\mathbf{q} \leftarrow \mathbf{q}/\ \mathbf{q}\ $
难直接编辑	4 个数无直观语义
双覆盖	$\mathbf{q}, -\mathbf{q}$ 同义

8. 四种表示对比与转换



维度	矩阵	Euler	轴角	四元数
数	9	3	3 (or 4)	4
应用 $\mathbf{p} \mapsto R\mathbf{p}$	$R\mathbf{p}$ 直接	转矩阵	转矩阵	$\mathbf{q}\mathbf{p}\mathbf{q}^*$
合成	R_2R_1	转矩阵后乘	一般无	$\mathbf{q}_2\mathbf{q}_1$
插值	不能	可, 但锁/不恒速	LERP 不恒速; 可借助偏移	LERP 投影 / SLERP
奇异引擎里常用	无内部计算 / 蒙皮	Gimbal Lock 美术 UI	$\theta = 0$ 轴未定 IK 增量	无 (双覆盖) 存储 / 网络 / 插值

9. 其他表示 (一笔带过)

6D 表示: 取旋转矩阵的前两列 $(\mathbf{r}_x, \mathbf{r}_y)$, $\mathbf{r}_z = \mathbf{r}_x \times \mathbf{r}_y$ 由叉积补回。优点: 连续光滑、神经网络回归比四元数稳定 (Zhou et al. CVPR 2019)。后续机器学习章节会用到。

记号速查

符号	含义
$\mathbf{a}, \mathbf{b}, \mathbf{u}, \mathbf{p}$	3D 向量; $\mathbf{u}$ 一般为单位轴
$\hat{\mathbf{a}}$	$\mathbf{a}$ 的单位化
$\mathbf{a} \cdot \mathbf{b}$	点积 $= \mathbf{a}^T \mathbf{b} = \ \mathbf{a}\ \ \mathbf{b}\ \cos \theta$
$\mathbf{a} \times \mathbf{b}$	叉积 $= [\mathbf{a}]_{\times} \mathbf{b} = \ \mathbf{a}\ \ \mathbf{b}\ \sin \theta \mathbf{n}$
$[\mathbf{a}]_{\times}$	反对称矩阵; $[\mathbf{a}]_{\times}^T = -[\mathbf{a}]_{\times}$
$R \in SO(3)$	旋转矩阵; $R^T R = I, \det R = +1$
$\mathbf{t}$	平移向量
$(\mathbf{u}, \theta)$	轴角; $\theta = \theta \mathbf{u}$ 旋转向量
$R(\mathbf{u}, \theta)$	Rodrigues: $I + \sin \theta [\mathbf{u}]_{\times} + (1 - \cos \theta) [\mathbf{u}]_{\times}^2$
α, β, γ	Euler 角, 绕 x, y, z
R_x, R_y, R_z	绕坐标轴的基本旋转
$\mathbf{q} = (w, \mathbf{v}) \in \mathbb{H}$	四元数; $\mathbf{v} = (x, y, z)$ 为虚部
$\mathbf{q}^*$	共轭 $(w, -\mathbf{v})$; 单位四元数下 $\mathbf{q}^{-1} = \mathbf{q}^*$
$\mathbf{q}_1 \mathbf{q}_2$	四元数乘
$\mathbf{q}\mathbf{p}\mathbf{q}^*$	单位四元数旋转向量 $\mathbf{p}$
SLERP	球面线性插值 $\frac{\sin((1-t)\theta)}{\sin \theta} \mathbf{q}_0 + \frac{\sin t\theta}{\sin \theta} \mathbf{q}_1$
$\text{tr}(R)$	矩阵迹; 用于提角 $\theta = \arccos \frac{\text{tr} R - 1}{2}$

Lecture 03 —Character Kinematics

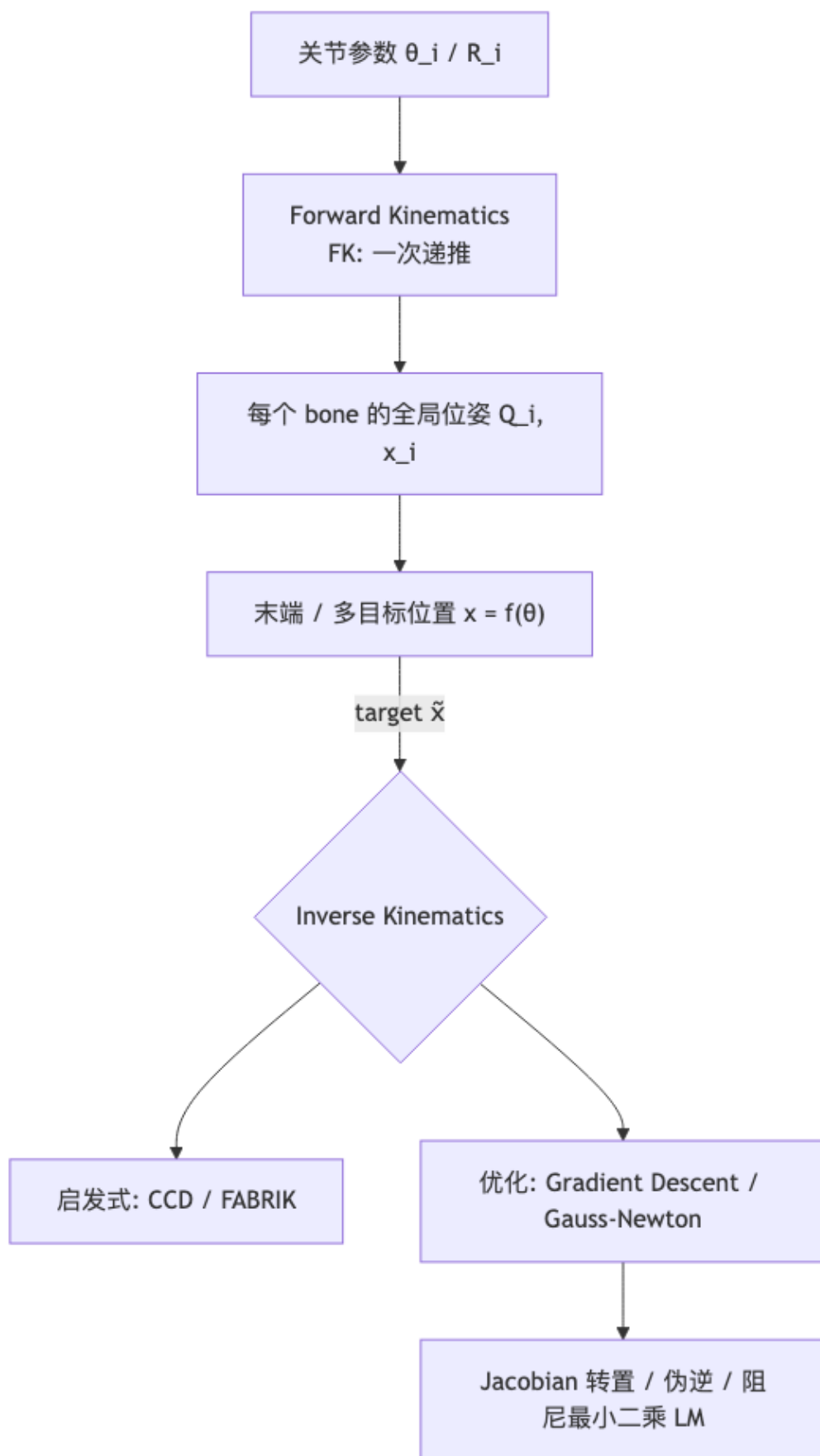
[TIP] 学习指南

本节核心：用关节角参数化角色姿态，正向求末端位置 (FK)，反向解关节角 (IK)。前置知识：旋转矩阵 / 四元数 (Lec 02)、链式法则与梯度。重点关注：① FK 的递推公式 $Q_i = Q_{p_i} R_i$, $x_i = x_{p_i} + Q_{p_i} l_i$; ② Reference pose 之间的 retargeting 公式；③ CCD / FABRIK 启发式；④ 把 IK 写成优化、计算 Jacobian、写出伪逆 / 阻尼最小二乘的更新；⑤ 全身 IK 中根关节的特殊处理。

0. 角色运动学全景 / Why Kinematics

kinematics —the study of the motion of bodies without reference to mass or force (Collins)

角色 (character) 由骨骼 (skeleton) 驱动：一组刚性 link / bone / body 通过关节 (joint) 相连。运动学只关心几何——给关节角，得到形状；它不关心力或质量（那是 Lec 08 物理仿真的事）。



本讲分四块：(1) Skeleton & FK；(2) Retargeting；(3) 启发式 IK (CCD, FABRIK)；(4) 优化 IK (Jacobian 方法)。

1. 骨骼与单链 FK / Skeleton & Chain Forward Kinematics

1.1 概念

- **Joint (关节)**: 定义 link 之间的相对旋转中心。
- **Link / bone / body**: 刚体段。
- **Root joint**: 链/树的根, 世界系下的位置 t_0 与朝向 R_0 唯一确定整体位姿。
- **End-effector (末端)**: 手、脚、头等关心位置/朝向的关节。

1.2 单链上的旋转: local 与 global

每个关节 i 有两个量: $-R_i$: 局部旋转 (joint rotation), 相对父关节坐标系; $-Q_i$: 全局朝向 (orientation), bone i 的世界姿态。

关键递推: 从 root 沿链向下,

$$Q_i = Q_{i-1} R_i, \quad Q_0 = R_0$$

展开: $Q_4 = R_0 R_1 R_2 R_3 R_4$ 。这就是”局部旋转的累乘 = 全局朝向”。

[NOTE] 反向: 从 orientation 还原 rotation

$$R_i = Q_{i-1}^T Q_i$$

因此任意两个关节之间的相对旋转: $R_i^{(j)} = Q_j^T Q_i$ (把中间所有 R 乘起来)。

1.3 链上点的位置

设 link i 长度向量为 l_i (关节 $i \rightarrow i+1$, 在 Q_i 局部系下表示), 关节 i 的世界位置为 p_i :

$$p_{i+1} = p_i + Q_i l_i$$

末端上某点局部坐标 x_0 对应的世界点:

$$x = p_E + Q_E x_0$$

1.4 单链 FK 算法 (root $\rightarrow$ end-effector)

```
Q_0 = R_0; p_0 = t_0
for i from 1 to E:
    Q_i = Q_{i-1} R_i
    p_i = p_{i-1} + Q_{i-1} l_{i-1}
x = p_E + Q_E x_0
```

1.5 反向递推 (end-effector $\rightarrow$ root)

把 $x = p_4 + Q_4 x_0 = p_3 + Q_3(l_3 + R_4 x_0)$ 一层层把 Q 拆开, 得到等价的”由内向外”形式:

$$x = x_0; \text{ for } i = E \downarrow 1: x \leftarrow l_{i-1} + R_i x$$

最后再左乘 Q_0 加上 p_0 。两种方向数学等价, 实现取其一即可。

1.6 局部坐标查询

末端点 x 在关节 k 的局部系下的坐标:

$$x_{Q_k} = Q_k^T(x - p_k)$$

对单链: $x_{Q_3} = l_3 + R_4 x_0$, $x_{Q_2} = l_2 + R_3 l_3 + R_3 R_4 x_0$ 等。

2. 角色 FK (树状骨架) / Character FK

2.1 父指针与遍历顺序

骨架是树而非单链: 每个关节 i 有父亲 p_i 。约定关节列表按“父先于子”排序 (topological order, BVH/FBX 标准做法)。

通用 FK:

```
for i in joint_list:          # 父先于子
    Q_i = Q_{p_i} R_i
    x_i = x_{p_i} + Q_{p_i} l_i
```

2.2 关节类型与自由度 (DoF)

关节 / Joint	物理类比	DoF
Free / Floating (root)	整体 6DoF	6 (t_0, R_0)
Ball-and-socket	髋、肩	3
Universal	腕	2
Hinge / Revolute	膝、肘	1
Fixed / Welded	—	0

[NOTE] Joint limits (关节角度限制)

真实人体不能 360° 转头/反折膝盖。每个 DoF 设上下界 $\theta_{\min} \leq \theta \leq \theta_{\max}$, IK 求解时需投影回该区间。

2.3 Pose 参数

完整姿态 = $(t_0, R_0, R_1, \dots, R_n)$ 。约定 root 既给位移也给朝向; 其它关节只给相对旋转 R_i 。

2.4 BVH 文件格式 (最常用)

- HIERARCHY 段: T-pose 拓扑、每个关节的 OFFSET (即 l_i) 和 channels。
- MOTION 段: 每帧 root 的位置 + 所有关节的 Euler 角。
- 默认 Euler 顺序为 extrinsic / fixed-axis: $R = R_z R_x R_y$ 。

2.5 T-pose / A-pose 与 reference pose

T-pose 定义为“所有 $R_i = I$ ”的姿态——它就是骨架的“零点”。同一段动作 (同样的 $\{R_i\}$ 序列) 作用在 T-pose vs A-pose 上看起来不同——这就引出了 retargeting 问题。

[WARN] 同样的局部旋转 $\neq$ 同样的姿势

给两个 reference pose 不同的角色播放同一份 BVH, 胳膊、腿的全局朝向会差出去 (“Rotate arms up by 90°”在 T-pose 下抬手成 X 形, 在 A-pose 下却是斜上)。

3. Retargeting: 在不同 reference pose 间迁移动作

3.1 单刚体情形

A、B 两个 reference pose 下同一刚体的全局朝向差为 $R^{A \rightarrow B}$ (known, 预先标定)。当 A 上施加 R^A 旋转后, B 应该施加:

$$R^B = R^A (R^{A \rightarrow B})^T \iff Q^B = Q^A (Q^{A \rightarrow B})^T$$

[TIP] 一句话记忆

“全局朝向”在两 pose 间差一个固定旋转 $Q^{A \rightarrow B}$; 要复现同一全局朝向, B 的旋转 = A 的旋转 $\times$ 朝向差的转置。

3.2 链式情形

每个关节有自己的 global offset $Q_i^{A \rightarrow B}$ (不只是 root! T-pose 与 A-pose 的肩关节朝向就不同)。

已知: $Q_i^A = Q_{p_i}^A R_i^A$, $Q_i^B = Q_{p_i}^B R_i^B$, $Q_i^B = Q_i^A (Q_i^{A \rightarrow B})^T$ 。

代入推导:

$$R_i^B = Q_{p_i}^{A \rightarrow B} R_i^A (Q_i^{A \rightarrow B})^T$$

这是 retargeting 的核心公式。注意是 $Q_{p_i}^{A \rightarrow B}$ (父亲的 offset) 和 $Q_i^{A \rightarrow B}$ (自己的 offset) 一前一后包夹 R_i^A 。

[NOTE] 直觉

$(Q_{p_i}^{A \rightarrow B})$ 把”B 父系”的旋转输入对齐到”A 父系”; R_i^A 是 A 上的相对旋转; 最后 $(Q_i^{A \rightarrow B})^T$ 把结果输出回 B 的子坐标系。

4. Inverse Kinematics: 问题与启发式解法

4.1 forward vs inverse

方面	Forward $x = f(\theta)$	Inverse: 给 $\tilde{x}$ 求 θ
难度	直接代入, 唯一	解非线性方程, 可能多解/无解
DoF 关系	$\dim \theta \gg \dim x$ 没事	$\dim \theta \gg \dim x \rightarrow$ 欠定 / 多解
物理直觉	θ 不直观	$\tilde{x}$ 直观 (“手到那儿”)

4.2 解的存在性 (两连杆例)

设两 link 长 l_0, l_1 , 目标距离 root 为 d : $-d > l_0 + l_1$: 无解 (够不到) $-|l_0 - l_1| \leq d \leq l_0 + l_1$: 多解 (肘可上可下) $-d < |l_0 - l_1|$: 无解

4.3 Two-joint 解析法

1. 调 joint 1 角度让 $\|l_0\| + \|l_1\|$ 弯到正好等于 d (用余弦定理)。
2. 调 joint 0 把整体”指”向 $\tilde{x}$ 。
3. 必要时绕 l_0 轴再转一下匹配朝向。

简单、稳定, 被广泛用于胳膊/腿这种短链。

4.4 CCD —Cyclic Coordinate Descent

[TIP] 一句话

从末端最近的关节开始，每次只转一个关节，把当前末端 x 对准目标 $\tilde{x}$ ；一圈圈循环。

伪代码：

```
repeat until  $\|x - \tilde{x}\| < \epsilon$ :  
  for i from end-effector-1 down to root:  
    rotate joint i so that vector (joint_i  $\rightarrow$  end) aligns with (joint_i  $\rightarrow \tilde{x}$ )
```

每一步只解一个简单的”两个向量对齐”小问题（旋转轴 = 两向量叉积，角度 = 两向量夹角）。

优缺点：– 实现 5 行；快、不需要导数。– 第一个被处理的关节（最靠末端）转得最多 $\rightarrow$ 姿态可能不自然。– 收敛慢，对初值敏感。

[NOTE] CCD 也能处理”伸缩 link”

若 link 长度可变 (stretch IK)，把”使 $x - \tilde{x} \perp l_{23}$ ”作为该关节的最优条件即可——本质上是对 θ_i 单变量做 $\arg \min F$ 。

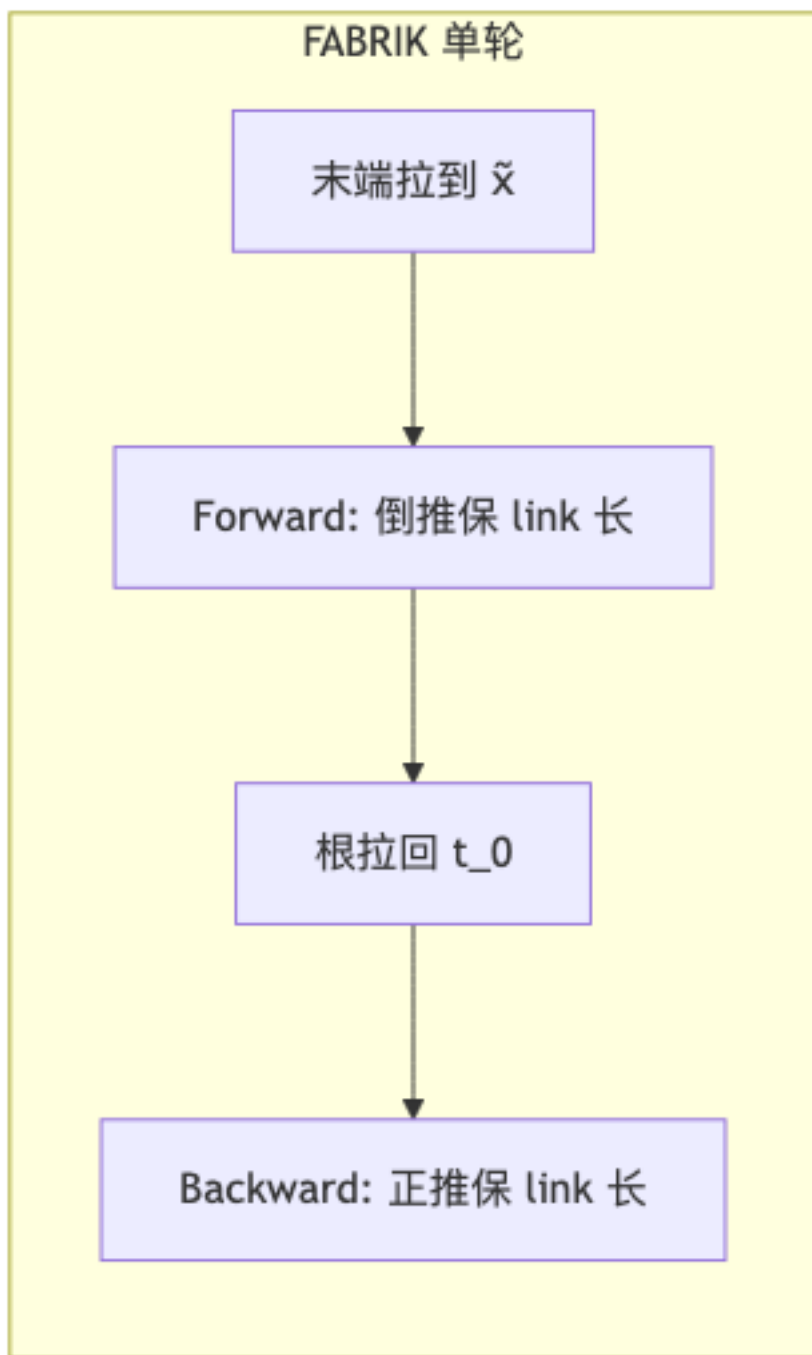
4.5 FABRIK —Forward And Backward Reaching IK

(Aristidou & Lasenby 2011, Graphical Models)

[TIP] 算法大意

在位置空间直接走两遍：1. **Forward pass**：把末端拉到目标，沿链向 root 反推每个关节位置（保持各 link 长度）。2. **Backward pass**：把 root 拉回原位，沿链向末端正推每个关节位置（再次保持长度）。反复直到末端到达目标。

特点：– 简单、快、保证收敛（无约束情形）。– **基于位置**，需要事后从相邻位置反算关节旋转。– 可加 joint constraints（投影到允许锥上），但收敛性不再保证。



5. IK 作为优化问题 / IK as Optimization

5.1 形式化

$$\min_{\theta} F(\theta) = \frac{1}{2} \|f(\theta) - \tilde{x}\|_2^2$$

其中 f 是 FK 函数, $\tilde{x}$ 是目标位置 (多目标时叠加)。这是经典 **非线性最小二乘 (nonlinear least squares)** 问题。

5.2 通用迭代框架

```
θ_0 ← initial guess
repeat:
    pick a descent direction d
    pick a step size α
    θ ← θ + α d
until converged
```

不同选 d 的策略 → 不同方法。

5.3 重新解读 CCD：坐标下降

[NOTE] 关键观察

CCD-IK 就是 F 上的 **coordinate descent**：每次锁住其它 θ_j ，对单个 θ_i 求 $\arg \min F$ 。“把 $l_{i,E}$ 转向 $\tilde{x}$ ”恰好是该单变量子问题的解析最优解。

这把启发式与优化串在一起：CCD 不是 hack，而是在选“坐标轴方向”作为 descent direction。

6. Jacobian 方法 / Gradient and Newton-style Updates

6.1 Jacobian 定义

$$J = \frac{\partial f}{\partial \theta} \in \mathbb{R}^{m \times n}, \quad f: \mathbb{R}^n \rightarrow \mathbb{R}^m$$

每列 $\partial f / \partial \theta_i$ 表示“扭动第 i 个关节角时末端位置移动的方向”。

6.2 几何法计算 Jacobian (hinge 关节)

设关节 i 旋转轴为单位向量 a_i （世界系），从关节 i 到末端的向量为 r_i 。由 Rodrigues 公式：

$$x' - x = \sin \delta \theta_i (a_i \times r_i) + (1 - \cos \delta \theta_i) a_i \times (a_i \times r_i)$$

取极限 $\delta \theta_i \rightarrow 0$ ：

$$\frac{\partial f}{\partial \theta_i} = a_i \times r_i$$

[TIP] 这就是 Jacobian

几何法的优雅之处：J 的每一列 = (关节轴) $\times$ (从关节到末端的向量)。

6.3 Ball joint 的 Jacobian

把 ball joint 视为串联三个 hinge（按 Euler 顺序 $R_i = R_{ix} R_{iy} R_{iz}$ ），三根虚拟轴：

$$a_{ix} = Q_{i-1} e_x, \quad a_{iy} = Q_{i-1} R_{ix} e_y, \quad a_{iz} = Q_{i-1} R_{ix} R_{iy} e_z$$

对应三列 $\partial f / \partial \theta_{i*} = a_{i*} \times r_i$ 。

[WARN] Axis-angle 不要直接拿来求导

用 axis-angle (θu) 参数化 ball joint 的话, Jacobian 表达式非常复杂——用 Euler 角 + 三 hinge 的视角更简单。

6.4 数值法: Finite Differencing & Autograd

- **Finite difference:** $\partial f / \partial \theta_i \approx (f(\theta + \delta e_i) - f(\theta)) / \delta$, 每列要做一次 FK, 简单但慢、且数值噪声敏感。
- **Autograd:** 用 PyTorch / TensorFlow / JAX 把 f 当作可微图, 自动得到精确 Jacobian。生产代码常见。

6.5 Jacobian Transpose / Gradient Descent

$$\nabla F(\theta) = J^T (f(\theta) - \tilde{x}) = J^T \Delta$$

$$\theta_{i+1} = \theta_i - \alpha J^T \Delta$$

[NOTE] 直觉

J^T 把“末端位置误差”反传成“关节角应该如何改”。每个关节 i 收到的更新是 $a_i \times r_i$ 与误差 Δ 的内积——若该关节的运动方向能减小误差, 它就动得多。

特点: 一阶; 不需要解线性方程; 步长 α 难调, 太大震荡, 太小很慢。

6.6 Gauss-Newton 与伪逆

把 f 在 θ_0 一阶展开: $f(\theta) \approx f(\theta_0) + J(\theta - \theta_0)$, 代入 F 得二次模型, 最优条件:

$$J^T J(\theta - \theta_0) = -J^T \Delta$$

情形 A: $J^T J$ 可逆 ($m \geq n$, 约束多于参数)

$$\theta = \theta_0 - (J^T J)^{-1} J^T \Delta, \quad J^+ = (J^T J)^{-1} J^T$$

情形 B: $J^T J$ 奇异, $J J^T$ 可逆 ($m < n$, 欠定, IK 常见情形)

直接令 $J(\theta - \theta_0) = -\Delta$ 用伪逆:

$$\theta = \theta_0 - J^+ \Delta, \quad J^+ = J^T (J J^T)^{-1}$$

此即 Moore-Penrose pseudoinverse 的两种形式。

[NOTE] 何时哪种

- 一只手 1 个目标 (3 维) + 多个关节 (≥ 3 DoF): $m = 3 < n \rightarrow$ 用 $J^+ = J^T (J J^T)^{-1}$ 。- 同时约束 2 只手 + 头 (9 维) + 链关节 9 DoF: 可能 $m \geq n \rightarrow$ 用 $J^+ = (J^T J)^{-1} J^T$ 。

伪逆法通常比 Jacobian 转置快得多——它直接走“目标点位移”而非“梯度”。

6.7 Damped Least Squares / Levenberg–Marquardt

伪逆的灾难：当链伸直或退化构型时， JJ^T 接近奇异， θ 更新爆炸。加阻尼项：

$$J^* = J^T(JJ^T + \lambda I)^{-1} \quad \text{或} \quad J^* = (J^T J + \lambda I)^{-1} J^T$$

对应优化目标变为

$$F(\theta) = \frac{1}{2} \|f(\theta) - \tilde{x}\|^2 + \frac{\lambda}{2} \|\theta - \theta_i\|^2$$

即“在最小化误差同时尽量少改 θ ”。 λ 大 $\rightarrow$ 更稳，收敛慢； λ 小 $\rightarrow$ 接近纯伪逆，奇异处不稳。

[TIP] LM 是工业默认

Levenberg–Marquardt 在 Maya / Blender / Unity 的 IK solver、机械臂控制器中无处不在；它用一个旋钮 λ 平滑插值“梯度下降”和“高斯牛顿”两端。

6.8 加权阻尼

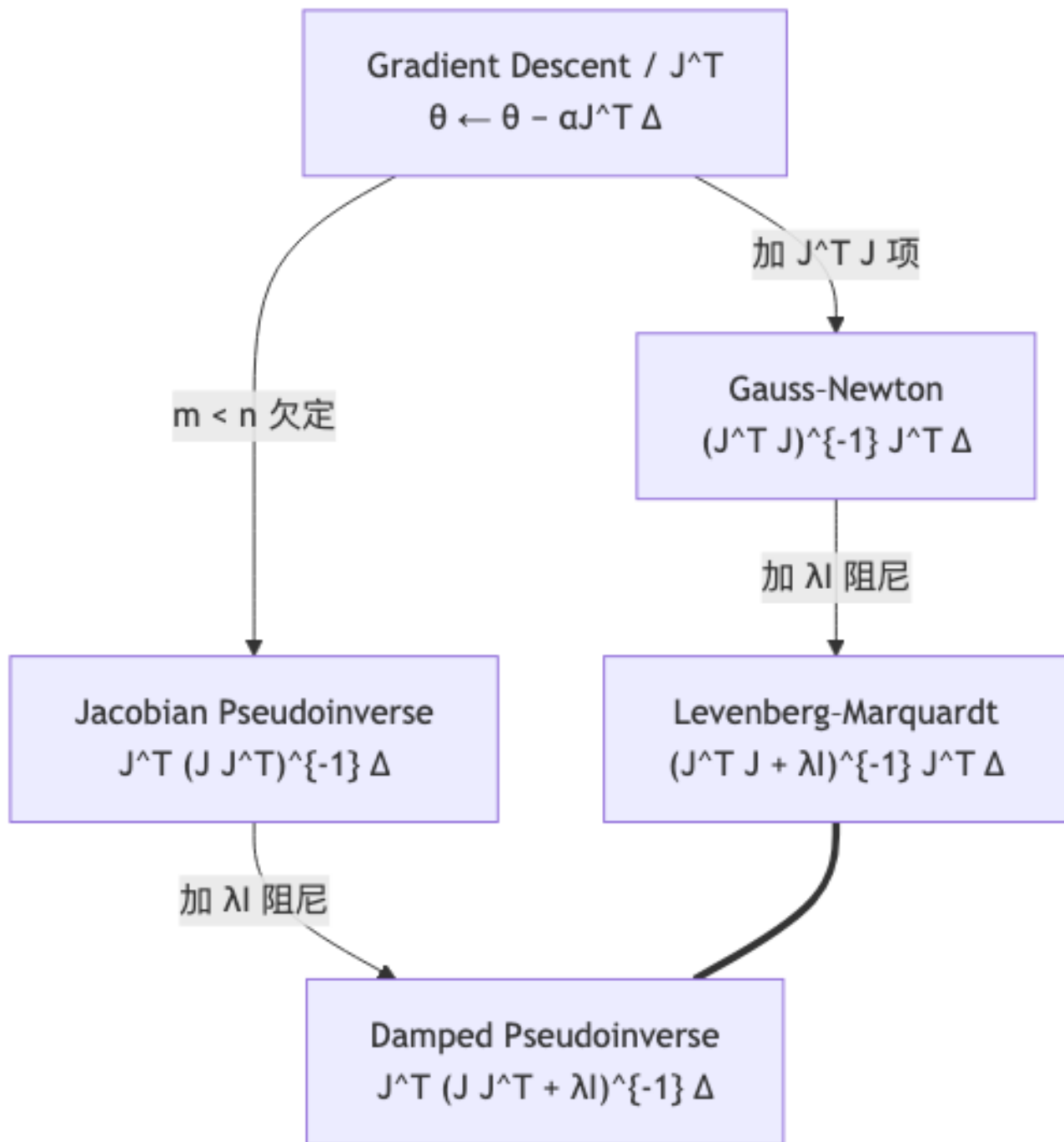
进一步给每个关节自由度一个权 w_i ：

$$F = \frac{1}{2} \|f - \tilde{x}\|^2 + \frac{\lambda}{2} (\theta - \theta_i)^T W (\theta - \theta_i)$$

$$\theta_{i+1} = \theta_i - \alpha (J^T J + \lambda W)^{-1} J^T \Delta$$

W 可惩罚“不希望动”的关节（脊椎、骨盆），让 IK 更倾向于动末梢关节。

6.9 Jacobian 方法对比一图



7. 全身 IK / Full-body IK

7.1 链穿过 root 怎么办

只想”把脚踩到某点”，但脚 → 骨盆 → 另一只脚的链经过了 root：

[TIP] 标准做法（对单约束）

1. 先把脚到 root 之间的关节角先反向旋转（视脚为新 root）；2. 在反向链上跑 IK；3. 用沿反向链的 FK 重新求出 root 的世界变换。

或更通用——直接把 t_0, R_0 放进 θ 一起优化（root 6 DoF 当作“虚拟关节”）。

7.2 多约束 IK

同时约束多个 end-effector（双脚 + 双手 + 头）：

$$F(\theta) = \frac{1}{2} \sum_i \|f_i(\theta) - \tilde{x}_i\|^2 + \frac{\lambda}{2} \|\theta - \tilde{\theta}\|^2$$

正则项 $\|\theta - \tilde{\theta}\|^2$ 把姿态拉向参考姿态 $\tilde{\theta}$ （如上一帧的 pose、用户手摆的“自然姿态”），缓解欠定带来的“妖怪 pose”。

7.3 Rig with multiple IK chains

商业 rig（Blender, Maya）允许同一个角色定义多条 IK 链：用户可激活其中几条，未被激活的关节自由（FK 控制）。这是动画师的工作流，不是新算法。

7.4 Constraint 处理

- **Joint limits**：每步更新后投影回 $[\theta_{\min}, \theta_{\max}]$ ；或在优化里加 barrier。
- **不可拉伸约束**：FABRIK 天然满足；伪逆法里把“link length”也写进 f 作软约束。

8. 全局总结

[TIP] 主线一页

1. FK = 沿父子链累乘旋转 + 累加位移；公式 $Q_i = Q_{p_i} R_i$, $x_i = x_{p_i} + Q_{p_i} l_i$ 必须能默写。
2. **Reference pose** 不同时用 retargeting 公式 $R_i^B = Q_{p_i}^{A \rightarrow B} R_i^A (Q_i^{A \rightarrow B})^T$ 。
3. **IK 启发式**：CCD（坐标下降的几何版）、FABRIK（位置空间两遍扫描）。
4. **IK 优化**：目标 $F = \frac{1}{2} \|f(\theta) - \tilde{x}\|^2$ ，关键工具是 Jacobian $J = \partial f / \partial \theta$ ，hinge 关节下 $J_i = a_i \times r_i$ 。
5. **更新公式四件套**：
- $\theta \leftarrow \theta - \alpha J^T \Delta$ （一阶 / Jacobian transpose）
- $\theta \leftarrow \theta - J^+ \Delta$ （伪逆 / 高斯牛顿）
- $\theta \leftarrow \theta - J^T (J J^T + \lambda I)^{-1} \Delta$ （LM / DLS，工业默认）
- 上式加权矩阵 $W \rightarrow$ 加权阻尼
6. **欠定与正则**：DoF 远多于约束 $\rightarrow$ 加 $\|\theta - \tilde{\theta}\|^2$ 把解拉向自然姿态。
7. **全身 IK**：链穿过 root 时把脚当 root 反推；多约束就把所有项加起来。

记号速查

符号	含义
R_i	关节 i 的局部旋转（相对父系）
Q_i	bone i 的全局朝向 = $Q_{p_i} R_i$
l_i	link 偏移向量（ Q_i 的局部系下）
p_i, x_i	关节 i 的世界位置

符号	含义
t_0, R_0	root 的位移与朝向
$\tilde{x}$	IK 目标位置
$f(\theta)$	FK 函数
$J = \partial f / \partial \theta$	Jacobian
J^+	Moore–Penrose 伪逆
λ	阻尼系数 (DLS / LM)
W	关节权重矩阵
a_i	关节 i 旋转轴 (世界系, hinge)
r_i	关节 $i \rightarrow$ 末端向量
Δ	$f(\theta) - \tilde{x}$ (位置误差)

Lecture 04 —Keyframe Animation

[TIP] 学习指南

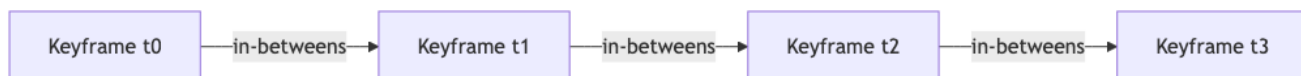
本节核心：**关键帧动画 (Keyframe Animation)** 的数学骨架就是**插值 (Interpolation)**。主线：Keyframe 思想 → 线性 / 多项式插值 → 样条 (Spline): Cubic / Hermite / Catmull-Rom → 旋转的插值 (SLERP / 四元数样条) → 多维插值 (双线性、重心、Blend Space、RBF)。重点掌握：① Hermite 基函数与矩阵形式 ② Catmull-Rom 切线选取 ③ SLERP 公式 ④ Barycentric / Blend Space 在动画混合中的用途。

0. 从 Zoetrope 到 Keyframe

[NOTE] Keyframe 动画 (关键帧动画)

动画师只指定少量**关键帧 (Keyframes)**，剩余的**中间帧 (in-betweens / tweens)** 通过插值自动生成。在 3D 中，“keyframe”通常对应某些时间点上**骨骼/控制器**的位姿 (位置、旋转、缩放、形态参数等)。

3D Rotoscoping: 参考真人动作录像逐帧“描”骨骼，再以 keyframe 形式存储。



中间帧的产生方式 = 在关键帧之间做插值，于是问题转化为「给定数据点，求平滑通过它们的函数」。

1. 插值问题的形式化

1.1 定义

给定数据对集合

$$D = \{(x_i, y_i) \mid i = 0, \dots, N\}$$

求函数 $f(x)$ 满足

$$f(x_i) = y_i, \quad \forall (x_i, y_i) \in D$$

- **插值 (Interpolation)**: 函数严格通过所有点。
- **拟合 (Fitting / Regression)**: 函数尽可能靠近，但不必经过。
- **外推 (Extrapolation)**: 在 $[x_0, x_N]$ 之外取值。

[WARN] 插值 vs 拟合

关键帧动画几乎总是用**插值**——动画师指定的关键姿态必须**精确出现**在对应时间点；偏离一帧都会被察觉。

1.2 阶梯插值 (Stepped)

$$f(x) = y_i, \quad x \in [x_i, x_{i+1})$$

– 完全不连续 (discontinuity), 常用于“开关型”参数 (如可见性、表情切换)。

1.3 线性插值 (Linear Interpolation, LERP)

在 $[x_1, x_2]$ 上:

$$f(x) = y_1 + \frac{x - x_1}{x_2 - x_1}(y_2 - y_1)$$

设归一化参数 $t = (x - x_1)/(x_2 - x_1) \in [0, 1]$:

$$f(t) = (1 - t)y_1 + ty_2$$

[NOTE] 平滑性 (Smoothness)

衡量两段曲线的拼接质量: – C^0 : 位置连续 (值相等) – C^1 : 位置 + 速度 (一阶导) 连续 – C^2 : 位置 + 速度 + 加速度 (二阶导) 连续
线性插值整体仅 C^0 , 关键帧处折角 → 速度跳变 → 角色动作“顿挫”。

2. 多项式插值与 Runge 现象

2.1 全局多项式

设 $f(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$, 代入 $N + 1$ 个数据点得 **Vandermonde** 线性系统:

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \dots & x_N^n \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_N \end{bmatrix}$$

要拟合 $N + 1$ 个点需 $n = N$ 阶多项式。

2.2 Runge 现象 (Runge's Phenomenon)

[WARN] 高阶多项式的振荡

在区间端点附近, 高阶多项式会出现**剧烈振荡**——即使数据本身平滑。

$$f(x) = a_0 + a_1x + \dots + a_{15}x^{15} \xrightarrow{\text{end of interval}} \text{wild oscillation}$$

因此实际中**偏好低阶多项式**——但单段低阶多项式没法穿过很多点。解决方案: **分段 (piecewise)** → 样条 (Spline)。

3. 样条插值 (Spline Interpolation)

3.1 总体思想

把定义域分成 N 段，每段用一个低阶多项式 $S_i(x)$ ，并在节点处加连续性条件：

$$f(x) = S_i(x), \quad x \in [x_i, x_{i+1}]$$

阶数	名称	平滑性
1	分段线性 (piecewise linear)	C^0
3	三次样条 (Cubic Spline)	C^2

3.2 Cubic Spline (自然三次样条)

每段： $S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$ ，共 N 段、 $4N$ 个未知数。条件：

条件	个数
插值条件 $S_i(x_i) = y_i, S_i(x_{i+1}) = y_{i+1}$	$2N$
$C^1: S'_{i-1}(x_i) = S'_i(x_i)$	$N - 1$
$C^2: S''_{i-1}(x_i) = S''_i(x_i)$	$N - 1$
边界条件 (S'_0, S'_{N-1} 或 S''_0, S''_{N-1})	2

合计 $4N$ ，得到一个大型线性方程组。

[WARN] Cubic Spline 的两个痛点

- **No local control (无局部控制)**：动一个数据点，整条曲线都会变。
 - **计算昂贵**：当 N 很大时要解极大的线性系统。
- 动画/CG 里更偏爱局部式样条 (Hermite、Catmull-Rom、Bezier)。

3.3 Cubic Hermite Spline

[NOTE] 关键转变

Hermite 不仅给数据点 (x_i, y_i) ，还给每个点处的一阶导 m_i 。这样每段只与本段两端的 (y_i, m_i) 有关——**天然局部控制**。

推导

要在 $(x_1, y_1, m_1), (x_2, y_2, m_2)$ 之间求三次曲线 $S(x) = ax^3 + bx^2 + cx + d$ ，且

$$S(x_1) = y_1, S(x_2) = y_2, S'(x_1) = m_1, S'(x_2) = m_2$$

归一化 $t = (x - x_1)/(x_2 - x_1) \in [0, 1]$ ，写成 $S(t) = at^3 + bt^2 + ct + d$ ，约束变成

$$\begin{cases} S(0) = d = y_1 \\ S(1) = a + b + c + d = y_2 \\ S'(0) = c = m_1 \\ S'(1) = 3a + 2b + c = m_2 \end{cases}$$

矩阵化：

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

求逆得到关键的 Hermite 基矩阵：

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ m_1 \\ m_2 \end{bmatrix}$$

Hermite Basis Functions (Hermite 基函数)

把上式写回 $S(t) = [t^3 \ t^2 \ t \ 1] M [y_1 \ y_2 \ m_1 \ m_2]^T$ 后得：

$$S(t) = H_0(t) y_1 + H_1(t) y_2 + H_2(t) m_1 + H_3(t) m_2$$

$$H_0(t) = 2t^3 - 3t^2 + 1$$

$$H_1(t) = -2t^3 + 3t^2$$

$$H_2(t) = t^3 - 2t^2 + t$$

$$H_3(t) = t^3 - t^2$$

性质（直接代入验证）：

	H_0	H_1	H_2	H_3
$t = 0$ 处值	1	0	0	0
$t = 1$ 处值	0	1	0	0
$t = 0$ 处导数	0	0	1	0
$t = 1$ 处导数	0	0	0	1

[TIP] PowerPoint 的“Curve Tool”就是 Hermite/Bezier 的可视化版本。

高维推广

向量值情形（动画里 $\mathbf{y}$ 是位置、欧拉角、关节角、形变系数等）：

$$\mathbf{s}(t) = H_0(t) \mathbf{y}_1 + H_1(t) \mathbf{y}_2 + H_2(t) \mathbf{m}_1 + H_3(t) \mathbf{m}_2$$

每个分量独立用同一组 Hermite 基。

3.4 Catmull–Rom Spline

Hermite 需要切线 m_i ，但动画师通常只指定关键帧的位置——切线如何自动求？Catmull–Rom 给出最简洁的答案：

[NOTE] Catmull–Rom 切线公式

对中间数据点 p_1 （前后邻居 p_0, p_2 ），切线

$$m_1 = \frac{1}{2} \cdot \frac{p_2 - p_0}{x_2 - x_0}$$

推广到任意 i ：

$$m_i = \frac{1}{2} \cdot \frac{p_{i+1} - p_{i-1}}{x_{i+1} - x_{i-1}}$$

把它代回 Hermite 基函数即得到 Catmull–Rom 段：

$$S(t) = H_0(t) p_1 + H_1(t) p_2 + H_2(t) \frac{p_2 - p_0}{2} + H_3(t) \frac{p_3 - p_1}{2}$$

特点：

- C^1 连续（不强求 C^2 ，比 Cubic Spline 弱一点，但够动画用）。
- 局部控制：移动 p_i 仅影响前后两段。
- 直接通过所有控制点（不像 Bezier 一般不过中间控制点）。

[EX] 命名小知识

Edwin Catmull —— Pixar 与 Disney Animation Studios 前总裁，与 Pat Hanrahan 共获 2019 ACM Turing Award，表彰其对 3D 计算机图形与 CGI 的奠基贡献。

3.5 三种样条对比

样条	控制	连续性	是否过点	是否需切线输入
Cubic Spline (natural)	全局	C^2	是	否（自动解出）
Cubic Hermite	局部	C^1	是	是（用户/算法给定）
Catmull–Rom	局部	C^1	是	否（用邻居自动求）

4. 旋转的插值 (Interpolation of Rotations)

4.1 旋转表示回顾

表示	形式	备注
旋转矩阵 (Rotation Matrix)	$R \in SO(3)$	9 个分量 + 6 个正交约束
欧拉角 (Euler Angles)	$R_x(\alpha)R_y(\beta)R_z(\gamma)$	万向锁 (gimbal lock)
轴角 (Axis–Angle)	$(\mathbf{u}, \theta)$ 或 $\theta = \theta \mathbf{u}$	简洁直观
单位四元数 (Unit Quaternion)	$\mathbf{q} = (w, \mathbf{v}), \ \mathbf{q}\ = 1$	球面流形，无奇异

4.2 直接插参数行不行？

[WARN] 朴素逐分量插值的问题

- 欧拉角：插值后 $R_x R_y R_z$ 通常不再是“短路径”旋转；而且容易撞万向锁。
 - 轴角： θ 与 $\mathbf{u}$ 各自插值会破坏单位长度。
 - 四元数线性插：直线穿过球内部 $\rightarrow$ 结果不再是单位四元数，且角速度不恒定。
- 共同问题：Rotational speed is usually not constant（旋转速度不均匀，动作看起来“先慢后快”）。

4.3 SLERP（球面线性插值）for Quaternions

在单位四元数球面 S^3 上沿大圆做匀速运动：

$$\mathbf{q}(t) = \frac{\sin((1-t)\theta)}{\sin\theta} \mathbf{q}_0 + \frac{\sin(t\theta)}{\sin\theta} \mathbf{q}_1$$

其中

$$\cos\theta = \mathbf{q}_0 \cdot \mathbf{q}_1$$

性质：

- 常数角速度（constant rotational speed）——旋转动作匀速，自然顺滑。
- 沿球面最短路径。
- 但只是“线性”——在多个关键姿态之间需要更高阶平滑。

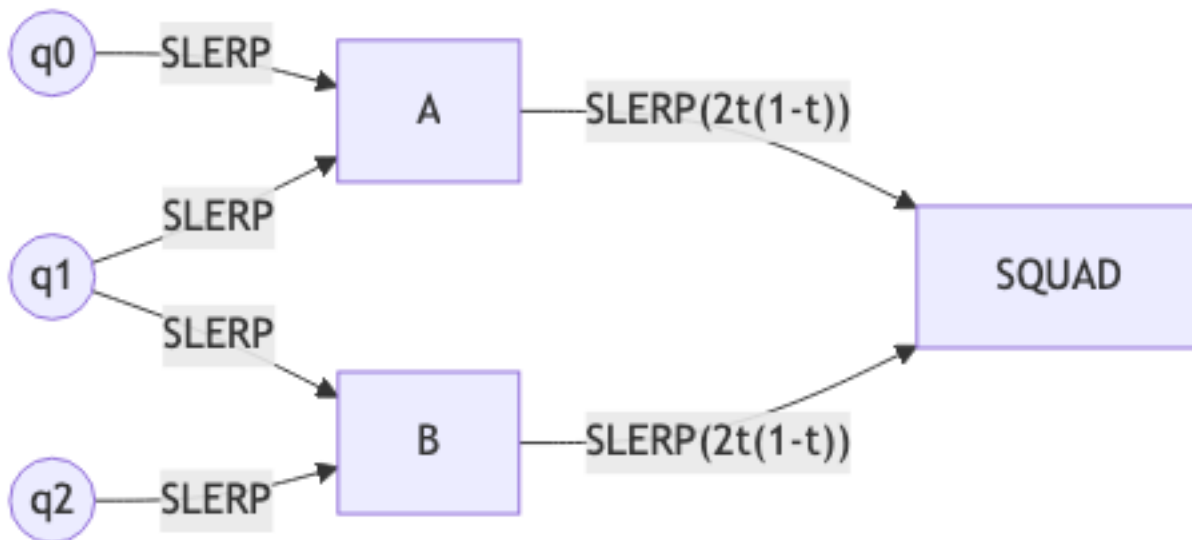
[TIP] 实现细节

- 若 $\mathbf{q}_0 \cdot \mathbf{q}_1 < 0$ ，先取 $-\mathbf{q}_1$ （同一旋转的双重表示）以选短弧。
- $\theta \rightarrow 0$ 时退化为线性插值（数值稳定性）。

4.4 四元数样条 (Quaternion Splines)

SLERP 是“线性”，要做 Hermite/Catmull-Rom 风格的高阶插值需要专用方法：

- SQUAD (Spherical and QUADrangle interpolation, Shoemake 1987)：用四个四元数和嵌套 SLERP 实现 C^1 平滑曲线。
- Kochanek-Bartels (TCB) splines：附加 Tension / Continuity / Bias 三个手感旋钮。
- 经典文献：Ken Shoemake. 1985. Animating rotation with quaternion curves. SIGGRAPH.



5. 多维 / 散点插值

5.1 双线性插值 (Bilinear Interpolation)

在规则网格 $\{p_{11}, p_{12}, p_{21}, p_{22}\}$ 内、参数 $(s, t) \in [0, 1]^2$:

$$\mathbf{x}(s, t) = (1 - t)[(1 - s)\mathbf{p}_{11} + s\mathbf{p}_{21}] + t[(1 - s)\mathbf{p}_{12} + s\mathbf{p}_{22}]$$

直觉: 先沿 s 在两条边各插值得 $\mathbf{m}_2, \mathbf{m}_4$, 再沿 t 在它们之间线性插值。

[NOTE] Bicubic Interpolation

把两个方向都换成三次 (Hermite/Catmull-Rom), 需要 $4 \times 4 = 16$ 个控制点。 C^1 平滑、用于纹理放大、地形等。

5.2 重心插值 (Barycentric Interpolation)

三角形 ABC 内任意点 P 表示为:

$$P = w_a A + w_b B + w_c C, \quad w_a + w_b + w_c = 1$$

权重由面积比给出:

$$w_a = \frac{\Delta PBC}{\Delta ABC}, \quad w_b = \frac{\Delta PCA}{\Delta ABC}, \quad w_c = \frac{\Delta PAB}{\Delta ABC}$$

四面体推广 (体积比):

$$P = w_a A + w_b B + w_c C + w_d D, \quad w_a = \frac{V_{PBCD}}{V_{ABCD}}, \dots$$

5.3 Blend Space (混合空间)

游戏引擎里多动画混合的标准做法:

[EX] 1D Blend Space

把“行走 / 跑步 / 冲刺”三个 clip 摆在 1D 速度轴上, 用速度参数线性混合。

2D Blend Space (如 Unreal Engine 的 BlendSpace): 横轴前进速度、纵轴侧向速度, 三角剖分后用重心权重:

$$\mathbf{x} = w_a \mathbf{p}_1 + w_b \mathbf{p}_2 + w_c \mathbf{p}_3$$

实时根据角色当前 (v_x, v_y) 求权重并混合骨骼姿态。

5.4 散点数据插值 (Scattered Data Interpolation)

控制点不在规则网格上时 (动画姿态库往往如此), 常用方法:

- 线性 / 最小二乘 (Linear / Least squares)
- 样条 (Splines)
- 反距离加权 (Inverse Distance Weighting)
- 高斯过程 (Gaussian Process)
- 径向基函数 (Radial Basis Function, RBF)

5.5 Radial Basis Function 插值

[NOTE] RBF 表达式

$$f(\mathbf{x}) = \sum_{i=1}^K w_i \varphi(\|\mathbf{x} - \mathbf{x}_i\|)$$

每个数据点 $\mathbf{x}_i$ 贡献一个只依赖距离的“基”。

求权重 w_i : 要求 $f(\mathbf{x}_i) = y_i$, 得到线性系统

$$\underbrace{\begin{bmatrix} R_{1,1} & \cdots & R_{1,K} \\ \vdots & \ddots & \vdots \\ R_{K,1} & \cdots & R_{K,K} \end{bmatrix}}_{R_{ij} = \varphi(\|\mathbf{x}_i - \mathbf{x}_j\|)} \begin{bmatrix} w_1 \\ \vdots \\ w_K \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_K \end{bmatrix}$$

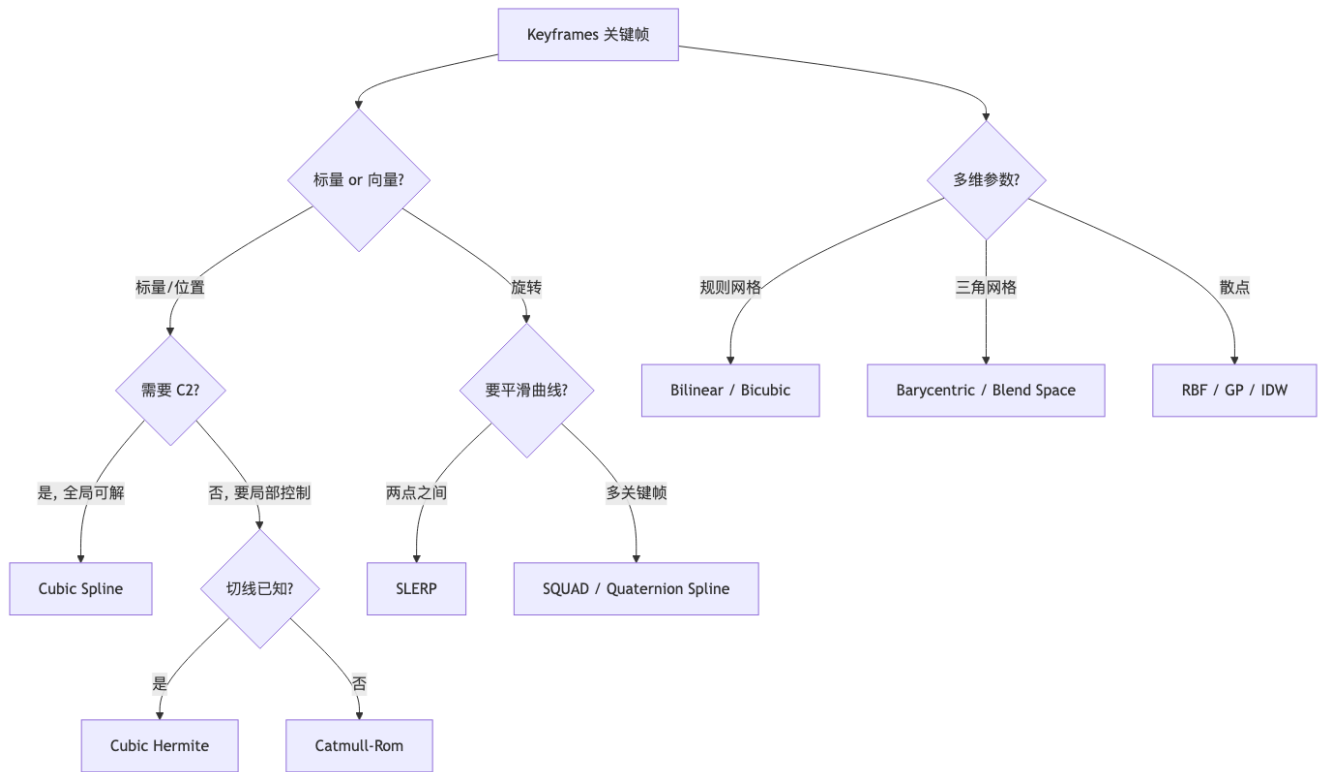
常用核函数:

名称	$\varphi(r)$
Gaussian	e^{-r^2/c^2}
Inverse multiquadric	$1/\sqrt{r^2 + c^2}$
Thin plate spline	$r^2 \log r$
Polyharmonic splines	r^k (k 奇) / $r^k \log r$ (k 偶)

[EX] Spatial Keyframing (Igarashi et al., SCA 2005)

在 3D 空间里把控制器拖到不同位置作为空间关键帧, 每个位置对应一种角色姿态。实时根据控制器当前位置用 RBF 混合姿态 → 即兴动画 (performance-driven animation)。

6. 总览：从关键帧到曲线的选择树



7. 速查表

概念	一句话总结
Linear Interp.	$f(t) = (1-t)y_1 + ty_2$, 仅 C^0
Stepped	$f(x) = y_i$, 常用于离散切换
Polynomial	阶数过高 $\rightarrow$ Runge 振荡
Cubic Spline	全局 C^2 , 无局部控制
Hermite	输入 (y_i, m_i) , 基函数 H_0, H_1, H_2, H_3
Catmull-Rom	$m_i = \frac{p_{i+1} - p_{i-1}}{2(x_{i+1} - x_{i-1})}$ 自动估切线
SLERP	$\mathbf{q}(t) = \frac{\sin(1-t)\theta}{\sin \theta} \mathbf{q}_0 + \frac{\sin t\theta}{\sin \theta} \mathbf{q}_1$, 匀角速
Bilinear	网格上两次 LERP 嵌套
Barycentric	三角形面积比作权重
RBF	$f(\mathbf{x}) = \sum w_i \varphi(\ \mathbf{x} - \mathbf{x}_i\)$, 散点万能

记号速查

符号	含义
$D = \{(x_i, y_i)\}$	关键帧数据集
$S_i(x)$	第 i 段样条多项式
m_i	x_i 处的一阶导 (切线)

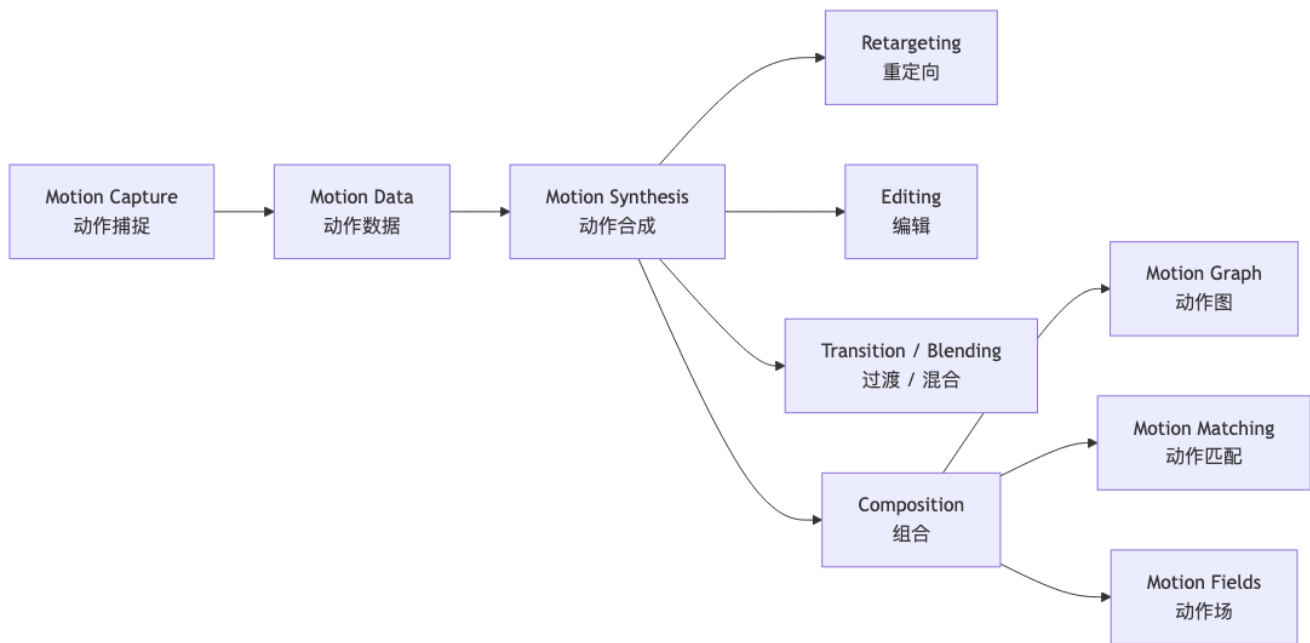
符号	含义
$H_0, \dots, H_3$	Hermite 基函数
$\mathbf{q} = (w, \mathbf{v})$	单位四元数
θ	$\cos \theta = \mathbf{q}_0 \cdot \mathbf{q}_1$, 两旋转的夹角
w_a, w_b, w_c	重心坐标 (面积比)
$\varphi(r)$	RBF 核函数
C^k	k 阶导数连续

Lecture 05 —Data-driven Animation

[TIP] 学习指南

本节核心：从”如何采集动作数据 (Motion Capture)“到”如何使用动作数据 (Motion Synthesis)“的完整数据驱动管线。前置知识：Lec 03 刚体运动学（旋转表示）、Lec 04 骨骼蒙皮 (Skeleton & Skinning)。重点关注：① mocap 系统分类与原理 ② Motion Retargeting 的难点 ③ Motion Transition 中的 facing frame 与坐标对齐 ④ Motion Graph 的图论结构 ⑤ Motion Matching 的特征向量与最近邻搜索。

0. 课程脉络 / Outline



数据驱动动画 (data-driven animation) 与传统关键帧 (keyframe animation) 的根本差异：用大量真实采集的动作片段作为”原料”，通过检索 / 拼接 / 混合在线生成符合用户输入的运动，而不是手工逐帧设计。

1. Motion Capture 动作捕捉

1.1 历史脉络 (History)

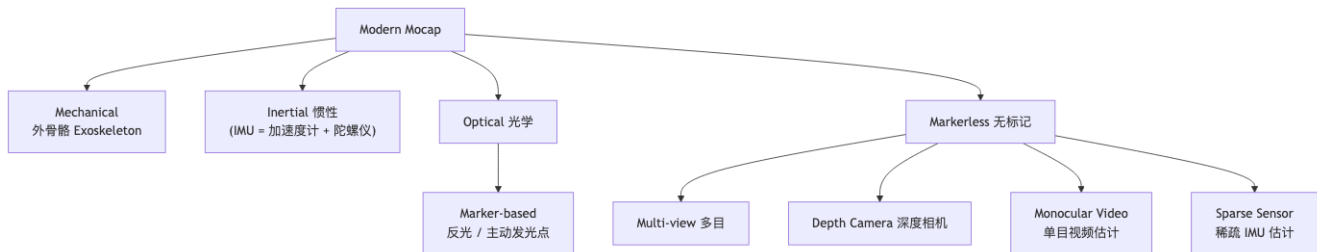
时期	关键人物 / 技术	思想
1878	Eadweard Muybridge — 《奔马》	用照相机分解运动

时期	关键人物 / 技术	思想
~1914	Max Fleischer —Rotoscoping (转描)	在拍摄的真人影片上逐帧描线
1937–1951	Disney 《白雪公主》《爱丽丝梦游仙境》	Rotoscoping 在 2D 动画里的应用
1941	万氏兄弟 《铁扇公主》	中国早期 rotoscope 实践
现代	3D Rotoscoping / Optical / Inertial / Markerless	把同一思想推广到 3D 数字角色

[NOTE] Rotoscoping 的本质

在已经拍好的 video 上 frame-by-frame 描出角色轮廓。它是“用真实运动驱动绘制角色”的最早形式——动作捕捉的精神祖先。

1.2 现代 mocap 系统分类



[EX] 各类 mocap 的优劣

– **Mechanical / Exoskeleton 机械外骨骼**: 直接读关节角, 不怕遮挡, 但笨重、限制动作。– **Inertial / IMU 惯性**: 穿戴轻便, 无需场地, 但有 drift (积分漂移), 位置依靠融合估计。– **Optical Marker-based 光学标记**: 精度最高, 是工业级标准 (Vicon, OptiTrack); 需多相机标定, 遮挡敏感; marker 数据需“solving”成骨骼姿态 (Holden 2018 用 denoising 思路)。– **Markerless Multi-view 多目无标记**: 不贴 marker, 更舒适; 依赖 2D pose detection + 三角化。– **Depth Camera 深度相机**: 单台 Kinect 即可, 但精度与范围受限。– **Monocular Video 单目**: OpenPose 等 2D pose → 3D lifting; 最便利、精度最低。– **Sparse Sensor 稀疏传感器**: 6 个 IMU + 学习模型 (TransPose 2021) 估计全身动作, 移动友好。

1.3 一次典型的 mocap session

布场 → 标定 (calibration) → T-pose / range-of-motion → 演员演出 → 求解骨骼 (solving) → 后处理 (去噪、补缺、retargeting)。

2. Motion Data 动作数据

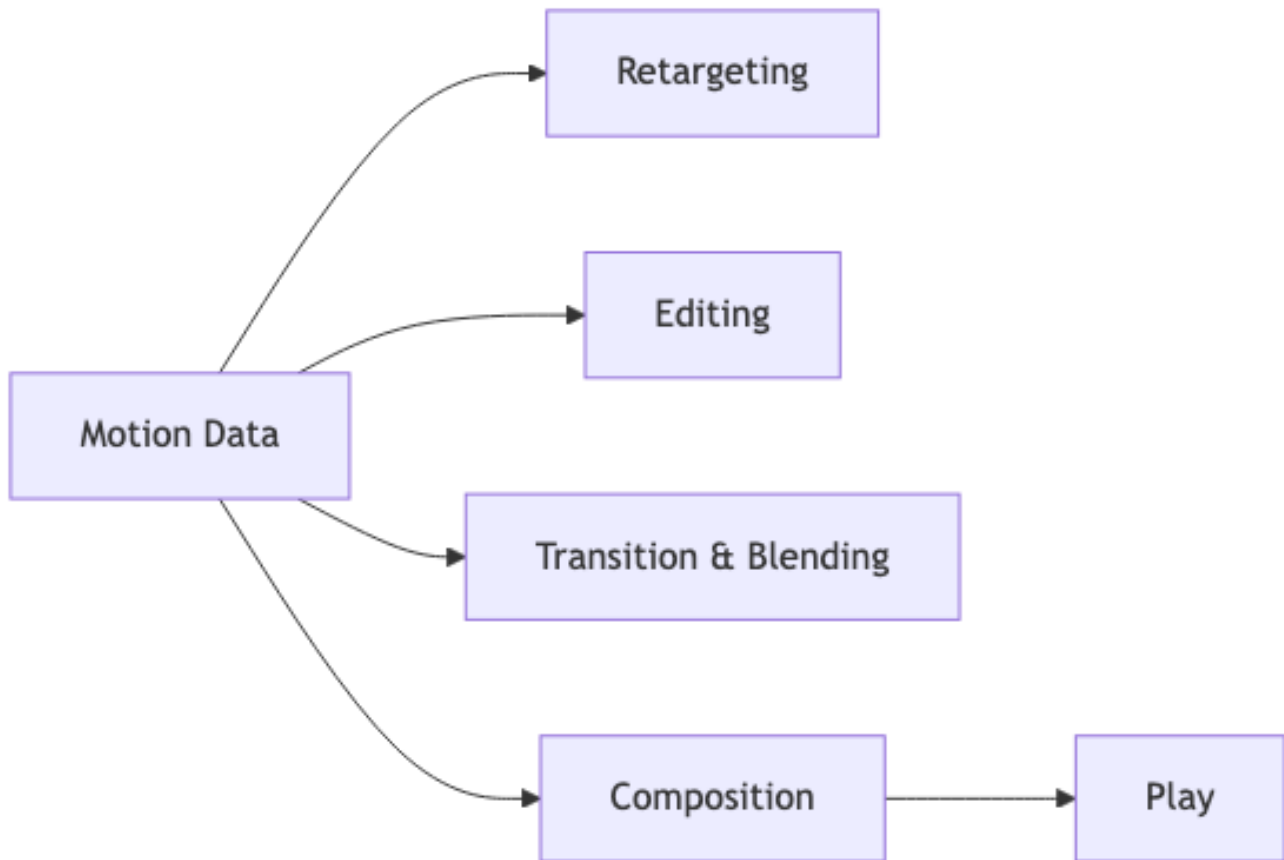
2.1 数据格式

一段动作是一组按时间排列的 pose:

$$\{p_t\}_{t=1,\dots,N}, \quad p_t = (t_0, R_0, R_1, R_2, \dots)$$

- t_0 : 根关节 (root) 的世界位置 (通常是骨盆 pelvis 或髋关节中点)。
- R_0 : 根关节朝向。
- $R_i, i \geq 1$: 每个子关节相对父关节的局部旋转 (local rotation)。
- 旋转的存储格式: 欧拉角 / 四元数 / 6D 表示等。

2.2 数据用法概览



3. Motion Retargeting 动作重定向

3.1 问题定义

把一段在角色 A 上录制的动作，迁移到结构不同的角色 B： – 不同骨骼数 (number of bones) – 不同骨骼名 (bone names) – 不同参考姿态 (reference / T-pose) – 不同骨长比例 (bone ratios) – 不同骨架结构 (skeletal structure)——例如有/没有锁骨，尾巴等。

3.2 一种可行的 pipeline

1. **Map bone names** 关节名映射 (建立 source target 对应关系)。
2. **Scale translations** 根据身高 / 骨长比例缩放平移。
3. **Copy or retarget joint rotations** 复制或转换局部旋转，并修正参考姿态差异。
4. **Postprocessing with IK** 用反向运动学 (Inverse Kinematics) 修复：
 - 脚滑 (foot-skating)
 - 自穿透 (self-penetration)
 - 与环境的接触 (contact-aware retargeting, Villegas 2021)

[NOTE] 工业实现

– Autodesk Maya 的 **HumanIK** – Unreal Engine 5 的 **IK Rig + IK Retargeter** – Ubisoft Toronto 的 **IK Rig** 原型展示了用一组中性 control rig + 描述骨架”语义”实现跨角色重定向。

4. Motion Editing 动作编辑

[NOTE] Motion Displacement Mapping

Lee & Shin 1999 —在原始关键帧动作 p_t 上叠加一条平滑的 displacement 曲线 d_t ，使得在指定帧满足约束（脚踏指定位置、手摸到物体），其余时刻平滑过渡：

$$\tilde{p}_t = p_t \oplus d_t$$

$\oplus$ 在旋转上是组合，在位置上是相加。这是一种层次化（hierarchical）编辑思想。

4.1 几何方法 (Geometric Editing)

把动作视作一条“骨骼”曲线 / 曲面”，借鉴几何处理工具：– **Laplacian operator** 拉普拉斯算子： $L(x_i) = x_i - \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} x_j$ ，保持局部形状的同时整体变形（Choi 2011 —钻进狭窄空间的角色）。– **Spatial relationship preserving** (Ho 2010) 保持角色与障碍 / 其他角色的相对空间关系。

5. Motion Transition 动作过渡

5.1 朴素拼接的问题

直接把片段 A 的最后一帧和片段 B 的第一帧首尾相连，会出现：– **Pose 不连续 (discontinuity)**：手脚突然跳变。
– **Root 位置 / 朝向不连续**：角色瞬移或转身突变。

5.2 线性混合 (Linear Blend)

在两段间选一段 transition window，逐帧线性插值：

$$p_t = (1 - t) p_0^{(i)} + t p_1^{(i)}, \quad t \in [0, 1]$$

更平滑的版本用一条单调过渡曲线 $\phi(t)$ （典型为 smoothstep $3t^2 - 2t^3$ 或 cosine ease）：

$$p_t = (1 - \phi(t)) p_0^{(i)} + \phi(t) p_1^{(i)}$$

[WARN] 旋转不能直接线性插值

局部 joint rotation R_i 必须用 **Slerp / Squad / log-quat blending**，否则会经过非单位四元数（速度时快时慢、可能反向）。位置 t_0 才用线性插值。

5.3 “Facing Frame”朝向坐标系

把角色 root 的旋转拆成“水平转动 + 倾斜”两部分，blending 时只对齐水平朝向，避免角色躺着或仰头被混乱旋转。

[NOTE] Facing Frame 定义

一个水平随角色平移、其中一轴指向角色”朝向方向 (facing direction)“的局部坐标系：

$$R_{\text{face}} = \theta e_y, \quad t_{\text{face}} = (t_x, 0, t_z)$$

即仅含绕 y 轴的旋转和水平平移，垂直分量被剥离。

朝向 R 的几种常见定义：– 让世界 z 轴对齐角色头顶向前的方向。– 让世界 x 轴对齐肩 – 髋两条向量的平均（解决大幅度上身扭转时的歧义）。– 把根旋转分解成 $R_0 = R_y R_{xz}$ ，取其中的 R_y 。

5.4 旋转分解 $R = R_y R_{xz}$

把任意 3D 旋转拆为”水平转动 (yaw) + 翻倒 (tilt)“：

$$R = R_y R_{xz}, \quad R_y = \theta_y e_y, \quad R_{xz} = \theta u_{xz}, \quad u_{xz} = (u_x, 0, u_z)$$

构造方法：1. 先找把局部 y 轴对齐到全局 y 轴的旋转 R' （旋转轴 $\in xz$ 平面）。2. 则 $R_y = R' R$ （把”翻倒”扣掉后剩下的就是 yaw）。3. 反推 $R_{xz} = R_y^T R$ 。

5.5 Root 对齐：两段动作的全局变换

两段 motion 的 root 状态分别为 (R_0, t_0) 与 (R_1, t_1) 。把第二段平移 / 旋转到与第一段衔接的”全局变换” (R, t) ：

$$R^{(i)} = R_0 R_1^T R_1^{(i)}, \quad t^{(i)} = R_0 R_1^T (t_1^{(i)} - t_1) + t_0$$

[TIP] 直观解释

第二段所有帧先减掉自身起点 t_1 平移到原点，再旋转 $R_0 R_1^T$ （“把 R_1 转到 R_0 ”），最后平移到 t_0 。仅作用 yaw 部分（用 facing frame）即可避免引入翻滚。

5.6 Path Fitting 路径拟合

当用户指定一条期望路径（角色应当沿曲线行走）时，把数据库中各段动作的 root 轨迹拟合 / 变形到该路径上——是 motion graph / matching 在做导航控制时的常见后处理。

5.7 No-Root-Translation / No-Root-Rotation 数据

许多数据集（如 in-place 跑步动画）会**移除 root 平移**或**移除 root 平移 + 旋转**：– 优点：数据更紧凑，可直接 looping，便于跨片段对齐。– 使用时再根据用户控制或物理仿真补回来——例如把 GTA 角色的 in-place 动作通过物理积分还原成”真实在世界中走动”的轨迹。

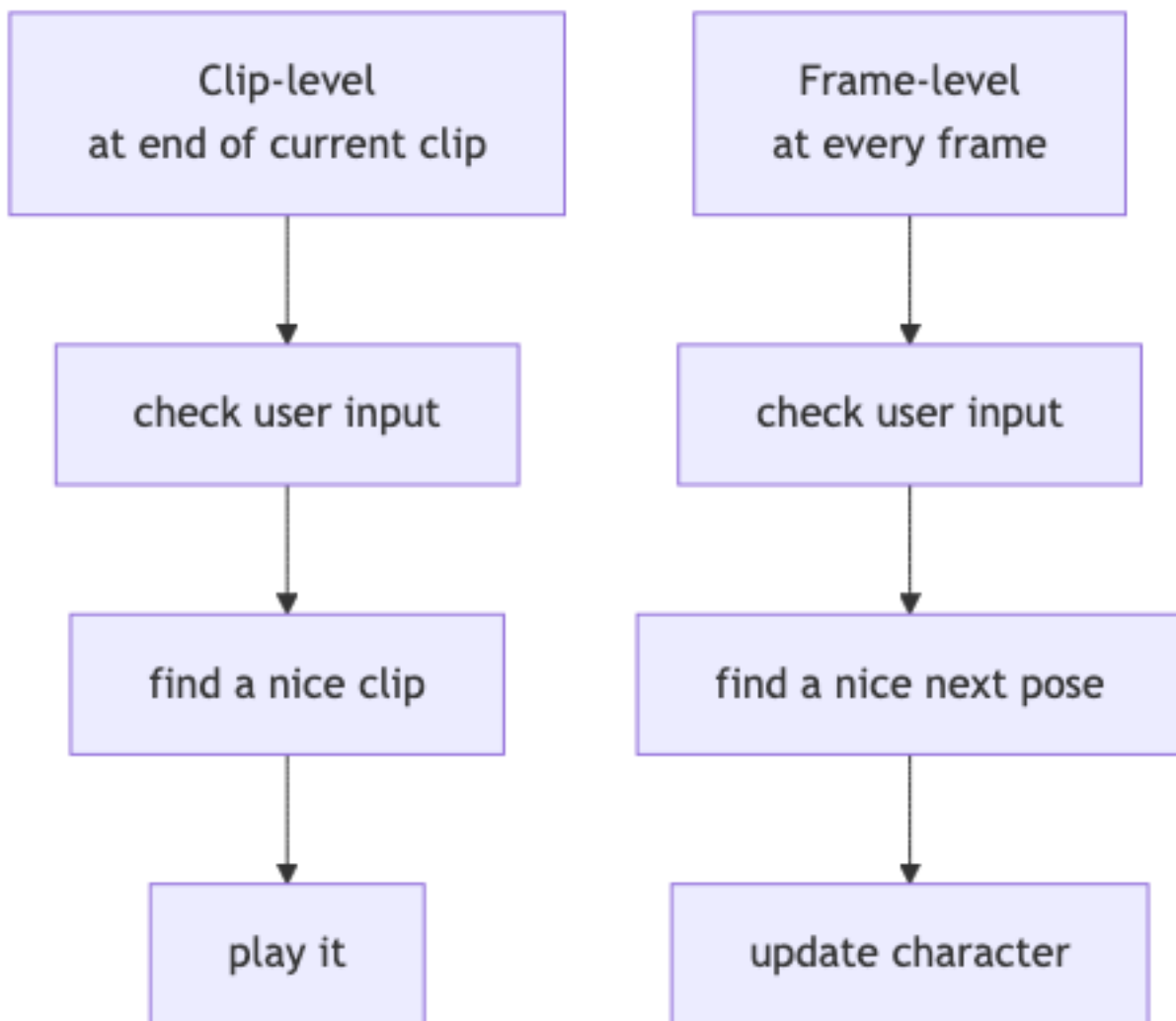
6. Motion Composition 动作组合

[NOTE] 目标

根据用户控制 / 环境物体 / 其他角色计算性地实时生成动作，让数据驱动的运动具有交互性。

两条主线：

范式	控制频率	代表方法
Clip-level ：以片段为单位	每个 clip 末尾决策一次	Motion Graph / State Machine
Frame-level ：每帧决策	每帧检索 / 求解	Motion Fields / Motion Matching

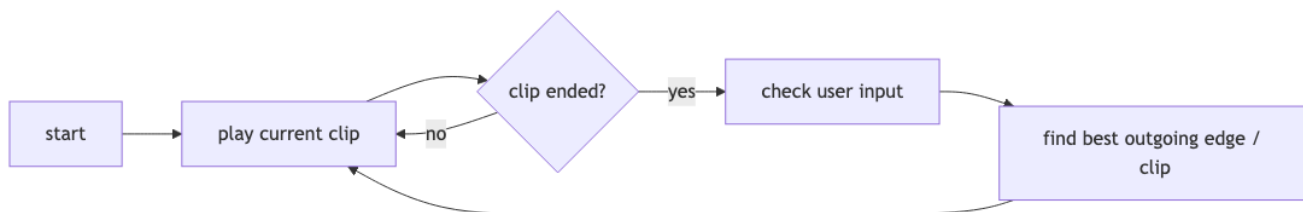


7. Motion Graph 动作图

7.1 结构

把动作合成视为状态机 (state machine) / 有向图：– 节点 (nodes)：motion clips (“走”、“跑”、“挥拳”等)。– 边 (edges)：clip 之间允许的过渡 (pose 足够接近才能连)。

7.2 运行流程



只有在当前 clip 结束时才检查用户输入并选下一段——因此响应延迟约等于 clip 长度。

7.3 构建：自动找过渡边

对每对 (clip A 第 i 帧, clip B 第 j 帧), 定义 pose 距离

$$D(i, j) = \|\text{pose}_A(i) - \text{pose}_B(j)\|^2$$

(位置 + 速度 + 关节旋转的加权和)。低于阈值的 (i,j) 即为可行的过渡点；过渡时再用 §5 的 transition 把两边混合起来。

7.4 控制：图上的搜索

给定用户目标（去某点、按某方向走），在 motion graph 上做 A* / 动态规划，挑选一条 cost 最低的 clip 序列。Heck & Gleicher 2007 的 **Parametric Motion Graphs** 进一步把“同类动作族”参数化（不同方向 / 不同速度），让控制更连续。

[WARN] 限制

– 响应慢：等当前 clip 走完才能换。– 覆盖差：图的节点 / 边由数据库决定，没采到的过渡无法做。

8. Motion Fields 与 Motion Matching

为提速响应，把决策从“clip 末尾”推到“每帧”。

8.1 Motion Fields (Lee et al. SIGGRAPH 2010)

[NOTE] 算法

在每一帧：1. 检查用户输入；2. 在数据库中找到当前状态的 N 个最近邻 (nearest neighbors)；3. 按用户输入对这些邻居加权混合，得到下一帧 pose；4. 更新角色。

权重学习用强化学习 (RL)：把“找邻居 + blending 权重”视为 action，回报由是否符合用户控制 / 是否平滑决定。优点：响应即时、轨迹连续；缺点：训练复杂、运行时还要做 RL 推断。

8.2 Motion Matching

工业界（Ubisoft《荣耀战魂》《刺客信条》等）的简化方案：

[NOTE] 算法

在每一帧：1. 读取用户输入（如目标速度、未来路径）；2. 在数据库中用一个特征向量距离找最相似的 pose；3. 平滑过渡到这个 pose。

$$\text{next_pose} = \arg \min_{i \in \text{Dataset}} \|x_{\text{curr}} - x_i\|$$

其中 x 是 feature vector（不是 raw 全身 pose）。

8.3 Feature Vector 特征向量

一组实践中常用的特征：

- 末端关节 (end effectors) 相对 root 的线 / 角速度
- root 的线速度 / 角速度
- 未来朝向 / 位置 (future heading)：例如 0.5 s, 1.0 s, 1.5 s 后的 root 位置 / 朝向——让“轨迹意图”参与匹配

- 脚接触 (foot contacts) 0/1 标签——避免在腾空帧匹配到落地帧
- 末端关节相对 root 的位置

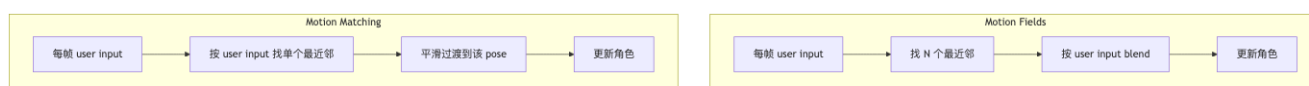
特征加权调参可以平衡”动作风格 vs 控制响应”。

8.4 工程优化

[TIP] 让 Motion Matching 跑得起来

– 不要每帧都搜：每隔若干帧（如 5 ~ 10 帧）检索一次，期间用平滑过渡；– 平滑过渡：用 **Inertialized blending** (Daniel Holden, “spring-roll-call”)——以当前姿态为初值，用临界阻尼弹簧追到目标姿态，时间常数极短；– 数据结构：KD-tree 加速最近邻；– **Dance card**：把数据库瘦身成高质量、覆盖均衡的”舞池”，去掉冗余帧。

8.5 Motion Fields vs Motion Matching 对比



维度	Motion Fields	Motion Matching
决策粒度	每帧	每帧（实际隔几帧搜一次）
邻居数	N 个并 blend	1 个 + 平滑过渡
权重	RL 学到	设计 feature + L2 距离
工程难度	高（训练 RL）	低（只调 feature 与 KD-tree）
工业采纳	学术	Ubisoft, EA 等大量在用

9. 各方法横向对比

方法	决策粒度	数据组织	控制响应	主要工具	局限
Keyframe	—	手工 pose	实时	DCC 软件	不真实，工作量大
Rotoscoping	帧级描线	真人视频	离线	描线	仅 2D，一次性
Motion Graph	clip 末尾	图 / 状态机	慢	A* on graph	覆盖受数据集限
Parametric Motion Graph	clip 末尾	参数化 clip 簇	中	插值 + 搜索	参数化建模复杂
Motion Fields	每帧	状态空间 + RL	快	RL 策略	训练困难
Motion Matching	每帧	KD-tree + features	快	距离 + inertialized blend	数据需均衡 / “dance card”

10. 一帧 Motion Matching 伪代码

```
# 离线
build_dataset(motions)

# 录制 + 清洗
```



```

for frame in motions:
    feat[frame] = extract_features(frame)    # 末端速度, 未来 heading, foot contact, ...
kd = build_kdtree(feat)

# 运行时 (每帧或每 K 帧)
input  = read_user_input()                 # 期望速度 / 路径
x_curr = current_features(input)            # 当前特征 + 用户期望未来轨迹
i_best = kd.nearest(x_curr)                 # 最近邻
target = motions[i_best]
pose   = inertialized_blend(pose, target)   # 平滑过渡
render(pose)

```

记号速查

符号	含义
$p_t = (t_0, R_0, R_1, \dots)$	第 t 帧 pose: root 位置 + 各关节旋转
t_0, R_0	root 平移 / 旋转 (世界系)
$R_i, i \geq 1$	第 i 关节相对父关节的局部旋转
$R = R_y R_{xz}$	root 旋转分解为 yaw + tilt
Facing Frame	$R = \theta e_y, t = (t_x, 0, t_z)$
$\phi(t)$	过渡混合曲线 (线性 / smoothstep / cosine)
$D(i, j)$	pose 距离, 用于 motion graph 自动建边
x	Motion Matching 的 feature vector
N	Motion Fields 的近邻数
KD-tree	加速最近邻搜索的数据结构
Inertialized blending	临界阻尼弹簧式平滑过渡

Lecture 06 —Learning-based Animation

[TIP] 学习指南

本节核心：从“人类运动是高维空间中的低维流形”出发，串起 **统计模型** → **自回归模型** → **生成模型** 三大类数据驱动动画方法。前置知识：线性代数（特征分解、SVD）、概率论（高斯分布、条件概率链式法则、KL 散度）、神经网络基础（MLP、SGD、反向传播）。重点关注：① PCA 既是降维工具又是高斯先验 ② 自回归模型 + Phase 解决 ambiguity ③ VAE/VQ-VAE/Diffusion 三类生成式 motion prior 的训练目标 ④ “motion as a foreign language”这一统一视角。

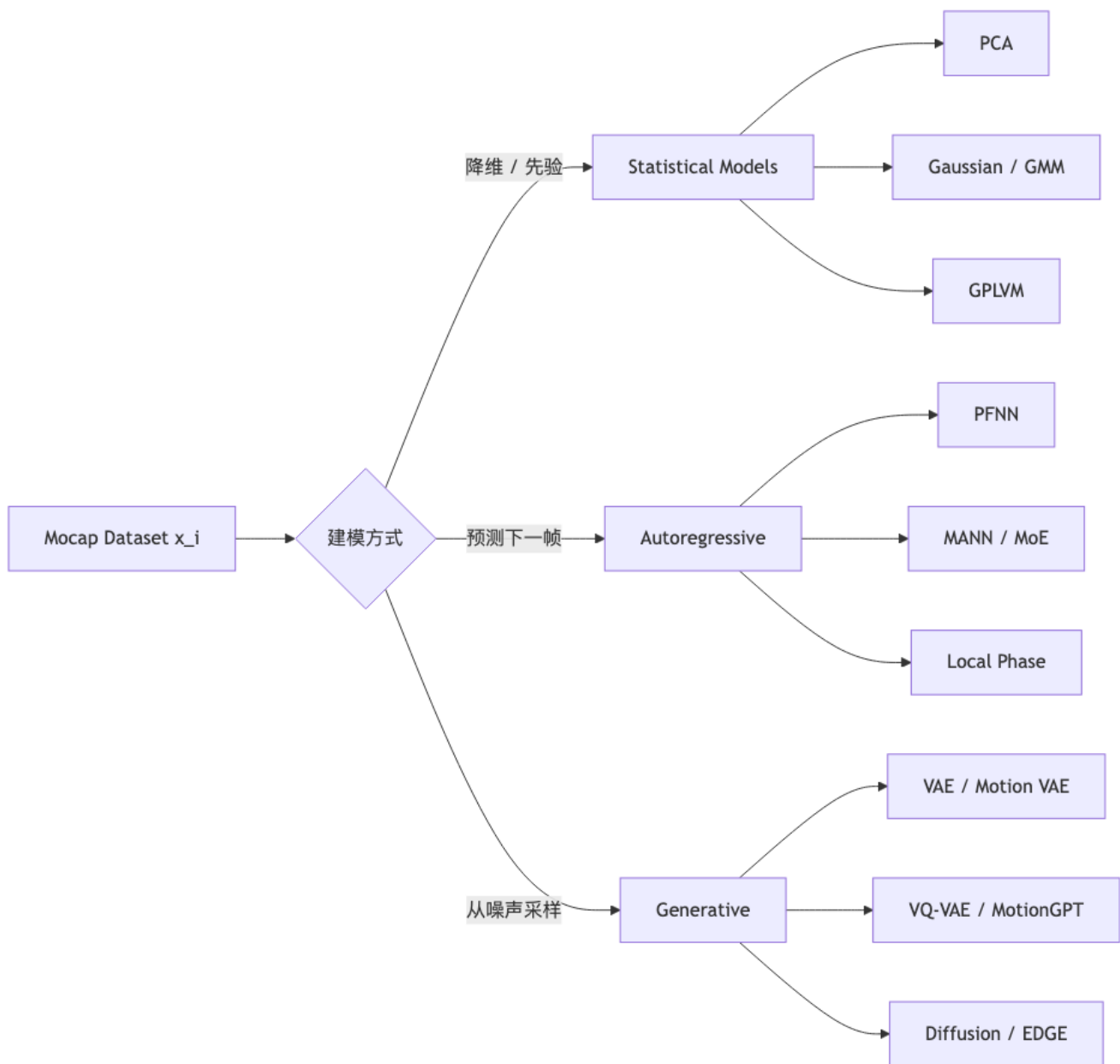
0. 核心问题 / Motivation

如何让计算机自动产生 自然 的人体运动 (natural-looking motion)? 人手动 keyframing 与 motion graph (Lec 04) 的拼接都需要大量 mocap 数据并且只能复述已有片段。**Learning-based animation** 试图把数据集 $\{x_i\}$ 蒸馏成一个分布 $p(x)$ 或一个可调用的函数 f ，从而：

1. 给一个姿态 x 评判它“像不像人”(motion prior, IK 正则项)；
2. 给定历史 $x_{t-1}, x_{t-2}, \dots$ 与控制信号 z_t ，自回归预测下一帧 x_t ；
3. 从噪声/文本/音乐采样出全新的运动序列。

[NOTE] 一图全览 / Big picture

Statistical: PCA, GMM, GPLVM $\implies$ Autoregressive: PFNN, MoE $\implies$ Generative: VAE, VQ-VAE, Diffusion
Lec 06 上半 Lec 06 中 Lec 06 下半



1. 人体运动的低维性 / Low-dimensionality

关键观察：虽然一个姿态 $x \in \mathbb{R}^N$ 维度很高 ($N \approx 60+$)，但所有“自然”姿态只占据其中的一小块流形 (manifold)：

- 协调的四肢运动 (coordinated arm/leg movement)
- 肌肉骨骼结构 (musculoskeletal structure)
- 物理约束 (laws of physics)

[!important] 中心思想 整个 pose 空间 $\mathbb{R}^N$ 的体积巨大；自然姿态的流形 $\mathcal{M}$ 维度远小于 N 。所有 learning-based 方法的本质是把 $\mathcal{M}$ 显式或隐式地参数化。

2. PCA 与运动先验 / Principal Component Analysis

2.1 PCA 的几何动机

给定数据 $\{x_i\} \subset \mathbb{R}^N$, 找单位方向 u ($\|u\| = 1$) 使投影 $w_i = x_i \cdot u$ 的方差最大:

$$\max_u \frac{1}{N} \sum_i (w_i - \bar{w})^2, \quad \text{s.t. } \|u\| = 1$$

把数据中心化后堆成

$$X = \begin{bmatrix} (x_0 - \bar{x})^T \\ (x_1 - \bar{x})^T \\ \vdots \\ (x_N - \bar{x})^T \end{bmatrix}$$

可证 u 是 $X^T X$ 最大特征值对应的特征向量; 继续取次大、次次大等等就得到一组正交基 $\{u_k\}$ 。

2.2 协方差与 SVD

$$\Sigma = X^T X = U \begin{bmatrix} \sigma_1^2 & & & \\ & \sigma_2^2 & & \\ & & \ddots & \\ & & & \sigma_N^2 \end{bmatrix} U^T, \quad U = [u_1, u_2, \dots, u_N]$$

[NOTE] 重建公式

$$x_i = \bar{x} + \sum_{k=1}^N w_{i,k} u_k, \quad w_{i,k} = (x_i - \bar{x}) \cdot u_k$$

取前 n 项 (e.g. 解释 95% 方差) 得到截断重建 $\tilde{x}_i = \bar{x} + \sum_{k=1}^n w_{i,k} u_k$ 。

2.3 PCA 用作 motion prior

对走路 mocap 做 PCA, 前几个主成分恰好对应 走路相位 (gait cycle)。一个姿态 x 的”自然程度”可以用其在主轴上的 Mahalanobis 距离 量化:

$$\text{naturalness}(x) \propto \exp\left(-\frac{1}{2} \sum_k \frac{((x - \bar{x}) \cdot u_k)^2}{\sigma_k^2}\right)$$

即”分量越落在小 σ_k 方向上, 姿态越奇怪”。

2.4 Character IK with motion prior

朴素 IK 只约束末端位置:

$$F(\theta) = \frac{1}{2} \sum_i \|f_i(\theta) - \tilde{x}_i\|_2^2 + \frac{\lambda}{2} \|\theta\|_2^2$$

加入 reference pose θ_0 : 第二项变成 $\frac{\lambda}{2} \|\theta - \theta_0\|^2$ 。

加入 PCA motion prior:

$$F(\theta) = \frac{1}{2} \sum_i \|f_i(\theta) - \tilde{x}_i\|_2^2 + \frac{w}{2} \sum_k \frac{((\theta - \bar{\theta}) \cdot u_k)^2}{\sigma_k^2}$$

把第二项重写为

$$-w \log \prod_k \exp\left(-\frac{1}{2} \frac{((\theta - \bar{\theta}) \cdot u_k)^2}{\sigma_k^2}\right) = -w \log p(\theta)$$

[!important] PCA = 高斯先验 在 PCA 主轴下假设各方向独立高斯, $p(\theta) = \prod_k \mathcal{N}(0, \sigma_k^2)$ 。IK + PCA 等价于 MAP 估计: 数据项 = 似然, PCA 项 = 先验。Grochow et al. SIGGRAPH 2004 (Style-Based IK) 是奠基工作。

3. 高斯模型族 / Gaussian Family

3.1 单高斯 + MLE

$$p(x) = \mathcal{N}(\bar{x}, \Sigma) = \frac{1}{\sqrt{(2\pi)^k |\Sigma|}} \exp\left(-\frac{1}{2}(x - \bar{x})^T \Sigma^{-1}(x - \bar{x})\right)$$

最大似然估计 (MLE):

$$\bar{x} = \frac{1}{N} \sum_i x_i, \quad \Sigma = \frac{1}{N} X^T X$$

代入 $\Sigma = U \text{diag}(\sigma_k^2) U^T$ 即恢复 §2.3 的 PCA 概率解释。

3.2 高斯混合 (GMM)

单高斯是 单峰 的——对走路 vs. 跳跃这种多模态数据无能为力。GMM:

$$p(x) = \sum_i \phi_i \mathcal{N}(\mu_i, \Sigma_i), \quad \sum_i \phi_i = 1$$

[NOTE] 代表工作

Min et al. SIGGRAPH 2009 用 GMM 做 motion synthesis, 可以同时容纳走、跑、跳等多种风格。

3.3 高斯过程 (GPLVM)

把每个高维姿态 x 映射到低维潜变量 z , 假设 x 是 z 的 GP 函数。Levine et al. SIGGRAPH 2012 用 GPLVM 学到走路风格流形, 支持 IK 与连续插值。

[WARN] 高斯方法的硬伤

高斯/GMM/GP 都假设了相对简单的解析形式; 对真实多任务多风格数据集不够灵活。进入神经网络时代: Holden 2017 PFNN, Lee 2019 Multi-Objective, Starke 2020 Local Motion Phases, Henter 2020 MoGlow, Holden 2020 Learned Motion Matching

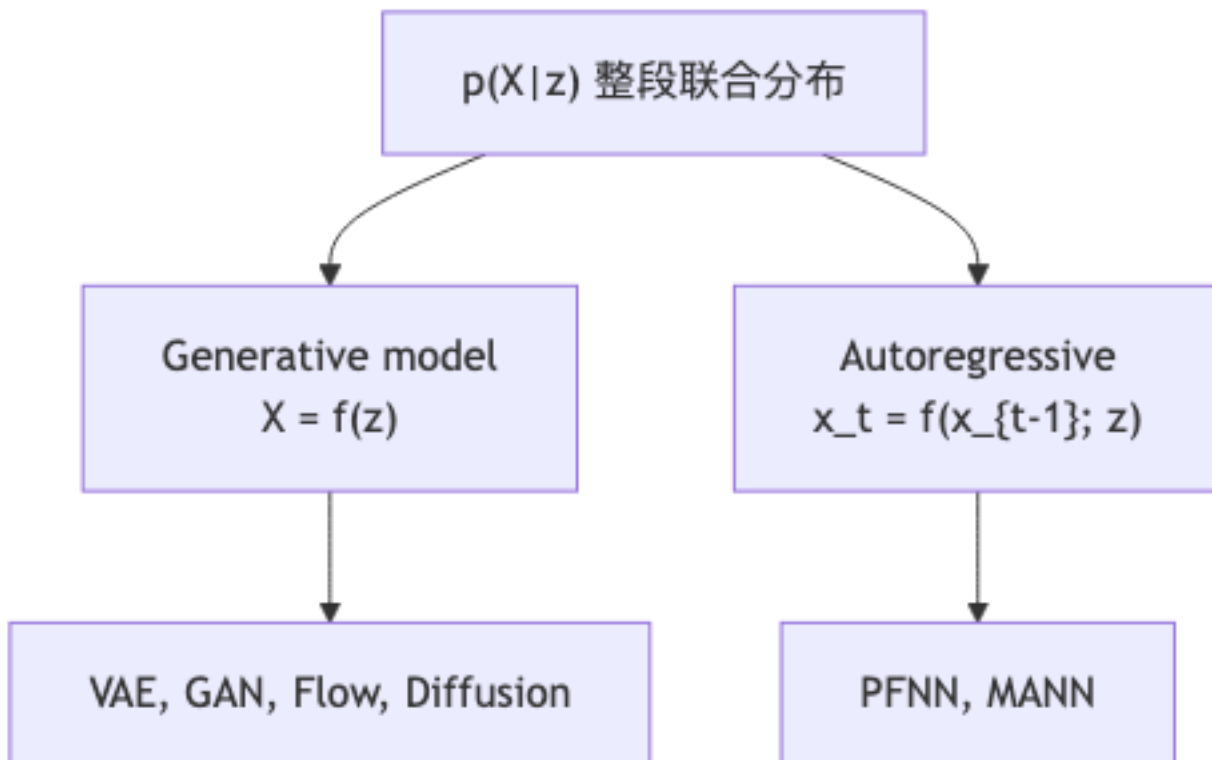
4. 学习运动模型的两视角 / Two Perspectives

把每帧姿态记为 $x_t = (t_0, R_0, R_1, R_2, \dots)$, 整段运动 $X = (x_1, \dots, x_T)$ 。可以建模 联合分布 $p(X|z)$, 其中 z 是控制参数 / 隐变量。

由条件概率链式法则:

$$p(X|z) = p(x_1) \prod_t p(x_t | x_{t-1}, \dots, x_1; z)$$

对每一项再做 Markov 假设 $p(x_t | x_{t-1}, \dots, x_1; z) \approx p(x_t | x_{t-1}; z)$:



[!important] 两种视角的对偶 **生成式**: 直接采样整段 $X = f(z)$ —— 适合 text-to-motion, music-to-dance; **自回归**: 一步一步 roll out —— 适合实时交互角色控制 (PFNN 90 FPS)。

5. 自回归模型 / Autoregressive Model

5.1 朴素形式

假设 Markov 一阶: $x_t = f(x_{t-1})$ 。给定一组 transitions $\{(x_{t-1}, x_t)\}$, 训练 MLP f_θ :

$$F(\theta) = \sum_{(x_{t-1}, x_t)} \|f(x_{t-1}; \theta) - x_t\|^2$$

SGD 更新:

$$\theta \leftarrow \theta - \alpha \nabla_\theta F(\theta), \quad \nabla_\theta F \approx \sum_i \nabla_\theta \|f(x_{t-1}^i; \theta) - x_t^i\|^2$$

5.2 Ambiguity 问题

[WARN] 一对多的歧义

在走路周期中，同一个 x_{t-1} 可能对应多种合理的 x_t （左脚要落地 vs 右脚要落地）。回归损失会把所有候选平均，输出 → 滑步、抽搐、收敛到平均姿态。

5.3 解决：Hidden / Phase 变量

$$x_t = f(x_{t-1}; z_t)$$

让 z_t 注入额外信息消除歧义。最经典的选择是 **phase** $\phi \in [0, 1)$ ——走路周期中的相位。

6. PFNN: Phase-Functioned Neural Networks (Holden et al. SIGGRAPH 2017)

6.1 输入

$z_t = (c_t, \phi_t)$ ，其中 c_t 是控制参数（轨迹、地形、目标速度）， ϕ_t 是当前 gait 相位。

6.2 关键想法：网络权重随 phase 变化

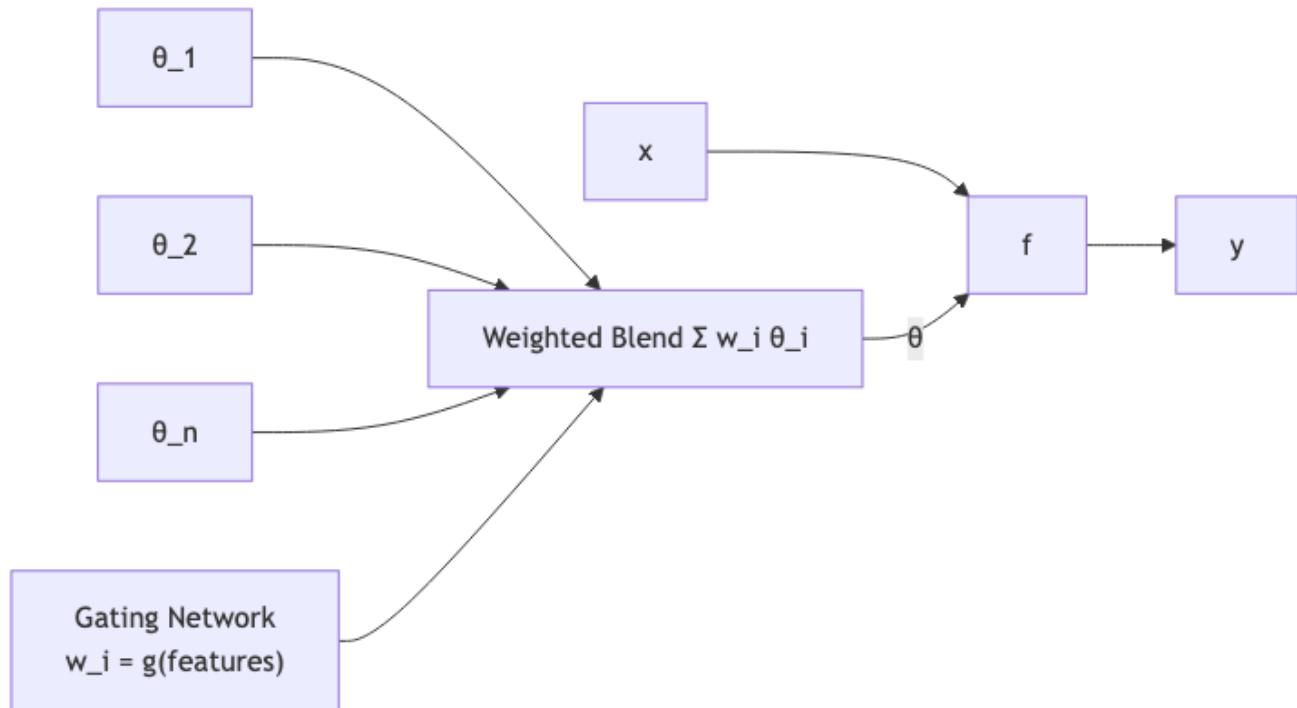
不是把 ϕ 当作普通输入特征（容易被忽略），而是让 整套权重 θ 本身是 ϕ 的函数：

$$x_t = f(x_{t-1}; c_t, \theta_t), \quad \theta_t = \sum_i w_i(\phi_t) \theta_i$$

权重在四个 control points $\theta_1, \dots, \theta_4$ 之间用 **Catmull-Rom cubic spline** 插值：

$$\begin{aligned} \theta(\phi) = & \theta_2 + \phi \cdot \frac{1}{2}(\theta_3 - \theta_1) \\ & + \phi^2(\theta_1 - \frac{5}{2}\theta_2 + 2\theta_3 - \frac{1}{2}\theta_4) \\ & + \phi^3(-\frac{1}{2}\theta_1 + \frac{3}{2}\theta_2 - \frac{3}{2}\theta_3 + \frac{1}{2}\theta_4) \end{aligned}$$

6.3 Mixture of Experts 视角



形式	公式	特点
Output-blended MoE	$y = \sum_i w_i f(x; \theta_i)$	多个网络输出加权平均
Weight-blended MoE	$y = f(x; \sum_i w_i \theta_i)$	权重直接混合, PFNN 即此

PFNN 的”门控权重” $w_i(\phi)$ 来自 Catmull–Rom 系数；**MANN / Local Motion Phases** 则用一个可学习的 gating network 替代 phase，自动发现局部相位。

6.4 演进路线

[NOTE] Phase 变量的演进

– **PFNN (2017)**: 全局单 phase，适合周期 locomotion – **MANN (2018)**: 学习 gating network，无需手工 phase，可处理四足 – **NSM (2019)**: 加入 environment encoding & action labels – **Local Motion Phases (2020)**: 每个肢体一个局部 phase，处理篮球等异步多接触 – **Starke et al. 2022**: deep phase 自编码器从数据自动提取相位流形

7. 从 PCA 到 Autoencoder

7.1 PCA 重新审视

PCA 重建可写成”编码 + 解码”:

$$z_k = (x - \bar{x}) \cdot u_k, \quad x' = \bar{x} + \sum_k z_k u_k$$

等价于矩阵形式 $z = U^T(x - \bar{x})$, $x' = \bar{x} + Uz$ —— 一个 线性 自编码器。

7.2 (Deterministic) Autoencoder

把线性 U, U^T 换成非线性神经网络 $\mathcal{E}, \mathcal{D}$:

$$z = \mathcal{E}(x), \quad x' = \mathcal{D}(z), \quad \min_{\theta, \phi} \|\mathcal{D}(\mathcal{E}(x; \theta); \phi) - x\|$$

[NOTE] 代表工作

Holden et al. 2015 Learning Motion Manifolds with Convolutional Autoencoders ——第一次用 conv-AE 把 mocap 嵌到流形上，潜空间里做编辑后解码回真实运动。

7.3 AE 的痛点

潜空间 z 没有正则化，任意采样 z 解码出来不一定是合理姿态——潜空间是“有洞的”。需要让 z 服从某种已知分布。

8. Variational Autoencoder (VAE)

8.1 目标：让 z 服从先验

强制 $z \sim \mathcal{N}(0, I)$ ，从而 任何 标准正态采样都能解码出有效运动。

8.2 MLE & ELBO 推导

数据集 $D = \{x_i\}$ 的 log-likelihood:

$$\log p(x) = \log \int_z p(x|z)p(z) dz$$

引入变分后验 $q(z|x)$:

$$\log p(x) = \log \int_z q(z|x) \frac{p(x|z)p(z)}{q(z|x)} dz$$

由 Jensen 不等式 ($\log \mathbb{E} \geq \mathbb{E} \log$):

$$\log p(x) \geq \int_z q(z|x) \log \frac{p(x|z)p(z)}{q(z|x)} dz = \underbrace{\mathbb{E}_{q(z|x)}[\log p(x|z)]}_{\text{重建项}} - \underbrace{D_{\text{KL}}(q(z|x) \| p(z))}_{\text{正则项}}$$

[!important] ELBO (Evidence Lower BOUND)

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - D_{\text{KL}}(q(z|x) \| p(z))$$

KL 散度衡量两个分布的差异: $D_{\text{KL}}(Q \| P) = \int q \log(q/p)$ 。最大化 ELBO 同时 (a) 把 x 重建好, (b) 把后验 $q(z|x)$ 推向先验 $p(z)$ 。

8.3 高斯参数化

设 $p(x|z) \sim \mathcal{N}(\mathcal{D}(z), \sigma_D)$, $q(z|x) \sim \mathcal{N}(\mathcal{E}(x), \sigma_z)$ 。Monte Carlo 估计重建项：

$$\mathbb{E}_{q(z|x)}[\log p(x|z)] \approx -\frac{1}{N} \sum_{i=1}^N \frac{\|x_i - \mathcal{D}(z_i)\|^2}{\sigma_D^2} + C$$

最终损失：

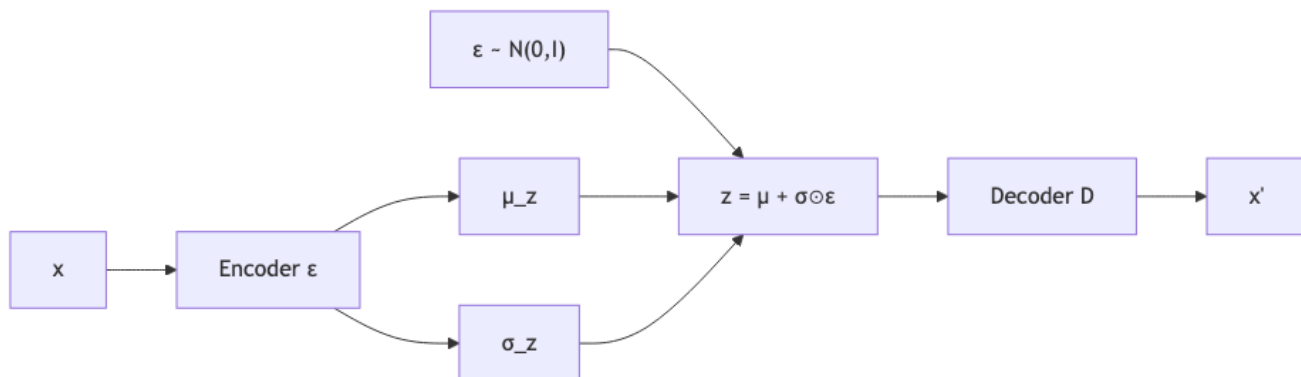
$$\min_{\theta, \phi} \|\mathcal{D}(z; \phi) - x\|^2 + D_{\text{KL}}(\mathcal{N}(\mu_z, \sigma_z) \parallel \mathcal{N}(0, I))$$

8.4 Reparameterization trick

直接对 $z \sim \mathcal{N}(\mu_z, \sigma_z)$ 采样会切断梯度。改写：

$$z = \mu_z + \sigma_z \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

则梯度可以从 z 反传到 μ_z, σ_z 。



8.5 应用

工作	用法
Image VAE	latent space 编辑（笑脸/年龄/眼镜插值）
Motion VAE (Ling et al. 2021)	learn motion latent → 训练 RL policy 在 latent 上 control
ControlVAE (Yao et al. 2022)	physics-based 角色控制器，统一 kinematic + dynamic

[NOTE] Motion VAE 思路

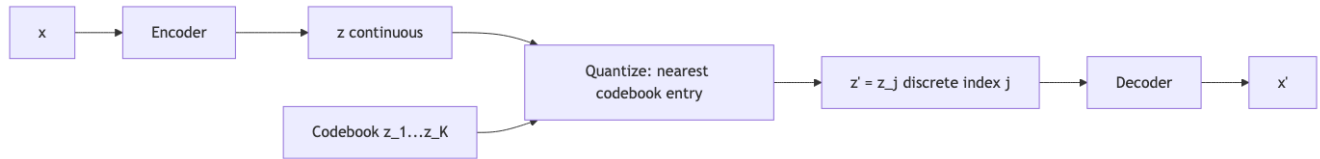
每帧 x_t 由 latent z_t 解码；策略网络 $\pi(z_t | x_{t-1}, \text{goal})$ 在 latent 空间里输出动作。把高维 motion 控制问题降到 32–64 维潜空间，RL 训练显著更稳。

9. VQ-VAE: Vector-Quantized VAE

9.1 想法：把潜变量”离散化”

VAE 用连续高斯先验，VQ-VAE 用一个 **codebook** $\{z_j\}_{j=1}^K$ (如 512 个码字)，把编码器输出 z snap 到最近码字：

$$z' = z_j, \quad j = \arg \min_i \|z - z_i\|$$



9.2 损失函数

argmin 不可导，用 straight-through 估计 + commit loss：

$$\mathcal{L} = \|\mathcal{D}(z') - x\|^2 + \|\text{sg}[z] - z'\|^2 + \beta \|z - \text{sg}[z']\|^2$$

其中 $\text{sg}[\cdot]$ 是 stop-gradient。第二项更新 codebook 让 z' 接近 z ；第三项 ($\beta \approx 0.25$) 让 encoder 不要漂移太远。

9.3 Sequential VQ Representation

把整段运动编码成一串离散 token：

..., 3, 5, 3, 8, 9, 0, 1, 0, 0, 2, ...

这正是 NLP 里的 token 序列！于是可以训练 GPT-style transformer 来生成 motion。

9.4 Motion as a Foreign Language

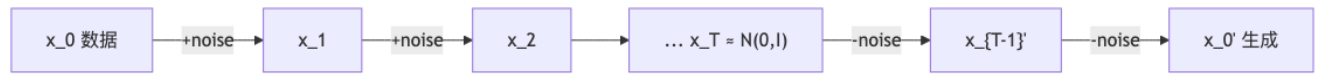
[!important] 统一视角 English Spanish 翻译 English Motion-token-sequence 翻译。文本 caption 是一种语言，VQ-VAE 把 motion 翻成另一种语言——同一个 transformer 可以双向翻译。

工作	贡献
MotionGPT (Jiang et al. 2023)	统一 text motion 的 GPT，在 VQ token 上训练 codebook + physical decoder，让 token 输出物理可执行动作
MoConVQ (Yao et al. 2024)	
Abstract Tasks w/ LLM	用 LLM 把“走方形轨迹”拆成“前走 → 右转 90° → 前走 →”再映射到 token

10. Diffusion Models

10.1 直觉

不去直接学 $p(x)$ ，而是学 如何一步步去噪：



10.2 Forward / Reverse 过程

Forward (固定, 加噪): 在 T 步里把 x_0 慢慢加 Gaussian noise 直到 $x_T \sim \mathcal{N}(0, I)$ 。**Reverse** (待学, 去噪): $p_\theta(x_{0:T})$ 不可解析, 需用网络学习。

10.3 Score-based 视角

把扩散写成 SDE: - Forward SDE: $dx = f(x, t)dt + g(t)dW$ - Reverse SDE: $dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t)d\bar{W}$

[NOTE] 关键观察

反向过程的唯一未知量是 **score function** $\nabla_x \log p_t(x)$ 。学一个网络 $s_\theta(x, t) \approx \nabla_x \log p_t(x)$ 即可。

10.4 训练目标

直接最小化 $\mathbb{E} \|s_\theta(x, t) - \nabla_x \log p_t(x)\|^2$ 不可计算 (p_t 不 tractable)。等价目标:

$$\mathcal{L} = \mathbb{E}_{t, x_0, x_t} \|s_\theta(x_t, t) - \nabla_{x_t} \log p_t(x_t | x_0)\|^2$$

由于 $p_t(x_t | x_0) \sim \mathcal{N}(\alpha_t x_0, \sigma_t^2 I)$:

$$\nabla_{x_t} \log p_t(x_t | x_0) = -\frac{x_t - \alpha_t x_0}{\sigma_t^2} = -\frac{\epsilon}{\sigma_t}$$

其中 $\epsilon \sim \mathcal{N}(0, I)$, $\sigma_t^2 = 1 - \alpha_t^2$, $\alpha_0 = 1, \alpha_T = 0$ 。

10.5 DDPM 训练算法

[!important] 一句话训练 采一个数据 x_0 , 采一个时间 t , 采一个噪声 ϵ , 让网络从 $x_t = \alpha_t x_0 + \sigma_t \epsilon$ 中预测出 ϵ 。

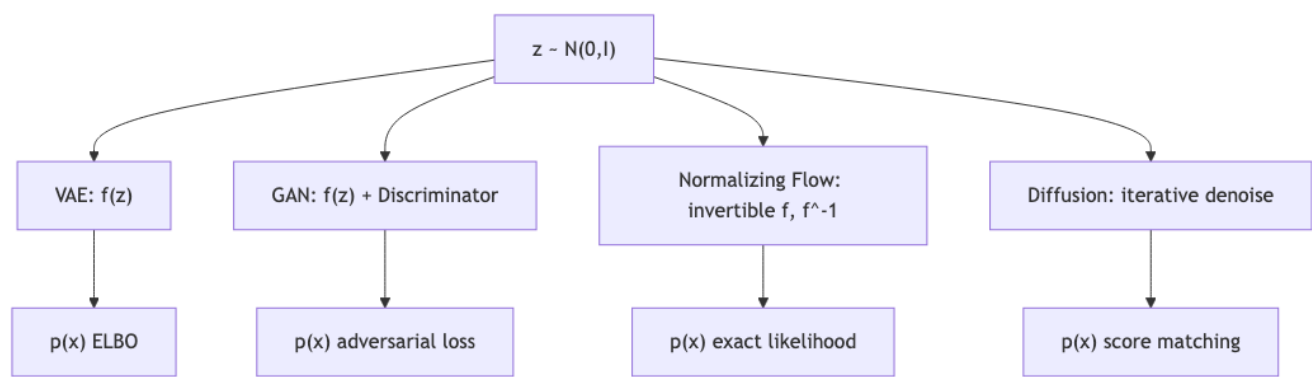
```
repeat:
  x_0 ~ data
  t ~ Uniform{1, ..., T}
  ε ~ N(0, I)
  x_t = α_t · x_0 + σ_t · ε
  L = ||s_θ(x_t, t) - (-ε/σ_t)||²          (or ||ε_θ(x_t, t) - ε||²)
  θ ← θ - η ∇_θ L
```

采样时反过来 roll: 从 $x_T \sim \mathcal{N}(0, I)$ 起, 每步用 s_θ 估计 score, 按 Langevin / DDIM 公式去一点点噪。

10.6 应用

工作	任务
Stable Diffusion / Midjourney	文生图
EDGE (Tseng et al. 2022)	music-to-dance, 可编辑
GestureDiffuCLIP (Ao et al. 2023)	speech-to-gesture, with CLIP latents
Kimodo (NVIDIA)	physics-based diffusion character control

11. 生成模型大全 / Generative Model Zoo



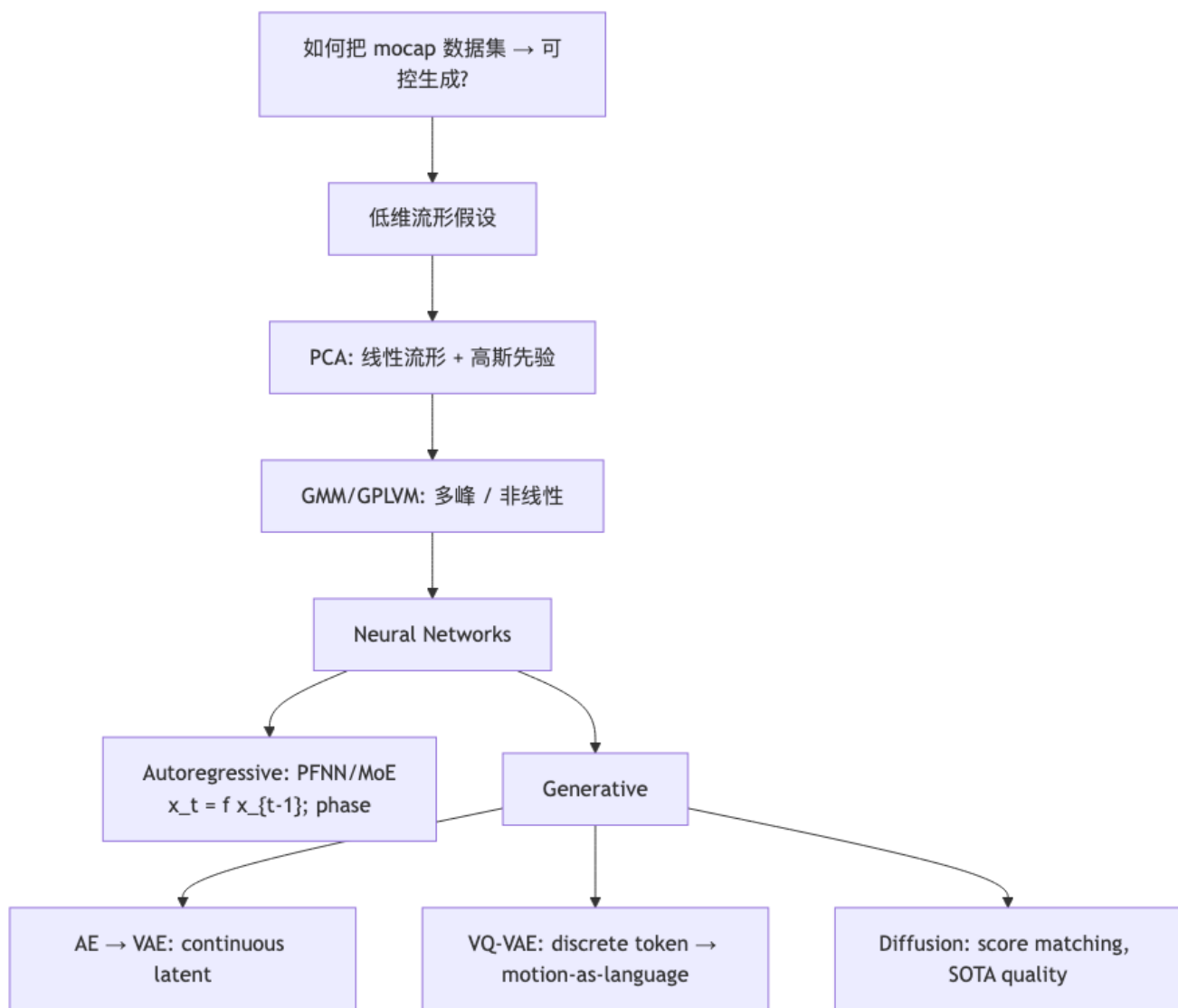
模型	训练目标	采样速度	似然
VAE	ELBO (重建 + KL)	一次前向, 快	下界
GAN	min-max ($\min_G \max_D$)	一次前向, 快	无
Normalizing Flow	exact $\log p(x)$ via Jacobian	一次前向 (但需可逆)	精确
Diffusion	denoising score matching	多步, 慢	ELBO
VQ-VAE + Transformer	重建 + autoregressive token	token 自回归	autoregressive 似然

12. 其他应用 / Other Applications

[NOTE] 数据驱动动画的版图

学到的 motion model 远不止生成: – **Motion Retargeting** —Aberman et al. 2020 Skeleton-aware networks 把动作迁移到不同骨架 – **Style Transfer** —Aberman et al. 2020 从视频抽风格嵌入, 把”愤怒”风格喷到中性走路上 – **Motion In-betweening** —Qin et al. 2022 用 two-stage transformer 给定首尾关键帧补全中间 – **Multi-Objective Control** —Lee et al. 2019 同时满足多个目标 (走到点 + 看相机 + 抓物体) – **Normalizing Flows** —MoGlow (Henter et al. 2020) 概率可控的 motion 合成 – **Learned Motion Matching** —Holden et al. 2020 把 motion matching 数据库压成神经网络

13. 总结 / Summary



[!important] 三句话总结 1. **低维流形** 是一切 learning-based animation 的物理依据；PCA 是它的线性化形式，神经网络是它的非线性扩展。2. **自回归 + phase / gating** 解决了“同一历史多种未来”的歧义，使实时角色控制成为可能 (PFNN 系列)。3. **生成模型** 把整段运动建模为分布：VAE 提供平滑 latent、VQ-VAE 把运动变成可被 GPT 处理的 token、Diffusion 提供最高生成质量；下一讲会看到这些表征如何与 物理仿真 + 强化学习 结合 (DeepMimic / AMP / ASE / ControlVAE)。

[TIP] 与下一讲的衔接

Lec 06 处理 运动学 数据驱动；Lec 07 把这些表征 (尤其是 VAE latent / VQ codebook) 接到 物理仿真 + 强化学习 上，得到既自然又能与环境真实交互的角色——DeepMimic 模仿单段 mocap，AMP/ASE 用 GAN 风格判别器，ControlVAE 把 Motion VAE 当成可微策略空间。

Lecture 07 —Skinning & Facial Animation

[TIP] 学习指南

本节主线：把”骨骼动画”翻译成”网格变形”。给定一组随时间变化的骨骼变换 $\{T_j(t)\}$ 与绑定姿态网格 $\{x_i\}$ ，如何计算每帧的网格顶点位置 $\{x'_i(t)\}$? – 骨骼侧 (Skinning / 蒙皮): LBS、DQS、PSD、SMPL – 表情侧 (Facial Animation / 面部动画): Blendshapes、可形变面部模型、表演捕捉、语音驱动 关键概念: bind pose (绑定姿态)、skinning weights (蒙皮权重)、candy-wrapper artifact (糖纸伪影)、dual quaternion (对偶四元数)、blendshape (混合形状)。

0. Pipeline 总览 / Character Animation Pipeline

骨骼动画 → Rigging & Skinning → 蒙皮变形后的角色网格。

Skeletal Animation (骨骼动画)

+

Rigging & Skinning (绑定与蒙皮)

↓

Animated Character (会动的角色网格)

输入：网格 mesh $\{v_i\}$ (如 Stanford Bunny) + 骨骼 $\{(o_j, Q_j)\}$ (关节位置 + 朝向) + 蒙皮权重 $\{w_{ij}\}$ 。输出：每个顶点的世界坐标 x'_i 。

1. Skinning Deformation 的几何推导

1.1 单个骨骼的变换 / Single bone

设骨骼 j 在绑定姿态下原点位于 o_j 、朝向 $Q_j \in SO(3)$ ；当前帧改成 o'_j 、朝向 Q'_j ，其增量 R, t 满足：

$$Q'_j = R Q_j, \quad o'_j = o_j + t$$

刚体上的点 x (绑定姿态) 经过这次刚体运动后，新的位置：

$$x' = R(x - o_j) + o'_j = Q'_j Q_j^T (x - o_j) + o'_j$$

1.2 引入 bone-local 残差 r

把”骨骼系内的偏移”显式分离：

$$r_j = Q_j^T (x - o_j)$$

则

$$x' = Q'_j r_j + o'_j$$

直观理解： r_j 是顶点在 bind pose 下相对骨骼局部坐标系的常量；动画时只需用新姿态 Q'_j 把它“再投回”世界系。

1.3 多骨骼加权混合 / Multiple bones

每个骨骼 j 都给出一个候选位置 $x'_{ij} = Q'_j r_{ij} + o'_j$ ，其中 $r_{ij} = Q_j^T (x_i - o_j)$ 。Linear blend:

$$x'_i = \sum_{j=1}^m w_{ij} (Q'_j r_{ij} + o'_j)$$

[NOTE] 三要素 / Three ingredients

1. Bind pose / rest pose (绑定姿态)： $\{o_j, Q_j\}$ 与 x_i 一起冻结。2. Skinning weights (蒙皮权重)： $w_{ij} \geq 0$ ，通常归一化 $\sum_j w_{ij} = 1$ ，描述顶点受哪些骨骼影响。3. Skinning transformation (蒙皮变换)：每帧的 $\{Q'_j, o'_j\}$ 。

2. Linear Blend Skinning (LBS / 线性混合蒙皮)

2.1 等价化简

把 $r_{ij} = Q_j^T (x_i - o_j)$ 代回：

$$x'_i = \sum_j w_{ij} [Q'_j Q_j^T x_i + (o'_j - Q'_j Q_j^T o_j)] = \sum_j w_{ij} (R_j x_i + t_j)$$

其中 $R_j = Q'_j Q_j^T$ ， $t_j = o'_j - R_j o_j$ ，组合成相对于 bind pose 的刚体变换 $T_j = (R_j, t_j) \in SE(3)$ 。

继续把求和拆开：

$$x'_i = \left(\sum_{j=1}^m w_{ij} R_j \right) x_i + \sum_{j=1}^m w_{ij} t_j$$

[TIP] 一句话总结 LBS

“对每根骨骼的世界刚体变换做线性加权”——快、易实现、GPU 友好；游戏与实时引擎的默认方案。

2.2 自动权重 / Automatic skinning weights

手动刷权重昂贵，工业 + 学术常用：– Pinocchio [Baran et al., 2007]：自动骨架嵌入 + 权重。– Bounded Biharmonic Weights [Jacobson et al., 2011]：基于双调和方程的平滑非负权重。– 近年工作 (SIGGRAPH 2019、2021) 用学习 / 隐式场进一步改进。

2.3 Candy-Wrapper Artifact (糖纸伪影)

[WARN] LBS 的根本缺陷

当两根骨骼相对旋转较大(如肘关节扭 180°)时,由于”线性加权旋转矩阵”不再属于 $SO(3)$ (旋转矩阵的凸组合一般不是旋转),网格会塌缩成糖纸状。

数学根源: 对 R_j 做线性混合 $\sum w_j R_j$ 不在 $SO(3)$ 流形上。

3. Dual Quaternion Skinning (DQS / 对偶四元数蒙皮)

3.1 思路: 在 $SE(3)$ 上做”内蕴”插值

LBS 在 $\mathbb{R}^{3 \times 4}$ 中线性组合 T_j , 等价于把流形拉直; DQS 想要近似 $SE(3)$ 上的内蕴均值 (intrinsic blending)。

类比: $SO(3)$ 上单位四元数 + SLERP 是球面插值; $SE(3)$ 上单位对偶四元数 + DLB 是它的推广。

3.2 对偶数与对偶四元数 / Dual numbers & dual quaternions

[NOTE] 对偶数 (Dual number)

$$x = a + b\varepsilon, \quad \varepsilon^2 = 0$$

与复数 $x = a + bi$ ($i^2 = -1$) 形式上对偶。- 共轭: $\bar{x} = a - b\varepsilon$ - 乘法: $(a + b\varepsilon)(c + d\varepsilon) = ac + (ad + bc)\varepsilon$

把 4 个分量从实数换成四元数:

$$\hat{q} = q_0 + \varepsilon q_\varepsilon, \quad \varepsilon^2 = 0$$

运算 (要点):

$$\begin{aligned} s\hat{q} &= sq_0 + sq_\varepsilon \varepsilon, & \hat{q}_1 + \hat{q}_2 &= (q_{r1} + q_{r2}) + \varepsilon(q_{\varepsilon1} + q_{\varepsilon2}) \\ \hat{q}_1 \hat{q}_2 &= q_{r1} q_{r2} + \varepsilon(q_{r1} q_{\varepsilon2} + q_{\varepsilon1} q_{r2}) \end{aligned}$$

三种共轭:

$$\hat{q}^* = q_0^* + \varepsilon q_\varepsilon^*, \quad \hat{q}^\circ = q_0 - \varepsilon q_\varepsilon, \quad \hat{q}^\star = q_0^* - \varepsilon q_\varepsilon^* = (\hat{q}^*)^\circ$$

模:

$$\|\hat{q}\| = \sqrt{\hat{q}^* \hat{q}} = \|q_0\| + \varepsilon \frac{q_0 \cdot q_\varepsilon}{\|q_0\|}$$

[NOTE] 单位对偶四元数

$$\|\hat{q}\| = 1 \iff \|q_0\| = 1 \text{ 且 } q_0 \cdot q_\varepsilon = 0.$$

3.3 单位对偶四元数 刚体变换 / Rigid transformation

[TIP] 对应关系

$$T = (R \mid t) \longrightarrow \hat{q} = q_0 + \varepsilon q_\varepsilon, \quad q_0 = r, \quad q_\varepsilon = \frac{1}{2} t r$$

其中 r 是 R 对应的单位四元数, $t = (0, t_x, t_y, t_z)$ 是平移的纯四元数。

向量 v 的变换:

$$\hat{v}' = \hat{q} \hat{v} \hat{q}^*, \quad \hat{v} = 1 + \varepsilon(0, v)$$

双重覆盖: $\hat{q}$ 与 $-\hat{q}$ 表示同一刚体变换; $\hat{Q}$ 是 $SE(3)$ 的二重覆盖 (与四元数对 $SO(3)$ 类似)。线性混合时必须先做“半球翻转”, 否则会跨越覆盖造成大错。

3.4 DLB & DQS

Dual-Quaternion Linear Blending (DLB):

$$\hat{q}_t = \frac{(1-t)\hat{q}_0 + t\hat{q}_1}{\|(1-t)\hat{q}_0 + t\hat{q}_1\|}$$

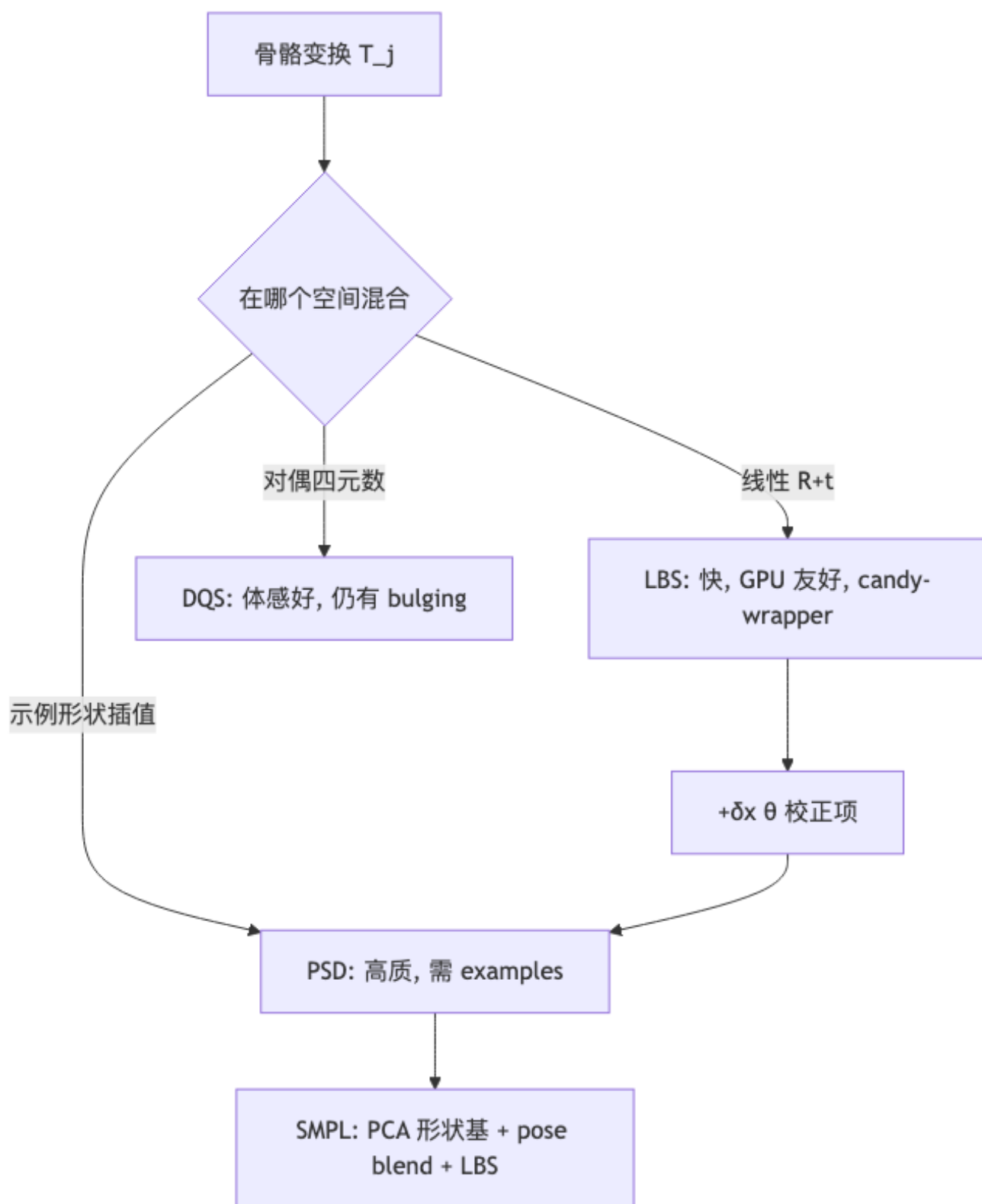
推广到多骨骼即 DQS:

$$\hat{q}_i = \frac{\sum_j w_{ij} \hat{q}_j}{\|\sum_j w_{ij} \hat{q}_j\|}, \quad x'_i = \hat{q}_i \hat{x}_i \hat{q}_i^*$$

[NOTE] 性质

- 是 $SE(3)$ 上内蕴均值的良好近似 (Kavan et al. 2008)。
- 没有 candy-wrapper: 扭转大角度时仍保体。
- 仍然存在 **bulging artifact (鼓包伪影)**: 骨骼弯曲时关节处轻微膨胀。

4. LBS / DQS / PSD 方法对比



方法	公式核心	优点	缺点
LBS	$\sum_j w_{ij}(R_j x_i + t_j)$	快、GPU 友好、易实现	candy-wrapper、关节体积塌缩
DQS	单位对偶四元数加权 + 归一化	保体扭转、内蕴混合	计算更贵、bulging artifact
PSD	$x' = \text{SKIN}(x) + \delta x(\theta)$, RBF 插值	姿态相关细节、肌肉鼓胀	需要例子、存储开销

5. Pose Space Deformation (PSD)

5.1 修正项 δx

在 LBS 之上加“姿态相关位移”:

$$x'_i = \sum_{j=1}^m w_{ij} T_j x_i + \delta x_i(\theta)$$

其中 θ 是当前关节配置。 $\delta x_i(\theta)$ 通过对若干**示例姿态**插值得到。

5.2 RBF 插值 / Radial Basis Function

[NOTE] 散点数据插值 (Scattered Data Interpolation)

给定示例 $\{(x_i, y_i)\}_{i=1}^K$,

$$y(x) = \sum_{i=1}^K w_i \varphi(\|x - x_i\|)$$

求权重要满足插值条件 $y(x_i) = y_i$, 即解线性方程 $Rw = y$, 其中 $R_{ij} = \varphi(\|x_i - x_j\|)$ 。

常用基函数: - 高斯: $\varphi(r) = e^{-r^2/c^2}$ - 反多重二次: $\varphi(r) = 1/\sqrt{r^2 + c^2}$ - 薄板样条: $\varphi(r) = r^2 \log r$
- 多调和样条: $\varphi(r) = r^k$ (k 奇) 或 $r^k \log r$ (k 偶)

5.3 PSD 实践要点

- $x' = \text{SKIN}(x) + \text{PSD}(\theta)$, PSD 由 RBF 实现。
- 示例形状可手工建模、或由 3D 扫描 (SMPL 思路) 得到。

[WARN] 设计权衡

- 整体 (per-shape) vs 顶点级 (per-vertex) 插值? - 局部 vs 全局插值? 是否所有关节都影响一个顶点? - RBF 是否足够? (也可换神经网络等)

5.4 EBS vs SSD (Example-based vs Skeleton Subspace)

类别	代表	优点	缺点
EBS (Example-based Skinning)	PSD	易控制、质量好、姿态相关细节 (鼓胀肌肉、暴起血管)	制作示例麻烦、额外存储、插值需调参
SSD (Skeleton Subspace Deformation)	LBS, DQS	易实现、快、GPU 友好	各种伪影、权重难调、姿态细节难做

6. SMPL / 一个工业级人体模型

[NOTE] SMPL = Skinned Multi-Person Linear Model

把 SSD (LBS) 与 EBS (PSD) 结合, 用真实扫描 + PCA 学到形状/姿态基, 是 ML/CV 中最常用的人体参数化模型。

6.1 形状基 / Body Shape (PCA)

PCA 概念: 给定数据 $\{x_i\} \subset \mathbb{R}^N$,

$$x_i = \bar{x} + \sum_{k=1}^n w_{i,k} u_k$$

u_k 是协方差矩阵 $\Sigma = X^T X$ 的第 k 大特征向量, $w_{i,k} = (x_i - \bar{x}) \cdot u_k$ 。

应用到身体形状:

$$T(\beta) = \bar{T} + \sum_{m=1}^{|\beta|} \beta_m S_m$$

β 是形状参数 (高/胖/肌肉...), S_m 是 PCA 主成分。

6.2 姿态相关形状基 / Pose Blend Shapes

$$T(\beta, \theta) = \bar{T} + \sum_m \beta_m S_m + \sum_n \theta_n P_n$$

θ 是关节角, P_n 通过优化得到 (替代 PSD 的 RBF)。

6.3 最终变形

$$x = \text{SKIN}(T(\beta, \theta), \theta, \mathcal{W})$$

其中 $\mathcal{W}$ 是蒙皮权重, SKIN 一般取 LBS 或 DQS。

[TIP] SMPL 的精髓

– 形状用 PCA 压缩 → 几十个 β 描述身材。– 姿态相关细节用 pose blend shapes (线性 in θ) → 弥补 LBS 的 candy-wrapper / 体积损失。– 最后挂上标准 LBS → 兼容现有引擎。

7. Facial Animation / 面部动画

7.1 数据获取 / Obtaining a face model

- 多视图扫描 (Light Stage、Triangulated photogrammetry)。
- 表演捕捉视频。
- 近年生成式：DreamFace 2023 (文本驱动 3D 脸)、HACK 2023 (头颈一体参数化)。

7.2 模型分解 / ID + Expression

[NOTE] 形变面部模型 / Morphable Face Model

把人脸分成”身份”和”表情”两组线性基：

$$X = X_0 + \underbrace{\sum_i \beta_i B_i^{\text{ID}}}_{\text{Identity}} + \underbrace{\sum_j \theta_j B_j^{\text{Exp}}}_{\text{Expression}}$$

– X_0 ：平均脸 (average face) 。 – $\{B_i^{\text{ID}}\}$ ：身份基，决定五官形状 (脸型定制) 。 – $\{B_j^{\text{Exp}}\}$ ：表情基 (Blendshapes)，决定当前表情。

7.3 Facial Blendshapes (混合形状)

混合形状把”特定表情下的位移网格”作为基，线性叠加得到任意表情。

$$X = \sum_i w_i B_i$$

(与 PSD/SMPL 同构：blendshape $\approx$ blend space)。

[TIP] ARKit 标准 52 个 blendshapes

包含眼 (eyeBlink/Look/Squint/Wide)、下颌 (jawForward/Left/Right/Open)、嘴 (mouthSmile/Frown/Funnel/Pucker/Roll/Shrug/Press/...)、眉 (browDown/InnerUp/OuterUp)、脸颊 (cheekPuff/Squint)、鼻 (noseSneer)、舌 (tongueOut) 等。iPhone TrueDepth 摄像头可实时输出这 52 个权重，工程上是事实标准。

Blendshapes vs Example-based Skinning：本质都是”在示例上插值”，但 blendshape 通常不挂骨骼、直接对面部小幅变形线性组合，更适合面部这种没有清晰刚体支撑的解剖结构。

7.4 Morphable Face Models 的工业落地

- MetaHuman (Unreal Engine)：高保真可形变面部。
- Egger et al. 2020：3DMM Past/Present/Future 综述。

7.5 表演捕捉 / Facial Performance Capture

输入观测 Y (图像 landmark / 多视图点云)，求最优 (β_i, θ_j) ：

$$\min_{\beta_i, \theta_j} \left\| X_0 + \sum_i \beta_i B_i^{\text{ID}} + \sum_j \theta_j B_j^{\text{Exp}} - Y \right\|^2 + E(\beta_i, \theta_j)$$

E 是正则项（先验： β 接近 0、 θ 时间平滑等）。

工业级例子：Industrial Light & Magic、Google ML Kit Face Detection、Apple ARKit。

7.6 语音驱动面部 / Speech-driven Facial Animation

输入音频/语音，输出 blendshape 权重时间序列。代表系统：NVIDIA Omniverse **Audio2Face**（实时口型 + 情绪）。



8. 总结一页

[NOTE] 核心公式速查

- LBS: $x'_i = (\sum_j w_{ij} R_j) x_i + \sum_j w_{ij} t_j$ - DQS: $\hat{q}_i = \frac{\sum_j w_{ij} \hat{q}_j}{\|\sum_j w_{ij} \hat{q}_j\|}$, $x' = \hat{q}_i \hat{x}_i \hat{q}_i^*$ - 对偶四元数: $\hat{q} = q_0 + \varepsilon q_\varepsilon$, $q_\varepsilon = \frac{1}{2} tr$ - PSD: $x' = SKIN(x) + \delta x(\theta)$, δx 用 RBF - SMPL: $T(\beta, \theta) = \bar{T} + \sum \beta_m S_m + \sum \theta_n P_n$; 外挂 LBS - Morphable Face: $X = X_0 + \sum \beta_i B_i^{ID} + \sum \theta_j B_j^{Exp}$

记号速查

符号	含义
x_i	第 i 个顶点（绑定姿态，世界系）
x_i	当前帧顶点位置
o_j, Q_j	骨骼 j 在 bind pose 下的原点与朝向
o'_j, Q'_j	当前帧骨骼 j 的原点与朝向
R_j, t_j	相对 bind pose 的刚体变换; $T_j = (R_j, t_j) \in SE(3)$
r_{ij}	x_i 在骨骼 j 局部系下的常向量, $= Q_j^T (x_i - o_j)$
w_{ij}	顶点 i 受骨骼 j 影响的蒙皮权重
$\hat{q} = q_0 + \varepsilon q_\varepsilon$	对偶四元数
β, θ	形状 / 姿态参数 (SMPL、Morphable Face)
B_i^{ID}, B_j^{Exp}	身份基、表情基 (blendshapes)
$\delta x(\theta)$	PSD 的姿态相关修正项

Lecture 08 —基于物理的仿真与铰接刚体

[TIP] 学习指南

本节核心：从牛顿第二定律出发，建立“质点 → 单刚体 → 铰接刚体 + 关节 + 接触摩擦”的完整数值仿真管线。
前置知识：线性代数（旋转矩阵、四元数、特征分解）、常微分方程、第 03 讲刚体运动学。重点关注：① 三种 Euler 积分的稳定性差异 ② 角速度与 $\dot{R}$ 的关系 ③ Newton-Euler 方程 ④ 用 Lagrange 乘子统一处理关节与接触 ⑤ 摩擦锥线性化 + LCP + PGS。

0. 核心问题 / Motivation

运动学方法（Lec 03-07）只能“指定”动作，无法回答“角色与环境真实交互后会发生什么”。物理仿真把动作交给牛顿定律：给定外力、关节、接触，下一帧的位姿由动力学方程决定。本节回答：

1. 给定 $f(x, v, t)$ ，如何稳定地从 (x_n, v_n) 推到 (x_{n+1}, v_{n+1}) ？
2. 刚体的姿态 R 怎么参与积分？ ω 与 $\dot{R}$ 是什么关系？
3. 多个刚体被关节连起来、又踩在地面上时，方程如何一次性求解？

1. 质点动力学与时间离散

1.1 连续问题

[NOTE] 牛顿第二定律

$$f = ma, \quad a = \dot{v}, \quad v = \dot{x} \implies a = f(x, v, t)/m$$

给定 $x_0, v_0, f(x, v, t)$ ，速度与位置由两次积分得到：

$$v(t) = v_0 + \int_{t_0}^t a \, dt, \quad x(t) = x_0 + \int_{t_0}^t v \, dt$$

只有当 f 与 x, v 无关（如纯重力 $f = mg$ 常量）时，才有解析的 $x_t = x_0 + v_0 t + \frac{1}{2} a t^2$ ；一般情形必须数值积分。

1.2 时间离散

设步长 h , $t_n = nh$, $x_n = x(t_n)$, 则

$$v_{n+1} = v_n + \int_{t_n}^{t_{n+1}} a \, dt, \quad x_{n+1} = x_n + \int_{t_n}^{t_{n+1}} v \, dt$$

把积分替换成矩形近似，得到三种 Euler 格式。

1.3 三种 Euler 积分

名称	速度更新	位置更新	直觉
显式 / 前向 Euler	$v_{n+1} = v_n + a_n h$	$x_{n+1} = x_n + v_n h$	全用当前帧信息
隐式 / 后向 Euler	$v_{n+1} = v_n + a_{n+1} h$	$x_{n+1} = x_n + v_{n+1} h$	用未知未来帧，需解方程
半隐式 / Symplectic Euler	$v_{n+1} = v_n + a_n h$	$x_{n+1} = x_n + v_{n+1} h$	速度先更新，位置用新速度

[WARN] 隐式 Euler 的代价

$a_{n+1} = f(x_{n+1}, v_{n+1})/m$ 依赖未知量，需要在每一步求解非线性方程组（多刚体时维度极高）。典型实现：把方程线性化得到 $A\Delta v = b$ ，然后调线性求解器。

1.4 弹簧例：稳定性差异 $f = -kx$, $\hat{k} = k/m$

把三种格式写成矩阵形式：

$$\text{Explicit: } \begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \begin{bmatrix} 1 & -\hat{k}h \\ h & 1 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix}$$

$$\text{Semi-implicit: } \begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \begin{bmatrix} 1 & -\hat{k}h \\ h & 1 - \hat{k}h^2 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix}$$

$$\text{Implicit: } \begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \frac{1}{1 + \hat{k}h^2} \begin{bmatrix} 1 & -\hat{k}h \\ h & 1 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix}$$

[TIP] 三句话总结

– **Explicit**：快、不解方程；大 h 下能量爆炸 → **conditionally stable**。– **Semi-implicit**：同样快、能量近似守恒 → 物理动画里默认选择。– **Implicit**：慢、要解方程；任何步长都不爆炸 → **unconditionally stable**，过大步长精度差但不发散。

1.5 精度问题

矩形近似的局部误差是 $O(h)$ ；因此减小 h 同时提高精度与稳定性。但极小 h 不能解决“穿墙”——若 $v_n h$ 一步跨过墙面而下一帧才检测，仍然漏检。

1.6 更高阶积分

- Runge-Kutta (RK2/RK4)：用中点/多采样换更高阶精度
- Variational integrator：从离散最小作用量原理出发，长时间能量更稳定
- Position-Based Dynamics (PBD)：直接对位置做约束投影

2. 刚体运动学：位姿、速度、积分

2.1 位姿表示

刚体上任意点 x' :

$$x' = x + Rr_0 = x + r$$

其中 x 是质心位置, $R \in SO(3)$ 是姿态, r_0 是该点在物体坐标系下的常向量, $r = Rr_0$ 是其在世界系下的瞬时偏移。

2.2 速度的拆分：线速度 + 角速度

对 x' 求时间导:

$$\dot{x}' = \dot{x} + \dot{R}r_0$$

但 $\dot{R}$ 没有直观意义。利用 $RR^T = I$:

$$\frac{d}{dt}(RR^T) = \dot{R}R^T + R\dot{R}^T = 0 \implies \dot{R}R^T = -(\dot{R}R^T)^T$$

[NOTE] 关键结论

$\dot{R}R^T$ 是一个反对称矩阵, 可用一个三维向量 ω (角速度) 表示:

$$\dot{R}R^T = [\omega]_{\times} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}, \quad \dot{R} = [\omega]_{\times} R$$

由此 $\dot{R}r_0 = [\omega]_{\times} Rr_0 = \omega \times r$, 于是:

$$\boxed{\dot{x}' = v + \omega \times r} \quad (v = \dot{x} : \text{线速度}, \omega : \text{角速度})$$

2.3 用 Rodrigues 公式独立推导 $\dot{x} = \omega \times x$

绕单位轴 u 转 $\delta\theta$: $\delta x = \sin \delta\theta (u \times x) + (1 - \cos \delta\theta) u \times (u \times x)$ 。求 d/dt 在 $\delta\theta \rightarrow 0$ 极限:

$$\dot{x} = \dot{\theta} u \times x = \omega \times x, \quad \omega = \dot{\theta} u$$

直接验证 $R = [e_x | e_y | e_z]$ 各列轴向量满足 $\dot{e}_i = \omega \times e_i$, 与 $\dot{R} = [\omega]_{\times} R$ 一致。

2.4 姿态的数值积分

朴素一阶:

$$x_{n+1} = x_n + h v_n, \quad R_{n+1} = R_n + h [\omega_n]_{\times} R_n$$

问题: 相加后 R_{n+1} 不再正交, 需要重新做 Gram-Schmidt 正交化 (昂贵且引入误差)。

四元数方案: 用 $q \in \mathbb{H}$ (单位四元数) 表示姿态, 则

$$\dot{q} = \frac{1}{2} \bar{\omega} q, \quad \bar{\omega} = (0, \omega) \in \mathbb{H}$$

积分:

$$q_{n+1} = q_n + h \dot{q}_n = q_n + \frac{h}{2} \bar{\omega}_n q_n$$

之后必须归一化 $q \leftarrow q / \|q\|$ 。比矩阵正交化便宜得多。

3. 刚体动力学：动量、惯量、Newton—Euler

3.1 质点动量与力矩

线动量 $p = mv$ ，对参考点 o 的角动量 $L_o = m r \times v$ (r 是 $o \rightarrow$ 质点向量)。力 F 、力矩 $\tau = r \times F$ 。

3.2 把刚体当成质点的集合

质心 $x_c = \frac{\sum_i m_i x_i}{\sum_i m_i}$ ，质心速度 $v_c = \frac{\sum_i m_i v_i}{M}$ ，总质量 $M = \sum m_i$ 。

总动量： $p = \sum_i m_i v_i = M v_c$ 。

3.3 角动量与惯量张量

把 $r'_i = r_c + r_i$ (r_i 从质心指到 i) 代入 $L_o = \sum m_i r'_i \times v_i$ ：

$$L_o = M r_c \times v_c + \underbrace{\sum_i m_i r_i \times v_i}_{L_{x_c} \text{ (关于质心的角动量)}}$$

刚体上 $v_i = v_c + \omega \times r_i$ ，关于质心部分中 $\sum m_i r_i \times v_c = (\sum m_i r_i) \times v_c = 0$ (r_i 从质心起算)。
所以：

$$L_{x_c} = \sum_i m_i r_i \times (\omega \times r_i) = \left(\sum_i -m_i [r_i]_{\times}^2 \right) \omega$$

[NOTE] 惯量张量

$$I = \sum_i -m_i [r_i]_{\times}^2, \quad L = I \omega$$

I 是 3×3 对称半正定矩阵，描述刚体绕质心的“转动惯性”。

3.4 惯量的旋转

体坐标系中常量惯量 I_0 ，世界系下：

$$I = R I_0 R^T$$

(推导： $R[r_0]_{\times} R^T = [Rr_0]_{\times}$ ，把 $r_i = Rr_{0,i}$ 代入定义即得。)

3.5 主轴

I_0 对称正定 $\rightarrow$ 特征分解 $I_0 = U \text{diag}(I_1, I_2, I_3) U^T$ 。主轴就是这组特征向量；选物体局部系沿主轴时 I_0 直接对角化，简化计算。

3.6 力与力矩在刚体上的合成

若有多外力 f_i 作用在 r_i (相对质心)，合力 $f = \sum f_i$ ，合力矩 (对质心) $\tau = \sum \tau_i = \sum r_i \times f_i$ 。两个等大反向但作用线不同的力合力为 0、合力矩非 0 (“力偶”)。

3.7 Newton—Euler 方程

$$\frac{dp}{dt} = f, \quad \frac{dL}{dt} = \tau$$

代入 $p = Mv$, $L = I\omega$, 注意 I 随 R 变化:

$$\dot{I} = \dot{R}I_0R^T + RI_0\dot{R}^T = [\omega]_{\times}I + I[\omega]_{\times}^T$$

$$\dot{I}\omega = [\omega]_{\times}I\omega + I(-\omega)_{\times}\omega = \omega \times (I\omega)$$

(因为 $\omega \times \omega = 0$)。所以:

[NOTE] Newton—Euler 方程组

$$\begin{bmatrix} MI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix}$$

把姿态视为状态变量后, 刚体仿真就是反复求解上式 + 用 v_{n+1}, ω_{n+1} 推进 x, q 。

3.8 单刚体一步仿真

半隐式离散:

$$\frac{1}{h} \begin{bmatrix} MI_3 & 0 \\ 0 & I_n \end{bmatrix} \begin{bmatrix} v_{n+1} - v_n \\ \omega_{n+1} - \omega_n \end{bmatrix} + \begin{bmatrix} 0 \\ \omega_n \times I_n \omega_n \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix}$$

$$x_{n+1} = x_n + hv_{n+1}, \quad q_{n+1} = q_n + \frac{h}{2}\bar{\omega}_{n+1}q_n, \quad I_n = R_n I_0 R_n^T$$

4. 铰接刚体与约束

4.1 多刚体堆叠

把每个 link 的状态 $(x_i, R_i, v_i, \omega_i)$ 拼成大向量 $\mathbf{v} = (v_1, \omega_1, \dots, v_n, \omega_n)^T$, 质量与惯量拼成块对角 $\mathbf{M}$, 则 Newton—Euler 整体写为:

$$\mathbf{M}\dot{\mathbf{v}} + \mathbf{C}(\mathbf{x}, \mathbf{v}) = \mathbf{f}$$

其中 $\mathbf{C}$ 收纳所有 $\omega \times I\omega$ 项。

4.2 关节带来的约束力

两个 link 共用关节点 x_J : $x_1 + R_1 r_1 = x_J = x_2 + R_2 r_2$ 。对时间求导:

$$v_1 + \omega_1 \times R_1 r_1 = v_2 + \omega_2 \times R_2 r_2$$

整理成 $J\mathbf{v} = 0$ 的线性约束 (J 是 Jacobian, 由各 link 的 r_i 反对称矩阵拼成):

$$\begin{bmatrix} I_3 & -[R_1 r_1]_{\times} & -I_3 & [R_2 r_2]_{\times} \end{bmatrix} \begin{bmatrix} v_1 \\ \omega_1 \\ v_2 \\ \omega_2 \end{bmatrix} = 0$$

4.3 一般约束 $g(x) = C$

$$g(x) = C \Rightarrow \frac{d}{dt}g = \frac{\partial g}{\partial x} \dot{x} = Jv = 0, \quad J = \nabla g^T$$

[NOTE] 约束力的方向与功

约束是被动的——不消耗也不增加能量： $f_c \cdot v = 0$ 。结合 $Jv = 0$ ，可知 f_c 与约束面 normal 平行，即

$$f_c = J^T \lambda, \quad \lambda \in \mathbb{R}^k (\text{Lagrange 乘子})$$

4.4 带约束的运动方程

$$M\dot{\mathbf{v}} + C = \mathbf{f} + J^T \lambda, \quad J\mathbf{v}_{n+1} = 0$$

半隐式离散得到一组关于 $\mathbf{v}_{n+1}, \lambda$ 的线性方程组。

消元求 λ ：从动量方程解出 $\mathbf{v}_{n+1}$ ，代入约束式：

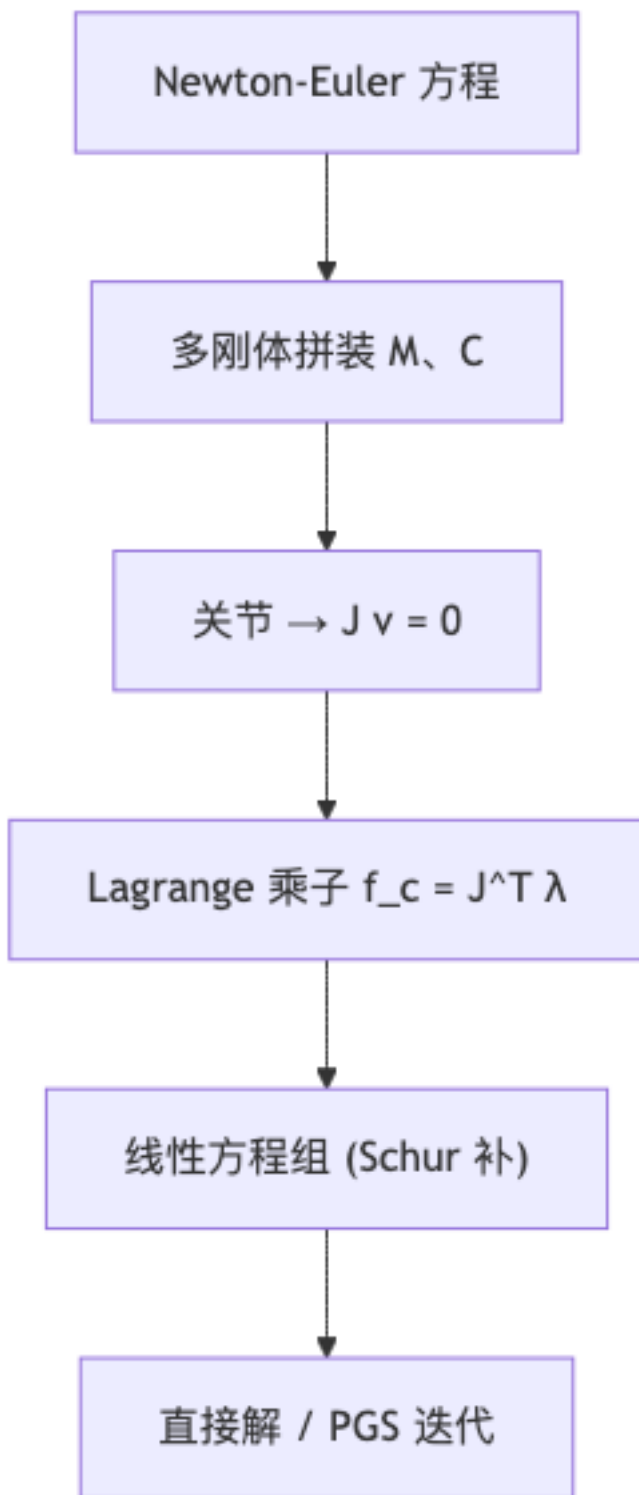
$$JM^{-1}J^T \lambda = c_n$$

– $JM^{-1}J^T$ 是 Schur 补，对称半正定。– 数值漂移修正 (**Baumgarte 稳定化**)： $J\mathbf{v}_{n+1} = \alpha \frac{C - g(x_n)}{h}$
(ERP, error reduction parameter)。– 软化 (**CFM**, constraint force mixing)： $(JM^{-1}J^T + \beta I)\lambda = c_n$ 。

4.5 关节类型与 Jacobian 行数

关节	DoF	约束行数
球窝 (ball-and-socket)	3	3 (仅位置共点)
万向 (universal)	2	4
铰链 / 旋转 (hinge/revolute)	1	5
焊接 (fixed)	0	6

J 的形式按关节类型选取相应行 (位置 3 行 + 受限的转动行)。



5. 接触

5.1 检测

广相位 (AABB / BVH) + 窄相位 (GJK / EPA / SAT), 输出接触点 x_c 、刚体上偏移 r_c 、穿透深度 d 、法向 n 。

5.2 Penalty-based 接触

把接触建模为弹簧 + 阻尼：

$$f_n = -k_p d - k_d v_{c,\perp}$$

– 优点：实现一行代码，与现有积分器无缝结合。– 缺点： k_p 大 $\rightarrow$ 需要极小步长（同 1.4）； k_p 小 $\rightarrow$ 物体下陷穿透。

5.3 Penalty 摩擦

Coulomb 律 $|f_t| \leq \mu f_n$ 。最朴素： $f_t = -\mu f_n \hat{v}_{c,\parallel}$ （动摩擦）。

[WARN] 静摩擦的难处

物体静止时要求 $|f_t| \leq \mu f_n$ 但方向未定，纯惩罚力做不到“恰好抵消所有外力”。Yamane–Nakamura 2006 的修正：记录接触起始点 p_0 ，

$$\bar{f}_t = k_p^f (p_0 - p) - k_d^f v_{c,\parallel}$$

若 $|\bar{f}_t| < \mu f_n$ 用静摩擦（粘着）；超过阈值则切到 $f_t = -\mu f_n \hat{v}_{c,\parallel}$ （动摩擦），并把 p_0 重置。

5.4 接触作为约束（更严格）

非穿透 = 单边速度约束：

$$v_{c,\perp} = J_{c,\perp} \begin{bmatrix} v \\ \omega \end{bmatrix} \geq 0, \quad \lambda \geq 0$$

即接触法向速度非负、法向冲量非负，且互补：

$$v_{c,\perp,j} > 0 \Rightarrow \lambda_j = 0, \quad \lambda_j > 0 \Rightarrow v_{c,\perp,j} = 0$$

合写为 $v_{c,\perp} \perp \lambda$ ，配上动量方程，整体是线性互补问题（LCP）：

$$0 \leq A\mathbf{x} + \mathbf{b} \perp \mathbf{x} \geq 0$$

求解器：Lemke（直接、单纯形思想）或 Projected Gauss–Seidel（迭代、易并行）。

5.5 摩擦锥与线性化

Coulomb 锥 $\|f_t\| \leq \mu f_n$ 是 3D 二次锥；为塞进 LCP，把切向圆周离散成 K 个基方向 $b_1, \dots, b_K$ ：

$$f_t = \sum_k \lambda_k b_k, \quad \lambda_k \geq 0$$

线性化后摩擦锥变成摩擦锥近似多面体，约束 $\mu \lambda_n - \sum_k \lambda_k \geq 0$ 。

滑动判据：引入辅助变量 $\beta \geq 0$ ，设 $\bar{v}_c = (v_{c,\parallel} \cdot b_1, \dots, v_{c,\parallel} \cdot b_K)^T$ ，

$$\beta \mathbf{e} + \bar{v}_c \geq 0, \quad \beta > 0 \iff \text{接触在滑动}$$

（“滑动 $\iff$ 摩擦锥被切向速度填满”。）

5.6 完整 LCP 系统（接触 + 摩擦）

把法向冲量 λ_n 、切向基系数 λ_k 、滑动指示 β 拼成 $\mathbf{x} = (\lambda_n, \lambda_k, \beta)$ ，整套 KKT 条件写成一个混合 LCP：

$$\boxed{0 \leq A\mathbf{x} + \mathbf{b} \perp \mathbf{x} \geq 0}$$

- 第一行： $0 \leq v_{c,\perp} \perp \lambda_n \geq 0$ （非穿透 + 非粘合）
- 第二行： $0 \leq \beta \mathbf{e} + \bar{v}_c \perp \lambda_k \geq 0$ （切向基取的方向才能滑）
- 第三行： $0 \leq \mu \lambda_n - \sum_k \lambda_k \perp \beta \geq 0$ （摩擦锥饱和才允许滑动）

5.7 Gauss–Seidel 与 Projected Gauss–Seidel

求解 $A\mathbf{x} = \mathbf{b}$ ($A = L_* + U$, L_* 含对角的下三角)：

$$L_* \mathbf{x}_{k+1} = \mathbf{b} - U \mathbf{x}_k$$

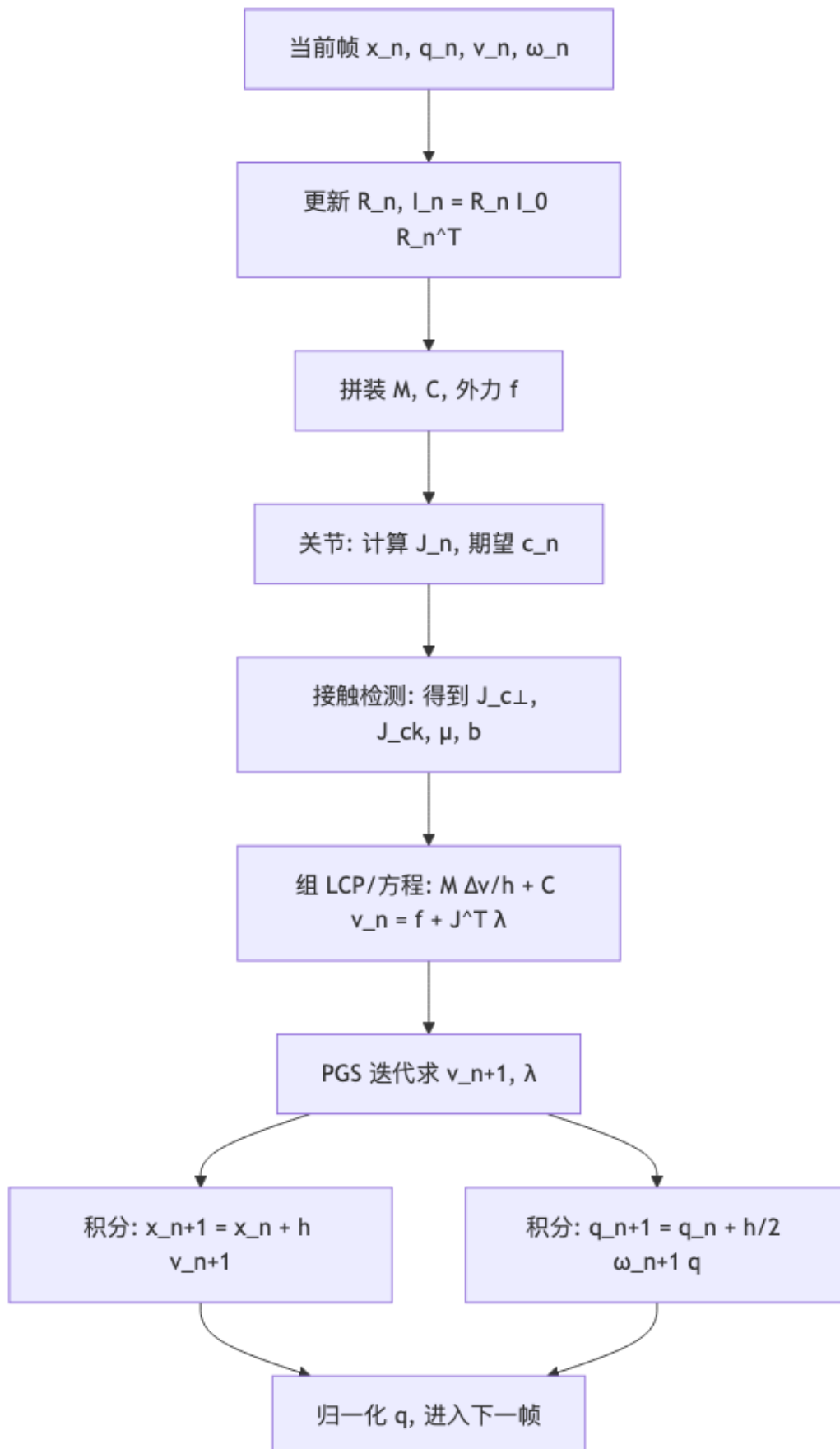
- 收敛条件： A 对称正定 或 严格 / 不可约对角占优。

求 LCP 时只需在每次更新后**投影到非负锥**：

$$\mathbf{x}_{k+1} = \max(0, L_*^{-1}(\mathbf{b} - U \mathbf{x}_k))$$

简单、易并行、容易加 warm-start，是 ODE/Bullet 等仿真器默认接触求解器。

6. 把所有东西拼起来：一步刚体系统仿真



[EX] 一帧伪代码 (Bullet/ODE 风格)

```
# 1) 预测: 仅外力
for body i:  $v^* = v_i + h M_i^{-1} (f_i - C_i v_i)$ 
# 2) 检测接触/关节, 组装  $J_n, J_c, b$ 
# 3) PGS 迭代求  $\lambda$ 
for iter = 1..K:
    for j in constraints: # 关节
        update  $\lambda_j$  and  $v_i$  by Schur 补
    for c in contacts:
         $\lambda_{n_c} = \max(0, \lambda_{n_c} + \alpha_n(\dots))$ 
        project ( $\lambda_t, \beta$ ) to friction cone
# 4) 用  $v_{\{n+1\}} = v^* + \Delta V(\lambda)$  推进  $x, q$ ; 归一化  $q$ 
```

记号速查

符号	含义
x, v	质心位置、线速度 (世界系)
R, q	姿态: 旋转矩阵 / 单位四元数
ω	角速度 (世界系, $\dot{R} = [\omega]_{\times} R$)
M, I	质量、惯量张量; $I = R I_0 R^T$
p, L	线动量 Mv 、角动量 $I\omega$
f, τ	合外力、合外力矩
J	约束 Jacobian, $Jv = 0$
λ	Lagrange 乘子 / 约束冲量
h	仿真步长 $t_{n+1} - t_n$
k_p, k_d	Penalty 接触刚度 / 阻尼
μ	Coulomb 摩擦系数
α, β	ERP / CFM (关节漂移修正与软化)

Lecture 09 —角色驱动：关节力矩、PD 控制与平衡

[TIP] 学习指南

本节核心：在 Lec 08 已建立的“被动 ragdoll 仿真”之上，给角色装上“肌肉”——以**关节力矩** τ 为执行器，由控制器 π 决定，把动作生成问题转化为“什么时候、对哪个关节、施加多大力矩”。前置知识：Lec 08 Newton—Euler 方程 $M\dot{v} + C(x, v) = f + J^T\lambda$ ；Lec 03 雅可比矩阵 / FK；半隐式 Euler。重点关注：① **极大坐标 vs. 广义坐标** 两种描述 ② **力 力矩** 在自由刚体上的等价性 ③ 关节力矩“正反对”的作用方式 ④ 正/逆动力学的二元性 ⑤ **PD / Stable PD** 控制律及其稳定性 ⑥ **Jacobian Transpose** 把“虚拟力”映射回关节力矩 ⑦ 静态平衡的 CoM / 支撑多边形 / 踝-腕策略。

0. 核心问题 / Motivation

Lec 08 给了我们一台“物理仿真器”：输入 $(x_t, v_t, R_t, \omega_t)$ + 所有外力，输出下一帧状态。但只有**重力**和**接触**的角色就是一个 ragdoll——它会摔倒。要让角色“自己动”，必须有一个“控制器” π 把当前状态映射成**主动力 / 力矩**，再喂给仿真器：

$$\pi : (s_t, t) \mapsto (f, \tau) \text{ Joint torques}$$

本节回答：

1. 主动驱动应该写在哪一层？为什么是**关节力矩** τ ？
2. 给定一对相连刚体， τ 在 Newton—Euler 方程里如何加入？
3. 角色动力学有两种坐标（极大坐标 / 广义坐标）——它们各自的代价是什么？
4. 正动力学 $(f \rightarrow \dot{v})$ 与逆动力学 $(\dot{v} \rightarrow f)$ 如何互为镜像？
5. 最简单的反馈控制律——**PD 控制**——稳定吗？大 k_p 为何爆炸？
6. 如何只用关节力矩“模仿”作用在末端的“虚拟力”（虚拟模型控制 / Jacobian Transpose）？
7. 站立时为何要把投影 CoM 拉到支撑多边形中心？

1. 从仿真到驱动：极大坐标与广义坐标

1.1 单刚体回顾（Lec 08 浓缩）

线性： $\dot{p} = f$, $p = mv$ ；角向： $\dot{L} = \tau$, $L = I\omega$ 。合写为 Newton—Euler：

$$\begin{bmatrix} mI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix}$$

铰接刚体（Lec 08 §4.4）：

$$M\dot{v} + C(x, v) = f + J^T \lambda, \quad Jv = 0$$

[NOTE] 一帧仿真步骤 (Lec 08 复盘)

① 检测碰撞与接触 → ② 装配 J, b → ③ PGS 求 λ (含关节/接触/摩擦) → ④ v_{n+1} 半隐式更新 → ⑤ $x_{n+1} = x_n + hv_{n+1}$ 、 $q_{n+1} = q_n + \frac{h}{2}\bar{\omega}_{n+1}q_n$ (四元数归一化)。

1.2 两种坐标系：Maximized vs. Generalized

坐标系	状态	维度	关节如何体现
极大坐标 (Maximized)	每个 link 独立 (x_i, Q_i, v_i, ω_i)	$13N$ 个变量	约束 $Jv = 0$ + Lagrange 乘子 λ
广义坐标 (Generalized)	关节角 $\theta = (\theta_0, \theta_1, \dots)$	等于 DoF	自动满足 (直接参数化)

[EX] 一条链的广义 → 极大映射 (Forward Kinematics)

设 link i 的局部转轴 u_i 、关节角 θ_i ，记 $Q_i = Q_{i-1}R(\theta_i u_i)$ (沿链累乘)：

$$x_i = x_{i-1} + Q_{i-1}d_{i-1} + Q_i l_i$$

其中 l_i 是 link i 局部系下从关节到质心的偏移， d_{i-1} 是 link $i-1$ 质心到下一关节的偏移。

速度递推：

$$\omega_i = \omega_{i-1} + Q_{i-1}\dot{\theta}_i u_i, \quad v_i = v_{i-1} + \omega_{i-1} \times Q_{i-1}d_{i-1} + \omega_i \times Q_i l_i$$

加速度递推 (再次对时间求导)：

$$\dot{\omega}_i = \dot{\omega}_{i-1} + \omega_{i-1} \times Q_{i-1}\dot{\theta}_i u_i + Q_{i-1}\ddot{\theta}_i u_i$$

$$\dot{v}_i = \dot{v}_{i-1} + \dot{\omega}_{i-1} \times Q_{i-1}d_{i-1} + \omega_{i-1} \times (\omega_{i-1} \times Q_{i-1}d_{i-1}) + \dot{\omega}_i \times Q_i l_i + \omega_i \times (\omega_i \times Q_i l_i)$$

[NOTE] 矩阵化

把链路上所有 (ω_i, v_i) 写成对 $\dot{\theta}$ 的线性映射：

$$\omega_i = J_i^\omega(\theta)\dot{\theta}, \quad v_i = J_i^v(\theta)\dot{\theta}$$

$$\dot{\omega}_i = J_i^\omega\ddot{\theta} + \dot{\theta}^T H_i^\omega \dot{\theta}, \quad \dot{v}_i = J_i^v\ddot{\theta} + \dot{\theta}^T H_i^v \dot{\theta}$$

其中 H_i 是 Hessian 项 (二次速度耦合)。这正是为什么“广义坐标 + 加速度”涉及的非线性项很多——但变量数最少。

1.3 角色的根关节

人形角色没有“机座”——根关节 (pelvis) 是漂在空间里的 6-DoF 自由体：

$$\theta = (t_0, \theta_0, \theta_1, \dots), \quad \dot{\theta} = (v_0, \omega_0, \omega_1, \dots), \quad \ddot{\theta} = (\dot{v}_0, \dot{\omega}_0, \dot{\omega}_1, \dots)$$

[WARN] 这就是为什么人形角色”欠驱动 (underactuated)”

6 个根 DoF 没有对应的电机——靠脚下接触力间接控制。下面 §3.3 会反复看到这一点。

2. 力 vs. 力矩：施加位置的等价变换

2.1 自由刚体上：单一外力的等效

一个外力 f 作用在刚体上的 r_f 点（相对质心）：

$$\begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} \leftarrow \begin{bmatrix} f \\ \tau + r_f \times f \end{bmatrix}$$

[EX] 力偶 (couple)

两个等大反向作用在不同点的力： f 在 r_1 、 $-f$ 在 r_2 。合力 $= 0$ ，合力矩 $= r_1 \times f + r_2 \times (-f) = (r_1 - r_2) \times f$ 。对任意参考点都相同——纯力偶完全由力矩描述。

这就是”力矩”在动力学上的真正含义：可以无视具体施力位置。

2.2 关节内部力的本性

回到两刚体共用关节 x_J 。关节本身是无质量的——它不能存储动量，也不能”凭空生成”净力。设内部有任意作用力集合 $\{f_i\}$ ：

$$\sum_i f_i = 0 \quad (\text{关节净外力为 } 0)$$

但作用在 link 1、link 2 上的力矩不一定为零：

$$\begin{aligned} \tau_1 &= \sum_i (r_1 + r_i) \times f_i = r_1 \times \left(\sum_i f_i \right) + \sum_i r_i \times f_i = \sum_i r_i \times f_i \\ \tau_2 &= \sum_i (r_2 + r_i) \times (-f_i) = - \sum_i r_i \times f_i = -\tau_1 \end{aligned}$$

[NOTE] 关节力矩 = 一对反向力偶

把内部细节抽象掉后：

$$\tau_1 = \tau, \quad \tau_2 = -\tau$$

实现一个”关节力矩 τ ”只需要：对子刚体加 τ ，对父刚体加 $-\tau$ 。

带关节力矩的两刚体运动方程：

$$\begin{bmatrix} m_1 I_3 & & & \\ & I_1 & & \\ & & m_2 I_3 & \\ & & & I_2 \end{bmatrix} \begin{bmatrix} \dot{v}_1 \\ \dot{\omega}_1 \\ \dot{v}_2 \\ \dot{\omega}_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \omega_1 \times I_1 \omega_1 \\ 0 \\ \omega_2 \times I_2 \omega_2 \end{bmatrix} = \begin{bmatrix} 0 \\ \tau \\ 0 \\ -\tau \end{bmatrix} + J^T \lambda, \quad Jv = 0$$

净外力为 0 (动量守恒)，净外力矩对 system 也为 0 (角动量守恒)——角色没法靠内部力矩起飞。

2.3 全身控制器接口

状态 $(x_t, v_t, R_t, \omega_t)$

↓
Controller $\pi(s_t, t) \rightarrow$ per-joint τ_k

↓
[For each joint k]: apply $+\tau_k$ to child, $-\tau_k$ to parent

↓
Dynamic Simulator (Lec 08 pipeline)

↓
Next state $(x_{t+1}, v_{t+1}, R_{t+1}, \omega_{t+1})$

[WARN] Underactuated vs. Fully-actuated

- **Fully-actuated**: 执行器数 $\geq$ DoF; 任意 $(x, v, \dot{v})$ 都能找到 f 实现 (如工业机械臂带固定基座)。
- **Underactuated**: 执行器数 $<$ DoF; 许多 $(x, v, \dot{v})$ 无解 (如人形: 根的 6 DoF 没有电机)。
人形角色之所以难, 本质就是欠驱动——你不能直接命令骨盆“立刻向上加速 2 m/s^2 ”, 只能通过腿一脚一地面接触链间接产生。

3. 正动力学 vs. 逆动力学

3.1 二元定义

运动方程 $M\dot{v} + C = f + J^T\lambda$ 提供了 $(x, v, f, \dot{v})$ 之间的关系。

	输入	输出	用途
Forward dynamics (FD)	x, v, f	$\dot{v}$	仿真器 (Lec 08)
Inverse dynamics (ID)	$x, v, \dot{v}$	f, τ	控制器

形式上:

$$f = M\dot{v} + C(x, v) - J^T\lambda$$

给定期望加速度, 反推驱动力。

3.2 Recursive Newton—Euler Algorithm (RNEA)

对铰接链 ID 的经典 $O(N)$ 算法:

[EX] RNEA 两个 pass

前向 pass (root $\rightarrow$ tip, 从广义坐标算极大坐标的运动学):

$$Q_i, x_i, \omega_i, v_i, \dot{\omega}_i, \dot{v}_i \text{ 用 §1.2 公式逐 link 累乘}$$

后向 pass (tip $\rightarrow$ root, 逐 link 写 NE 方程解 f_i, τ_i):

$$m_i \dot{v}_i = f_i - \sum_{k \in c_i} f_k + \sum_j f_j^{\text{ext}}$$

$$I_i \dot{\omega}_i + \omega_i \times I_i \omega_i = \tau_i + r_i \times f_i - \sum_{k \in c_i} (\tau_k + d_{ik} \times f_k) + \sum_j \tau_j^{\text{ext}}$$

其中 c_i 是 link i 的子链路集合; f_k, τ_k 是子链 k 通过关节作用在 i 上的反作用 (牛顿第三定律)。

3.3 示例: 重力补偿 (Gravity Compensation)

[EX] 三段串行 link, 全身静止: $\dot{v}_i = 0, \dot{\omega}_i = 0$

Tip (link 3): $0 = f_3 - m_3 g \Rightarrow f_3 = -m_3 g, \tau_3 = r_3 \times m_3 g$ 。

Link 2 (要承担 link 3 的反作用 f_3, τ_3): $f_2 = -m_2 g - m_3 g$ 。

Link 1: $f_1 = -(m_1 + m_2 + m_3)g$ (每段都要往上”撑住”上方所有质量)。

用 Jacobian Transpose 视角 (见 §4.3) 可一次性写出:

$$\tau_1 = (x_1 - o) \times (-m_1 g) + (x_2 - o) \times (-m_2 g) + (x_3 - o) \times (-m_3 g)$$

$$\tau_2 = (x_2 - p_2) \times (-m_2 g) + (x_3 - p_2) \times (-m_3 g), \quad \tau_3 = (x_3 - p_3) \times (-m_3 g)$$

4. PD 控制: 最简的反馈律

4.1 闭环 vs. 开环

[NOTE] Feedforward / Open-loop

$(f, \tau) = \pi(t)$: 与状态无关, 照本宣科。任何扰动都会让结果偏离预期。

Feedback / Closed-loop

$(f, \tau) = \pi(s_t, t)$: 根据当前状态修正。能抵抗一定扰动, 是真实物理控制的标配。

4.2 P 控制: 弹簧

让质点从 x 移到目标 $\bar{x}$:

$$f = k_p(\bar{x} - x) = k_p e, \quad e = \bar{x} - x$$

k_p 越大，刚度越高，越快收敛。但纯 P 在重力下有稳态误差 e_0 ：

$$k_p e_0 = mg \Rightarrow e_0 = \frac{mg}{k_p}$$

增大 k_p 减小 e_0 ，但越大越容易数值发散。

4.3 PD 控制：弹簧 + 阻尼

加阻尼项：

$$f = k_p(\bar{x} - x) + k_d(\dot{\bar{x}} - \dot{x}) = k_p e + k_d \dot{e}$$

在角色场景里 $\dot{\bar{q}}$ 常取 0：

$$\tau = k_p(\bar{q} - q) - k_d \dot{q}$$

[WARN] PID 中的 I 项

$f = k_p e + k_d \dot{e} + k_i \int e dt$ 。积分项专门消除稳态误差，但在角色动画里很少用——它会引入记忆和过冲，调起来烦。重力补偿一般直接用 ID 求解。

4.4 PD 在角色上的应用

每个关节有自己的 $(\bar{q}_j, k_p, k_d)$ ，独立计算 τ_j ：

- 小 $k_p \rightarrow$ “软”角色，跟踪迟缓、迟钝
- 大 $k_p \rightarrow$ “硬”角色，跟踪精准但易抖动 / 发散
- 小 $k_d \rightarrow$ 振荡明显
- 大 $k_d \rightarrow$ 阻尼过强、动作“黏稠”

[NOTE] 典型经验值 (50 kg 角色)

$k_p = 200$, $k_d = 20$, 仿真步长 $h = 0.5 \sim 1$ ms (1000 ~ 2000 Hz)。经验法则：轻 link 减小 gain；动态动作加大 gain；高 gain 必须配合更小 h 。

4.5 PD 的稳定性分析

考虑一维 $f = -k_p x - k_d v$ (无重力, $m = 1$)。半隐式 Euler：

$$\begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 - k_d h & -k_p h \\ h(1 - k_d h) & 1 - k_p h^2 \end{bmatrix}}_A \begin{bmatrix} v_n \\ x_n \end{bmatrix}$$

$$s_{n+1} = A s_n = A^n s_1$$

对 A 特征分解 $A = P \text{diag}(\lambda_1, \lambda_2) P^{-1}$, 记 $z = P^{-1} s$ ：

$$z_{n+1} = \text{diag}(\lambda_1, \lambda_2) z_n \Rightarrow z_n = \text{diag}(\lambda_1^n, \lambda_2^n) z_1$$

[WARN] 稳定条件

$$|\lambda_i| \leq 1 \quad \forall i$$

任一 $|\lambda_i| > 1 \rightarrow$ 指数发散。

A 的特征值依赖 (k_p, k_d, h) 。 k_p 越大、 h 越大 $\rightarrow$ 越容易越界。

4.6 Stable PD Control (Tan et al. 2011)

把 PD 的 v 写成“下一帧”——对阻尼项隐式化：

$$v_{n+1} = v_n + h(-k_p x_n - k_d v_{n+1})$$

显式解：

$$v_{n+1} = \frac{v_n - h k_p x_n}{1 + h k_d}, \quad x_{n+1} = x_n + h v_{n+1}$$

整体格式 ($m = 1$):

$$\begin{bmatrix} v_{n+1} \\ x_{n+1} \end{bmatrix} = \frac{1}{1 + h k_d} \begin{bmatrix} 1 & -k_p h \\ h & 1 + k_d h - k_p h^2 \end{bmatrix} \begin{bmatrix} v_n \\ x_n \end{bmatrix}$$

[NOTE] 收益

对阻尼项隐式化使系统对 k_d 无条件稳定，对 k_p 的容忍度也大幅提升。Stable PD 让 h 可以放宽到 $\sim 1/120 \sim 1/60$ s (120 / 60 Hz)，从而与渲染同步，工程价值巨大。

真正版本（多 DoF）需在每步局部线性化整套铰接动力学 $\rightarrow$ 解 $O(N)$ 线性方程组。

5. Jacobian Transpose Control (虚拟力控制)

5.1 问题：把“末端目标力”翻译成关节力矩

想象在末端 x （如手）上施加一个“虚拟力” f （不真存在），希望靠关节力矩 τ 重现同样的瞬时效果（同样的功率）。

5.2 功率匹配 $\rightarrow$ 雅可比转置

末端速度 $\dot{x} = J\dot{\theta}$ (Lec 03 FK 雅可比, $J = \partial g / \partial \theta$)。

外力 f 在末端做的功率 = 关节力矩 τ 在所有关节做的功率：

$$f^T \dot{x} = \tau^T \dot{\theta} \iff f^T J \dot{\theta} = \tau^T \dot{\theta}, \quad \forall \dot{\theta}$$

$$\tau = J^T f$$

5.3 几何解读：每个 hinge 关节

对 hinge 关节 i (轴向 a_i , 从关节到末端的偏移 r_i):

$$\frac{\partial g}{\partial \theta_i} = a_i \times r_i$$

代入:

$$\tau_i = (a_i \times r_i) \cdot f = a_i \cdot (r_i \times f)$$

对一般 ball joint:

$$\tau_i = (x - p_i) \times f \quad (p_i : \text{第 } i \text{ 个关节位置})$$

即把虚拟力关于关节点的力矩当作关节力矩。

5.4 应用

[EX] 虚拟力 + Jacobian Transpose 的典型用法

1. 平衡的辅助力: 往 CoM 推一个虚拟力 f_{CoM} , 再用腿/脚关节的 J^T 翻译成关节力矩。2. 末端跟踪: 手要跟目标 $\bar{x} \rightarrow f = k_p(\bar{x} - x) - k_d\dot{x}$, 再 $\tau = J^T f$ 。3. 重力补偿: $f = -mg$ 作用在每个 link 的质心, 沿链 J^T 累加。

6. 跟踪控制：让角色”跳舞”

6.1 全身 Tracking Controller

每个关节 j 独立 PD:

$$\tau_j = k_p(\bar{q}_j - q_j) - k_d\dot{q}_j$$

目标 $\bar{q}(t)$ 来自: ① mocap 序列; ② 关键帧; ③ 手编 FSM; ④ 抽象模型 (如 IPM) + IK。

[EX] 经典工作

- Hodgins & Wooten 1995 “Animating Human Athletics”: 手编关节轨迹 + PD 完成跳水、骑车、跑步。
- NaturalMotion Endorphin: 把关键帧动画交给物理跟踪, 做”会受力反应”的角色。
- SAMCON (Liu et al. 2010): 把 mocap 切片, 采样多条扰动 PD 轨迹再用 MCTS 拼成稳定 tracking。

6.2 PD 是 feedforward 还是 feedback?

PD 中” $\bar{q}$ ”是 feedforward 指令;” $-k_p q - k_d \dot{q}$ ”是 feedback 修正。所以——**两者都是**, 只是反馈环路简单。

6.3 PD 的固有问题

- 稳态误差: 手臂在重力下永远到不了目标角度 (除非加 I 项或 ID 补偿)。
- 滞后: 跟踪 mocap 时仿真轨迹永远滞后参考一个相位。
- 大 k_p 精度高但不稳定 (§4.5); 小 k_p 稳定但跟不上。

6.4 Residual Force / Torque: 作弊但好用

[WARN] 残差力的合法性

角色腰部 (root link) 本不应有外力。但工程里常额外加一对 (f_0, τ_0) 给 root, 称为**残差力 / 残差力矩 (residual force/torque)**。

优点: 消除跟踪误差, 让 mocap 跟踪极其稳健。代价: **违背动量守恒**——总外力净不为零。用法: 物理插值动画 (如 Zordan et al. 2005)、训练 RL 时作辅助; “展示动画”接受这种作弊。

7. 静态平衡 (Static Balance)

7.1 平衡的几何定义

[NOTE] CoM / Support Polygon

– **Center of Mass (CoM)** $c = \sum m_i x_i / M$: 质心位置。– **Projected CoM** $c_\perp$: CoM 向地平面的垂直投影。– **Support Polygon**: 地面与角色接触点 (脚底、手等) 的凸包。

静态平衡: $c_\perp$ 落在 support polygon 内部。

落在外部 → 角色翻倒。“落在边界”是临界稳定。

7.2 简单平衡策略

[EX] 三步骤

1. 设目标 $\bar{c}$ = 支撑多边形中心。2. PD 反馈力矩:

$$\tau = k_p(\bar{c} - c) - k_d\dot{c}$$

3. 应用在踝关节 (ankle strategy) 或髋关节 (hip strategy)。

或者用虚拟力 + Jacobian Transpose: 先算 CoM 上需要的水平校正力

$$f = k_p(\bar{c} - c) - k_d\dot{c}$$

再用腿上各关节 $\tau_i = (c - p_i) \times f$ 翻译成关节力矩——这就是“Virtual Model Control”。

7.3 高级策略: Momentum Control

Macchietto et al. 2009 “Momentum Control for Balance”:

[EX] 单步 QP

把“目标 CoM 加速度 + 目标角动量”放入二次规划:

$$\min_{\tau, \lambda} \|M\dot{v}_{\text{des}} - (M\dot{v} + C - \tau \circ e - J^T \lambda)\|^2 + w\|\tau\|^2$$

约束: 接触力在摩擦锥内、关节力矩饱和、跟踪 mocap。

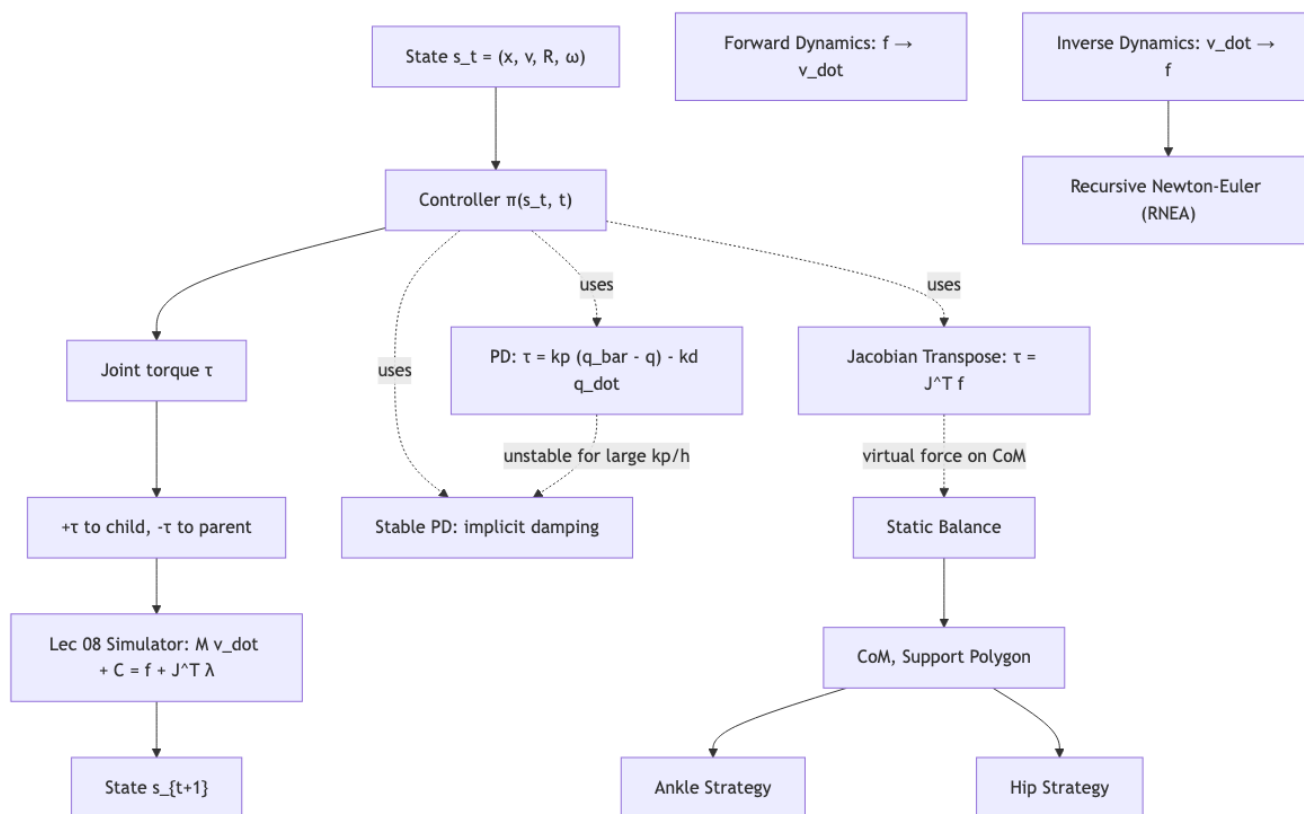
在每个仿真步求解。跟纯 PD 比, 可以“主动调度”上下肢的角动量贡献, 跌倒前能伸臂、屈膝等。

7.4 踝策略 vs. 髋策略

- **Ankle strategy**: 微小扰动, 只动踝关节, 整身像一根杆子前后摇摆。

- **Hip strategy**: 扰动较大，髋关节屈伸来重新分配角动量。
- **Stepping strategy**: 进一步——迈步重置支撑多边形 (→ 进入 Lec 10 动态平衡)。

8. 概念结构图



9. 工程要点速记

[TIP] 实现细节备忘

– 力矩单位: $N \cdot m$; 50 kg 角色髋关节峰值 $\sim 200 N \cdot m$ 。– 关节力矩限幅: 必加上电机/肌肉饱和 (如 $|\tau| \leq \tau_{\max}$), 否则容易 PD 爆出非物理力矩。– PD 的 k_d 不能用临界阻尼公式硬套: 在多刚体耦合下“等效质量”会变。经验是先调 k_p 再扫 $k_d = \eta \sqrt{k_p}$, $\eta \in [0.1, 0.5]$ 。– 目标姿态 $\bar{q}$ 必须是关节局部角 (相对父 link), 不是世界角; 否则 PD 会和 root 旋转耦合。– **Residual force**: 训练 RL 时常常关掉以保证 sim-to-real 可行; 做电影动画时开启换稳定性。– **接触状态机**: 站立/支撑/摆动腿的 PD gain 不同; 摆动腿需放软 k_p 避免“踢到地”产生大冲量。

记号速查

符号	含义
$\theta, \dot{\theta}, \ddot{\theta}$	广义坐标位姿 / 速度 / 加速度
$q, \dot{q}$	关节角 / 角速度 (同 θ)
$\bar{q}, \dot{\bar{q}}$	目标关节角 / 角速度

符号	含义
u_i, l_i, d_i	link i 局部转轴 / 关节 $\rightarrow$ 质心偏移 / 质心 $\rightarrow$ 下关节偏移
Q_i, x_i, v_i, ω_i	link i 极大坐标姿态、位置、速度、角速度
J, J^T	雅可比; $J^T f$ 把末端力翻译成关节力矩
J_i^ω, J_i^v, H_i	角/线速度雅可比 + 二阶 Hessian 项
$\tau, \tau_1 = -\tau_2$	关节力矩 / 一对反向施力
$f_j^{\text{ext}}, \tau_j^{\text{ext}}$	link 上的外力 / 外力矩 (接触、重力等)
f_0, τ_0	Residual root force / torque
k_p, k_d, k_i	PD/PID 比例 / 微分 / 积分系数
$e, \dot{e}$	PD 误差 $\bar{x} - x$ 、误差导数
$c, \bar{c}$	CoM 当前 / 目标位置
h	仿真步长
s, π	控制器状态、控制策略

10. 与上下游课的衔接

[NOTE] 课程脉络上的位置

– 从 Lec 08 (被动仿真) 继承所有动力学方程。– 从 Lec 03 继承 FK / Jacobian。– 向 Lec 10 (Learning to Walk) 输出: 踝/髌静态平衡 + Stable PD 已足够站立, 但走路是连续的失衡——必须迈步, 引入 ZMP、Inverted Pendulum、SIMBICON 等动态平衡机制。– 向 Lec 06 (Learning-based) 输出: PD 是 RL 中“动作空间”的标配——策略网络输出 $\bar{q}$, 再由 PD 转 τ ; DeepMimic / AMP 都基于此。

[TIP] 期中复习要点

1. 极大 vs. 广义坐标: 维度、约束处理、典型 ID 算法 (RNEA) 是否用得上。2. 关节力矩的施加方式: 为何“对子刚体加 τ 、对父刚体加 $-\tau$ ”? 把它写进 NE 方程能体现什么物理量守恒? 3. FD vs. ID: 能写出 $f = M\dot{v} + C - J^T \lambda$ 并解释为什么“角色站不直”问题要在 ID 里加重力补偿项。4. PD 稳定性: 从矩阵 A 的 $|\lambda| \leq 1$ 出发, 解释为什么 stable PD 中“对 v 隐式化”会显著扩大稳定区。5. Jacobian Transpose: 从 $f^T \dot{x} = \tau^T \dot{\theta}$ 推 $\tau = J^T f$, 能写出 hinge 关节的 $\tau_i = (x - p_i) \cdot (a_i \times f)$ 形式。6. 静态平衡: CoM、support polygon、ankle vs. hip strategy 三个概念加上 momentum control 的 QP 思路。7. Residual force 的争议: 什么时候用、违背了什么、与 RL 训练的关系。

Lecture 10 —Learning to Walk: 动态平衡与简化模型

[TIP] 学习指南

本节核心：走路 = 连续可控的失衡 + 步伐规划。静态平衡 (Lec 09) 让角色“立得住”，但只要 CoM 投影离开支撑多边形，就需要迈步重置支撑区。本节用三类经典简化模型分析迈步逻辑：① ZMP (零力矩点) 作为脚不翻转的判据；② Linear Inverted Pendulum Model (LIPM) 把走路抽象为一根倒立摆；③ Cart-Table 模型把“给定 ZMP 反求 CoM”转化为三对角线性系统。最后用 SIMBICON 把这些原理压成一个 4 状态 FSM 上工。前置知识：Lec 08 多体动力学 + 接触力；Lec 09 CoM、support polygon、PD、Jacobian Transpose、关节力矩。重点关注：① 单 / 双支撑相的几何 ② 静态平衡为何不足以走路 ③ ZMP 的力学定义与“在支撑多边形内”等价于脚不翻转 ④ LIPM 微分方程 $\ddot{x} = \frac{g}{z}(x - x_{zmp})$ 与守恒能量 ⑤ Cart-Table preview control ⑥ SIMBICON 的 FSM + linear feedback。

0. 核心问题 / Motivation

[NOTE] 静态 vs. 动态平衡

– 静态平衡 (Static balance)：投影 CoM 在支撑多边形内，速度 = 0。– 动态平衡 (Dynamic balance)：投影 CoM 可以瞬时离开多边形，只要在系统重新跌倒之前迈步重置支撑区。

为什么需要新的工具？Lec 09 末尾的 ankle / hip strategy 假设角色脚一直贴地。但走路本质上：

Single Stance — Double Stance — Single Stance — ...

左脚撑 右脚抬起 / 落下 右脚撑

- **Single stance phase**：只有一只脚接地，support polygon = 一只脚。
- **Double stance phase**：两脚同时接地，support polygon = 两脚之间的凸包。
- **Flight phase** (跑步、跳跃)：双脚离地 → 没有 GRF → 只能靠空中角动量。

[WARN] 静态平衡走路 vs. 动态平衡走路

“慢速”静态平衡走路——每一帧 CoM 投影都被强行控制在 polygon 内——是 ASIMO 早期演示的姿态：步幅小、僵硬、能量低。真正像人的走路：CoM 大部分时间位于支撑多边形外——只在 double stance 短暂回到内部。

1. 零力矩点 ZMP：脚不翻转的判据

1.1 直观图景

设角色单脚站立。脚底每个接触点 p_i 给出反作用力 f_i (法向 + 摩擦)：

$$f_i = f_i^y + f_i^{xz} \quad (f_i^y: \text{竖直分量}, f_i^{xz}: \text{水平分量})$$

合 GRF 与对参考点 p 的总力矩：

$$f_{\text{GRF}} = \sum_i f_i, \quad \tau_{\text{GRF}}(p) = \sum_i (p_i - p) \times f_i$$

1.2 拆水平 / 竖直

假设地面平坦，则 $(p_i - p)$ 都在水平面，可以拆：

$$\tau_{\text{GRF}} = \underbrace{\sum_i (p_i - p) \times f_i^y}_{\text{水平分量 } \tau^{xz}} + \underbrace{\sum_i (p_i - p) \times f_i^{xz}}_{\text{竖直分量 } \tau^y}$$

1.3 ZMP 的形式定义

寻找 p 使得水平分量 $\tau_{\text{GRF}}^{xz} = 0$ ：

$$\sum_i (p_i - p) \times f_i^y = \sum_i p_i \times f_i^y - p \times \sum_i f_i^y = 0$$

$$p_{\text{ZMP}} = \frac{\sum_i p_i f_i^y}{\sum_i f_i^y}$$

[NOTE] 几何含义

p_{ZMP} 就是法向力的加权中心 = Center of Pressure。在平地静止接触场景下， $\text{ZMP} \equiv \text{CoP}$ 。

1.4 ZMP 何时存在 / 何时脚翻转

ZMP 公式来自“CoP”的几何定义，但它必须落在脚底接触区域内才是合理 CoP——因为法向力只能从真实接触面发出。

[WARN] 脚翻转判据

设角色摆位姿要求满足“脚不动”，写出静力 / 力矩平衡 (f_{ankle} 是脚踝接受小腿传来的内力 / 力矩)：

$$f_{\text{ankle}} + f_{\text{GRF}} + mg = 0$$

$$(u - o) \times f_{\text{ankle}} + (p - o) \times f_{\text{GRF}} + (x - o) \times mg + \tau_{\text{ankle}} = 0$$

取水平分量解出 p 。两种情况：- $p \in \text{Support Polygon} \rightarrow \text{ZMP}$ 真的存在，脚保持平贴地。这是动态平衡的指标。- $p \notin \text{Support Polygon} \rightarrow$ 物理上不可能有 CoP 在那里 $\rightarrow$ 真实 CoP $p' \neq p \rightarrow \tau_{\text{GRF}}^{xz} \neq 0 \rightarrow$ 脚开始翻转（绕脚边滚动），不再“贴地”。

1.5 ZMP 与 CoM 的关系：单点近似

把整个角色简化成一个集中在 CoM 的质点（先忽略机身角动量）。- 总外力： $f_{\text{GRF}} - mg$ （向上 GRF + 向下重力）。- 由 Newton: $m\ddot{c} = f_{\text{GRF}} - mg$ 。- 对 ZMP 求力矩（单点模型）自动为 0 $\rightarrow$ ZMP 落在 CoM 加速度向量与地面的交点。

这种简化让我们可以仅用 CoM 的运动方程直接讨论“今天脚下该在哪里”。

2. 线性倒立摆模型 LIPM

2.1 倒立摆 (IPM)

绕一个支点 p 、长度 l 、质量 m 全集中在顶端的杆，仅受重力：

$$\ddot{\theta} = \frac{g}{l} \sin \theta$$

$\theta = 0$ (垂直向上) 是不稳定平衡点：微小扰动指数发散。

[EX] 倒立摆 on a cart

把支点装在可平移的小车上 → 经典控制基准。前后移车产生水平惯性反力 = 把杆“拉回”垂直位置。SLIP (spring-loaded inverted pendulum) 则在腿里加弹簧建模跑步。

2.2 LIPM 的简化假设

约束 CoM 运动在固定高度 z_c 的水平面内 (“不上下浮动”)。考察支点 ZMP 在 $p = (p, 0)$ 、CoM 在 $x = (x, z_c)$ ，重力 g 。

杆受力 f (GRF 沿杆方向)。水平 / 竖直方程：

$$m\ddot{x} = f \sin \theta, \quad f \cos \theta = mg$$

其中 $\tan \theta = (x - p)/z_c$ 。消 f ：

$$\ddot{x}_{\text{com}} = \frac{g}{z_c} (x_{\text{com}} - p_{\text{zmp}})$$

这是 LIPM 的核心方程——线性、二阶、横向独立。每个水平轴一条。

2.3 轨道能量 (Orbital Energy)

当 $p_{\text{zmp}} = 0$ (支点固定原点)：

$$\ddot{x} = \frac{g}{z_c} x$$

能量：

$$E = \frac{1}{2} \dot{x}^2 - \frac{g}{2z_c} x^2$$

(E 是守恒量；注意第二项负号——这是“虚势能”，因为是不稳定平衡。)

[NOTE] 用能量判断步骤

走一步前后能量改变 ΔE 取决于换支撑脚的时机：

$$E_1 = \frac{1}{2} v_f^2 - \frac{g}{2z_c} x_f^2$$

$$E_2 = \frac{1}{2} v_f^2 - \frac{g}{2z_c} (x_f - s)^2$$

其中 s 是迈步距离。 $E_2 < E_1$ 减速； $E_2 > E_1$ 加速。 - 早换脚 (small x_f) → 走得慢。 - 晚换脚 (大 x_f) → 走得快。

走路的“自然控制旋钮”= 步长 / 步频。这种用支撑切换控制能量的视角叫 “Passive Dynamic Walking” (McGeer 1990)：放在斜坡上的纯被动机械人也能稳定走下来。

2.4 用 IPM 做步态规划 (Coros et al. 2010)

[EX] Generalized Biped Walking Control 流程

1. 把角色的 CoM + 站立脚抽象成 IPM。2. 解析 / 数值预测：要让下一步落地后 CoM 静止在新支点上方，下一步该放在哪？3. 根据脚的目标位置生成摆动腿轨迹 (spline / 简单 sine)。4. IK 求出对应的关目标角 $\bar{q}(t)$ 。5. 关节 PD 跟踪，加上 Lec 09 的 Jacobian Transpose 维持上半身姿态。

这个 pipeline 是 2010 年代物理动画“controllable walker”的事实标准。

3. ZMP-Based Walking Pattern Generation (Cart-Table 模型)

3.1 反问题

IPM 给的是 CoM $\rightarrow$ ZMP 的正映射（步态规划侧）。如果需求是：

给定期望 ZMP 轨迹（脚步规划好了），求 CoM 轨迹。

这是工业人形机器人（ASIMO 等）做行走规划的标准设置——脚先排好，再算腰部如何动。

3.2 Cart-Table 模型

把 LIPM 翻过来：一个无质量“桌面”立在 ZMP 之上，桌面上有一辆小车 (mass m)。小车 $\ddot{x}$ 决定桌子翻不翻：

$$p_{\text{zmp}} = x - \frac{z_c}{g} \ddot{x}$$

(这正是 LIPM 方程 $\ddot{x} = \frac{g}{z_c}(x - p)$ 解出 p 的形式。)

[NOTE] 物理直觉

桌面只要保持不翻，净水平力矩对 ZMP 必须为零——这恰好把“脚不翻转”的约束完全卡死。

3.3 离散化与三对角系统

时间离散 $x_i = x(i\Delta t)$ ，用二阶中心差分：

$$\ddot{x}_i \approx \frac{x_{i-1} - 2x_i + x_{i+1}}{\Delta t^2}$$

代入：

$$p_i = x_i - \frac{z_c}{g} \cdot \frac{x_{i-1} - 2x_i + x_{i+1}}{\Delta t^2} = a_i x_{i-1} + b_i x_i + c_i x_{i+1}$$

$$a_i = -\frac{z_c}{g\Delta t^2}, \quad b_i = \frac{2z_c}{g\Delta t^2} + 1, \quad c_i = -\frac{z_c}{g\Delta t^2}$$

[EX] 三对角线性方程组

给定 $\{p_1, \dots, p_{N-1}\}$ (期望 ZMP 序列), 未知 $\{x_0, \dots, x_N\}$:

$$\begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_{N-1} \end{bmatrix} = \begin{bmatrix} a_1 & b_1 & c_1 & & & \\ & a_2 & b_2 & c_2 & & \\ & & \ddots & \ddots & \ddots & \\ & & & a_{N-1} & b_{N-1} & c_{N-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_N \end{bmatrix}$$

$O(N)$ 时间用 Thomas 算法解。每个水平轴独立求解。

3.4 Preview Control (Kajita 2003)

Cart-Table 离散方程的线性 + 因果性意味着可以做 LQR 风格的滚动时域控制 (MPC): 在每一帧, “预览”未来 $T_{\text{preview}} \sim 1$ s 的 ZMP 序列, 求最优 $\ddot{x}$ 。

[NOTE] Why preview?

因为 LIPM 是不稳定系统 ($\ddot{x}$ 与 x 同号), 单纯反馈无法稳定。Preview 把“未来要去的 ZMP”作为 feedforward 输入, 本质和飞行器 / 自行车 controller 用 lookahead 一样。
这就是 ASIMO、HRP-2、Atlas 等工业人形机器人步态规划的核心算法。

3.5 SLIP 与更复杂的简化模型

[EX] Spring-Loaded Inverted Pendulum (SLIP)

腿上加一根弹簧 → 能描述跑步、跳跃中的能量回弹。Mordatch et al. 2010 “Robust Physics-Based Locomotion Using Low-Dimensional Planning”用 SLIP 在地形中规划, 再用 ID 把简化轨迹转回 full-body 力矩。

4. SIMBICON: 4 状态 FSM 走路控制

[NOTE] SIMBICON = SIMPLE Biped LOCOMOTION CONTROL

Yin et al. 2007, SIGGRAPH。在 PD + FSM 框架上加一个简单 linear feedback, 就能让物理仿真的双足角色稳定行走, 并对外力扰动有较好鲁棒性。是基于物理的角色动画领域的 milestone。

4.1 Step 1: 周期 base motion (FSM)

定义 4 个状态:

状态	角色姿态	时长	切换条件
0	左脚抬起摆动	0.3 s	时间到
1	左脚触地	—	foot strike
2	右脚抬起摆动	0.3 s	时间到
3	右脚触地	—	foot strike

每个状态指定一组关节目标角 $\bar{q}$ → 关节 PD 跟踪 → 自然产生迈步循环。

4.2 Step 2: World-frame 控制躯干与摆动腿

[WARN] 单纯关节 PD 不够

用关节相对父 link 的角度做 PD → 躯干会跟随支撑腿一起前倾。要让躯干在世界系下保持直立，必须用“world-frame angle”而非“joint angle”。

设 stance hip 关节 A、躯干、swing hip 关节 B：– 用 PD 控制躯干在 world 系下的角度 → 得到 τ_{torso} 。– 用 PD 控制 swing hip 在 world 系下角度 → 得到 τ_B 。– **stance hip 的力矩由牛顿第三定律决定：**

$$\tau_A = -\tau_{\text{torso}} - \tau_B$$

为什么是这个？stance leg 已经把脚撑在地上不能动，所以躯干 / swing leg 朝想要的方向旋转 → 它们的反力矩必须由 stance hip 经地面承担。

4.3 Step 3: CoM Feedback (关键创新)

把当前 swing hip 的目标角度调整为：

$$\bar{\theta}_d = \bar{\theta}_{d0} + c_d d + c_v v$$

其中：– d = CoM 相对 stance foot 的水平位移（前后偏离）– v = CoM 在 stance frame 下的水平速度 – $c_d, c_v > 0$ 是反馈增益

[EX] 直觉

– $d > 0$ (CoM 偏前) → $\bar{\theta}_d$ 加大 → swing 腿跨更远，**前迈一大步**接住前倾的躯干。– v 大 → 同理，要迈得更远才能“接住”。– $d < 0$ (CoM 偏后) → swing 腿后缩 → 重心被往前拉。
这就是 Lec 09 末尾说的 **stepping strategy** 的具体实现，本质上是用脚的位置改变下一步的支撑多边形。

4.4 SIMBICON 的工程优势

[NOTE] 三层之和能跑

FSM (base motion) + world-frame torso/swing + linear COM feedback

就足以让仿真角色稳定走、跑、扭头、躲球。

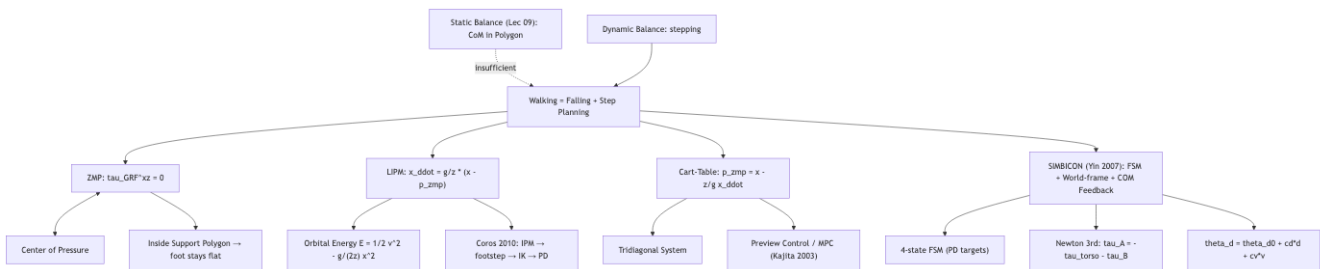
这是物理动画里第一次在通用 simulator (ODE) 上用极简控制律实现 robust locomotion。后续 PFNN、DeepMimic 等“学习派”的 baseline 全是 SIMBICON 类的扩展。

4.5 SIMBICON 的局限

- 只对**周期步态**好用——突发动作（跳跃、踉跄、特技）需要更复杂的 FSM 或学习方法。
- c_d, c_v 仍要手调。
- 推广到不同身形 / 不同步态需要重做 FSM。

这些局限正是 Lec 06 学习派方法 (PFNN / DeepMimic / AMP / ControlVAE) 要解决的问题——把“控制律的设计”转化为“用数据学回归”。

5. 概念结构图

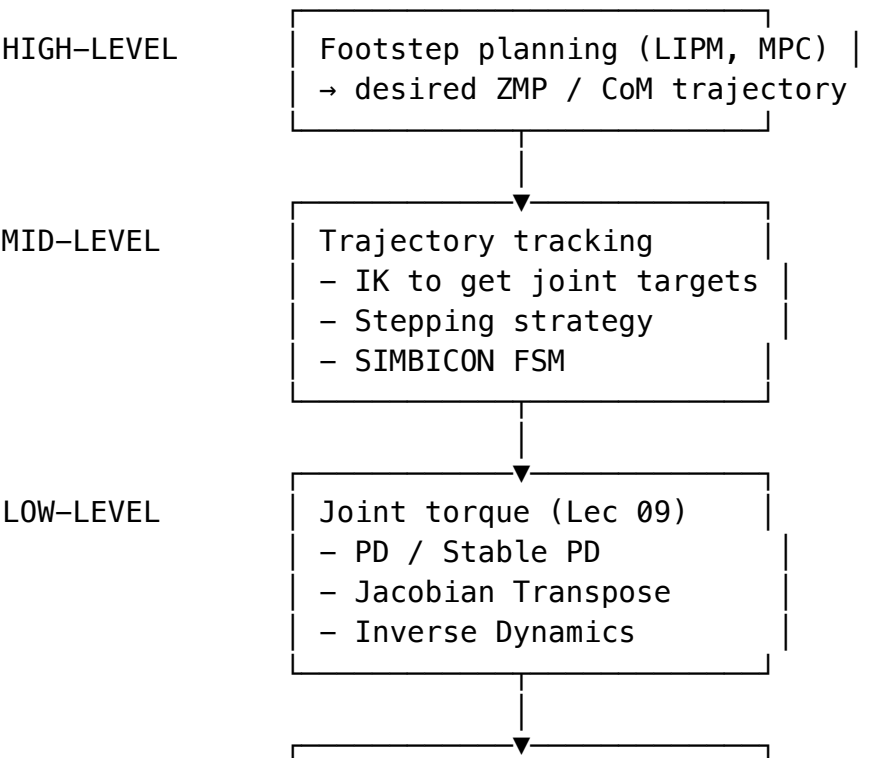


6. 三种简化模型对比

模型	假设	用途	给出	代价
倒立摆 IPM	单点质量 + 杆	直觉 / 步长规划	$\ddot{\theta} = g/l \sin \theta$	不能控
LIPM	CoM 固定高度	步态规划、能量分析	$\ddot{x} = g/z(x - p)$ (线性)	z 固定
Cart-Table	LIPM 反向	ZMP→CoM 三对角解	$p = x - z/g \ddot{x}$	一维线性
SLIP	弹簧腿	跑步 / 跳跃	非线性	需数值积分
完整角色	多刚体 NE	仿真 / 学习	$M\dot{v} + C = f$	高维

每一阶模型抹掉细节换简洁，把”全身控制”压缩为低维 trajectory 规划。

7. 走路控制律的层级



符号	含义
c_d, c_v $\tau_A, \tau_B, \tau_{\text{torso}}$	SIMBICON 反馈增益 stance hip / swing hip / 躯干力矩

10. 关键文献

[EX] 经典论文

– Vukobratovic & Borovac (1972) ——ZMP 概念提出。– Kajita et al. (2001) ——LIPM 命名与论证。– Kajita et al. (2003) ——ZMP preview control + Cart-Table。– McGeer (1990) ——Passive Dynamic Walking。– Yin et al. (2007) ——SIMBICON。– Coros et al. (2010) ——Generalized Biped Walking Control (IPM-based)。– Mordatch et al. (2010) ——Low-dim planning with SLIP。– Macchietto et al. (2009) ——Momentum Control for Balance (Lec 09 末尾)。– Peng et al. (2018) ——DeepMimic (衔接 Lec 06)。– Peng et al. (2021) ——AMP (衔接 Lec 06)。

[TIP] 期中复习要点

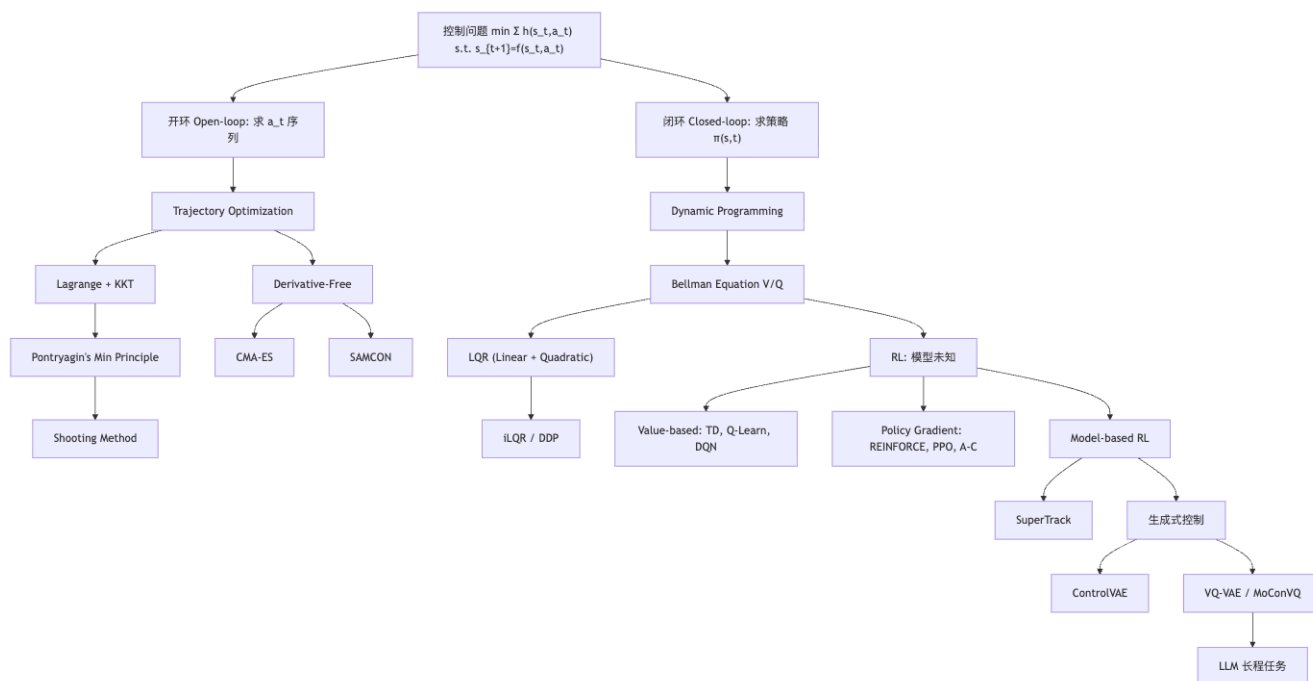
1. 静态 vs. 动态平衡的几何刻画 (CoM, support polygon, single/double stance)。2. ZMP 的定义与判据：能推出 $p_{\text{ZMP}} = \sum p_i f_i^y / \sum f_i^y$ ；能用“ p 是否在 polygon 内”判断脚是否翻转。3. LIPM 推导：从 $\ddot{\theta} = g/l \sin \theta$ 加“固定高度”假设到 $\ddot{x} = g/z(x - p)$ ，能从能量公式 E 看出 early / late switch 对应加 / 减速。4. Cart-Table：三对角系统的形式 + Thomas $O(N)$ 求解 + preview control 的必要性 (LIPM 不稳定)。5. SIMBICON 三步骤：FSM、world-frame torque、linear COM feedback；能解释 stance hip 力矩为何由牛顿第三定律决定。6. 走路控制层级：footstep planning $\rightarrow$ trajectory $\rightarrow$ joint torque $\rightarrow$ simulator；理解每层的边界。7. 经典 vs. 学习：DeepMimic / AMP 在哪些层“替换”了哪些经典模块，哪些层是共享的。

Lecture 11 —Optimal Control & Reinforcement Learning: 从 Pontryagin 到 LLM

[TIP] 学习指南

本节核心：把“控制角色完成任务”的工程目标统一成一个最优化问题 $\min \sum h(s_t, a_t)$ s.t. $s_{t+1} = f(s_t, a_t)$ ，然后沿两条主线展开求解——① **Trajectory Optimization (开环)**：在已知动力学 f 时用 Lagrange 乘子写出 KKT，得到 Pontryagin's Minimum Principle (PMP) 与 shooting 法；② **Dynamic Programming (闭环)**：用 Bellman 最优性原理把整段轨迹的成本递归成 $V(s) = \min_a [h(s, a) + V(f(s, a))]$ ，进而得到 LQR / iLQR / DDP 这种“线性反馈策略 + 二次值函数”的闭式解。后半节把“已知模型”这个假设撤掉就得到 **Reinforcement Learning (MDP)**：模型未知 → 改用采样估计值函数 (TD/Q-Learning/DQN) 或直接优化策略 (Policy Gradient / Actor-Critic / PPO)。最后引出 **Model-based RL** 与 **生成式控制**：把动力学/动作分布也学下来 (SuperTrack, ControlVAE, VQ-VAE, MoConVQ)，从而和 LLM 串接成“自然语言 → 长时序角色控制”。前置知识：Lec 04 PD 跟踪 / 时空约束；Lec 08-10 多刚体动力学 + SIMBICON；Lec 06 数据驱动；线性代数 (Riccati)，多元微分 (Jacobian)，概率论 (期望、马尔可夫链)。重点关注：① “开环 vs 闭环 / Trajectory Opt vs DP”这张大表 ② Lagrange 乘子 → KKT → PMP 的链式推导，理解 λ 为何是“co-state / adjoint”③ Bellman 方程递归形式与 Q-function 定义，记忆 $\pi(s) = \arg \min_a Q(s, a)$ ④ LQR 的 Riccati 反向递归：“末端 → 起点”反推 P_t, K_t ，最优策略是线性反馈 $a_t^* = -K_t s_t$ ⑤ Pontryagin (shooting) $\leftrightarrow$ LQR (Riccati) $\leftrightarrow$ Q-Learning $\leftrightarrow$ Policy Gradient 是同一棵决策树的四个叶子 ⑥ Terrain Runner：用 SAMCON + linear feedback 把高维问题压到 12-维 state、9-维 action (CMA-ES 优化 $12 \times 9 = 108$ 参数) ⑦ DQN/PPO/Actor-Critic 之间的关系，TD target 公式 ⑧ SuperTrack/ControlVAE/MoConVQ 走的“学动力学 + 学 latent 控制空间”路线

0. 章节地图



1. 问题设定：从 Trajectory Tracking 到 Optimal Control

1.1 时空约束 (Spacetime Constraints)

[NOTE] 历史脉络

Witkin & Kass 1988 提出 Spacetime Constraints, 把动画问题第一次写成显式的轨迹优化。本节复用其形式：在物理仿真约束下找最优轨迹。

形式化：找仿真状态序列 $s_0, s_1, \dots, s_T$ 和控制序列 $a_0, \dots, a_{T-1}$, 使

$$\min_{s_t, a_t} h(s_T) + \sum_{t=0}^{T-1} h(s_t, a_t) \quad (1.1)$$

$$\text{s.t. } M\dot{v} + C(x, v) = f + J^\top \lambda, \quad g(x, v) \geq 0$$

经典化简：把连续 EoM 离散化为递推形式

$$s_{t+1} = f(s_t, a_t) \quad (1.2)$$

两者等价，但 (1.2) 更通用——任何“给定 s_t, a_t 算下一刻状态”的仿真器都能套进来。

1.2 简单例子：拟合正弦曲线

PD-servo 受重力小球，状态 $s = (x, v)$ ，控制 $a = \bar{x}$ (PD 目标位置)，力 $f = k_p(\bar{x} - x) - k_d v$ 。希望 $x_t \rightarrow \sin t$ ：

$$\min_{x_n, v_n, \bar{x}_n} \sum_{n=0}^N (\sin t_n - x_n)^2 + \sum_{n=0}^N \bar{x}_n^2 \quad (1.3)$$

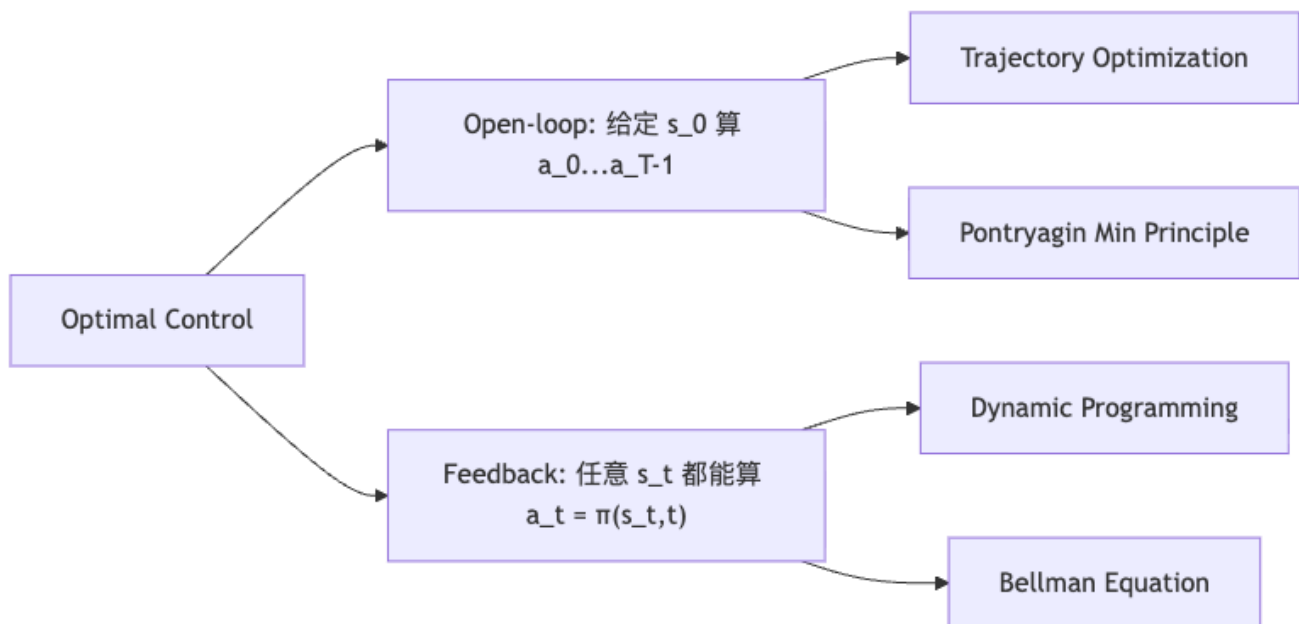
$$\text{s.t.} \quad \begin{cases} v_{n+1} = v_n + h[k_p(\bar{x}_n - x_n) - k_d v_n] \\ x_{n+1} = x_n + h v_{n+1} \end{cases}$$

第二项 $\sum \bar{x}_n^2$ 是正则项，鼓励 PD 目标小幅波动——否则会得到“硬切”解。

1.3 开环 vs 闭环

[WARN] Feedforward 与 Feedback 不只是工程区别，对应两种数学求解范式

– **Feedforward / Open-loop**: 预先算好动作序列 $\{a_t\}$ 。一旦扰动，无纠错能力——见课件 p11-13 受扰后仿真发散。– **Feedback / Closed-loop**: 学一个策略 $\pi(s_t, t)$ ，每帧据当前状态算动作 $a_t = \pi(s_t, t) + \hat{m}_t$ 加上 corrective offset。– 数学上：开环对应 **Trajectory Optimization** (Lagrange/PMP/Shooting)，闭环对应 **Dynamic Programming** (Bellman/LQR/RL)。



2. Trajectory Optimization: Lagrange / KKT / Pontryagin

2.1 约束优化：硬约束的几何含义

考虑标量例子 $\min_x f(x)$ s.t. $g(x) = 0$ 。

[WARN] 软约束行不通

直接转 $\min_x f(x) + w g(x)$ (penalty): 解不一定满足约束。 $w \rightarrow \infty$ 时数值病态。

几何最优条件：最优 x^* 处 $f'(x) \parallel g'(x)$ ，即存在 λ 使

$$f'(x) + \lambda g'(x) = 0 \quad (2.1)$$

2.2 Lagrange Multiplier

引入 Lagrange 函数

$$L(x, \lambda) = f(x) + \lambda^\top g(x) \quad (2.2)$$

最优必要条件 (KKT, equality 版):

$$\frac{\partial L}{\partial x} = f'(x) + \lambda^\top g'(x) = 0, \quad \frac{\partial L}{\partial \lambda} = g(x) = 0 \quad (2.3)$$

[NOTE] 直觉理解

λ 衡量“违反约束一单位带来的代价梯度”。后续 PMP 里 λ_t 就是 **co-state / adjoint variable**, 等价于“状态 s_t 关于最终成本的灵敏度 $\partial J / \partial s_t$ ”。

2.3 应用到轨迹优化

把 (1.2) 约束写成 $f(s_t, a_t) - s_{t+1} = 0$, 对每一时刻引入 λ_{t+1} :

$$L(s, a, \lambda) = h(s_T) + \sum_{t=0}^{T-1} \left[h(s_t, a_t) + \lambda_{t+1}^\top (f(s_t, a_t) - s_{t+1}) \right] \quad (2.4)$$

对 s_T , a_t , s_t , λ_t 分别取偏导为 0:

偏导	方程	含义
$\partial L / \partial s_T$	$\lambda_T = h'_s(s_T)$	末端协态边值
$\partial L / \partial a_t$	$h'_a(s_t, a_t) + f'_a(s_t, a_t)^\top \lambda_{t+1} = 0$	控制方程
$\partial L / \partial s_t$	$\lambda_t = h'_s(s_t, a_t) + f'_s(s_t, a_t)^\top \lambda_{t+1}$	协态反向递推
$\partial L / \partial \lambda_t$	$s_{t+1} = f(s_t, a_t)$	正向动力学

把 (b) 改写成 $a_t = \arg \min_a \{h(s, a) + f(s, a)^\top \lambda_{t+1}\}$, 即得:

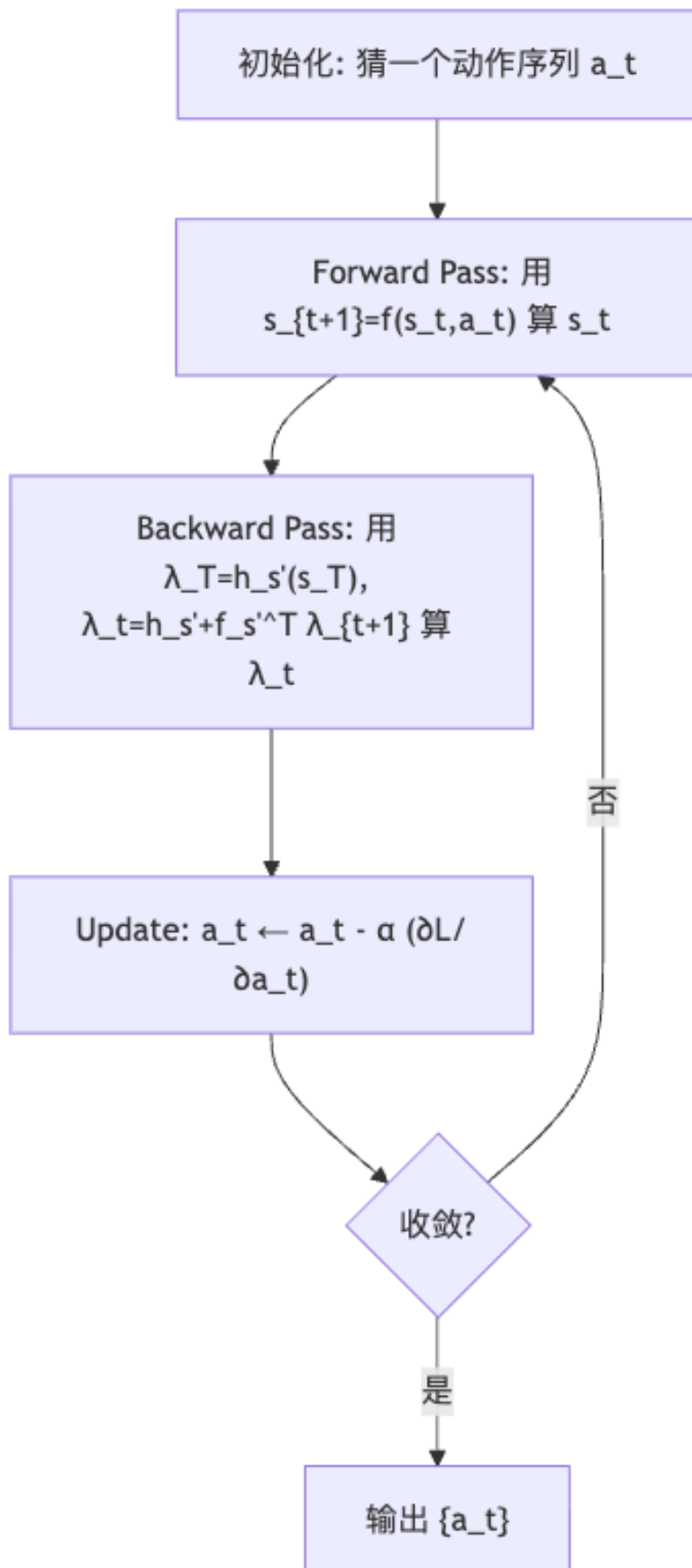
2.4 Pontryagin's Minimum Principle (PMP, 离散版)

[TIP] 三组等式 = 离散 PMP

$$\begin{aligned} \lambda_T &= h'_s(s_T) \\ \lambda_t &= h'_s(s_t, a_t) + f'_s(s_t, a_t)^\top \lambda_{t+1} \\ a_t &= \arg \min_a \left[h(s_t, a) + f(s_t, a)^\top \lambda_{t+1} \right] \\ s_{t+1} &= f(s_t, a_t) \end{aligned} \quad (2.5)$$

λ_t 通常叫 **co-state**。它从 T 向 0 反向递推; s_t 从 0 向 T 正向递推; 动作在中间被两者夹着求 $\arg \min$ 。

2.5 Shooting Method (求解 PMP 的迭代算法)



[WARN] Shooting 的局限

- 高度非线性：人体动力学的 cost landscape 极其崎岖 (Hämäläinen 2020)，梯度方向常常误导。– 黑盒动力学：物理仿真器（接触 + 约束）通常不可微， f'_s, f'_a 不可得。
- 应对：① collocation / multiple shooting 把长轨迹切成小段；② derivative-free (CMA-ES、SAMCON)。

2.6 Derivative-Free Optimization

通用框架（适合“ f 可仿真但不可导”）：

初始猜测 x ；循环：

- 围绕 x 采样候选 x_i
- 仿真评估 $f_i = f(x_i)$
- 用 $\{x_i, f_i\}$ 更新估计 x

代表：Bayesian Optimization、Evolution Strategies、Sequential Monte Carlo。

2.7 CMA-ES (Covariance Matrix Adaptation – ES)

[EX] CMA-ES 一轮迭代

1. 维护高斯分布 $\mathcal{N}(\mu, \Sigma)$
2. 采样 $x_i \sim \mathcal{N}(\mu, \Sigma)$
3. 跑仿真，得 $f_i = f(x_i)$
4. 按 f_i 排序，留 top- N “elite”
5. 由 elite 估计新的 μ, Σ 并自适应步长

代表工作：Wampler & Popovic 2009 (Optimal Gait), Al Borno et al. 2013 (Full-Body Movements with Complex Contacts)。

2.8 SAMCON (SAmpling-based Motion CONtrol)

[NOTE] SAMCON 核心理念 (Liu et al. 2010/2015)

把“跟踪 mocap”分解成短段 δt 。每段独立采样 K 个 PD 目标 a ，并行仿真后选一个最接近 reference 的 state 当作下段起点（受 Sequential Monte Carlo 启发）。最终把“全局非线性大问题”拆成 $T/\delta t$ 个低维子问题。



最终拼出开环控制轨迹 $\tilde{a}_0, \dots, \tilde{a}_{T-1}$ 。但开环对扰动无鲁棒——需要叠加 feedback。下一节解决。

3. Dynamic Programming 与 Bellman 方程

3.1 最短路视角

把状态空间当作图，每条边带权 $h(s_t, s_{t+1})$ ，目标找 $\min J(s_0) = \sum_t h(s_t, a_t)$ 。在离散有限图上：逆序动态规划给出 $O(|S||A|)$ 算法。

3.2 Bellman's Principle of Optimality

[TIP] Bellman 最优性原理 (1957)

“An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.”

⇒ 问题具有 **optimal substructure**: 最优解的尾段也是最优的。

3.3 值函数 $V(s)$

$$V(s) = \min_{\pi} \sum_{t=0}^{\infty} h(s_t, a_t) \Big|_{s_0=s} \quad (3.1)$$

= 从 s 出发、按最优策略走完任务的总最小成本。

利用最优子结构展开第一步:

$$V(s_0) = \min_{a_0} \left[h(s_0, a_0) + \min_{\pi} \sum_{t=1}^{\infty} h(s_t, a_t) \right] = \min_{a_0} [h(s_0, a_0) + V(s_1)] \quad (3.2)$$

3.4 Bellman 方程 (最优控制版)

$$V(s) = \min_a [h(s, a) + V(f(s, a))] \quad (3.3)$$

$$\pi(s) = \arg \min_a [h(s, a) + V(f(s, a))] \quad (3.4)$$

3.5 Q-function (状态-动作值函数)

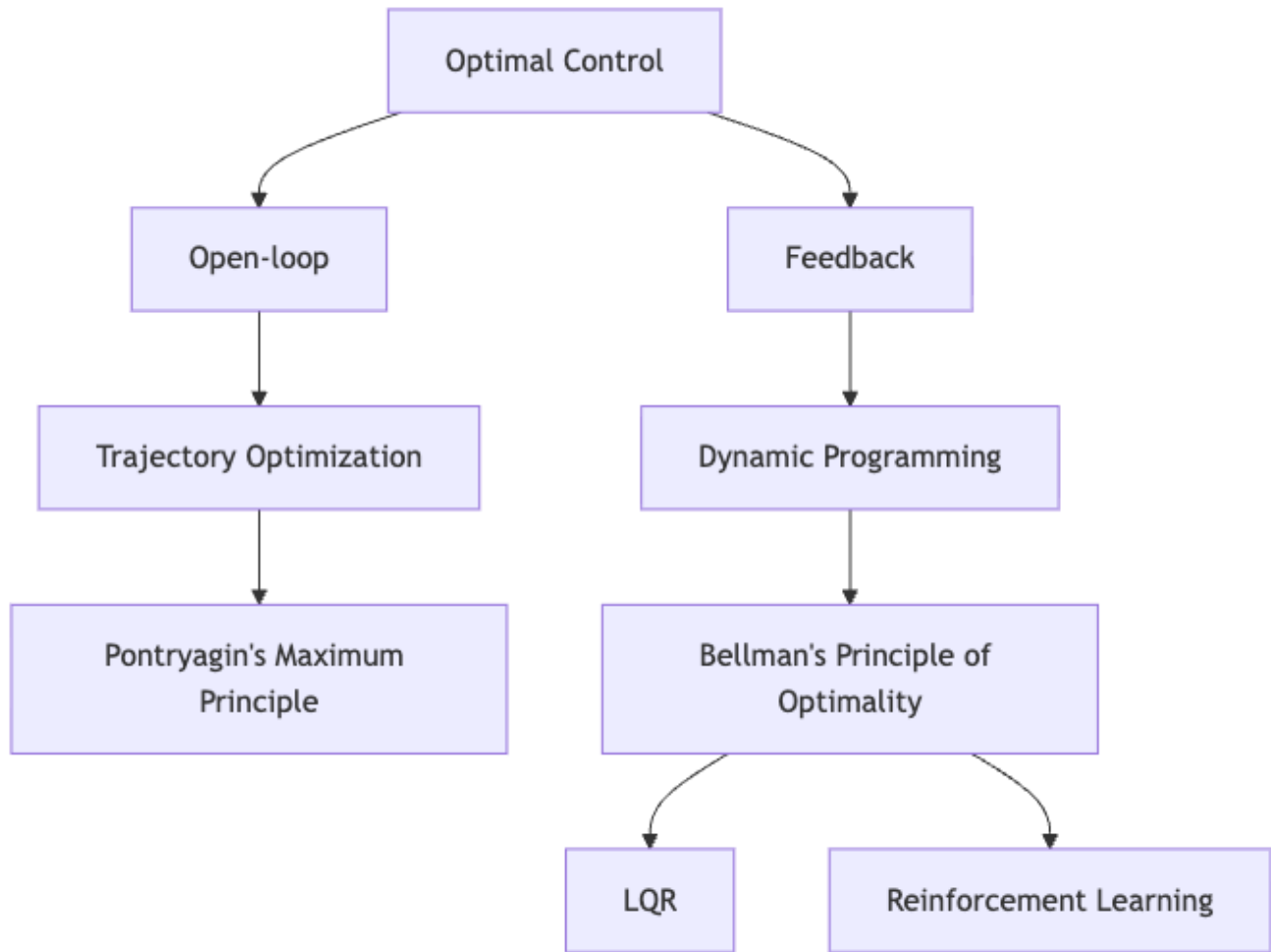
$$Q(s, a) = h(s, a) + V(f(s, a)) \quad (3.5)$$

$$V(s) = \min_a Q(s, a), \quad \pi(s) = \arg \min_a Q(s, a) \quad (3.6)$$

[NOTE] 为什么要区分 V 和 Q

$V(s)$ 只告诉你“这个状态值多少”; $Q(s, a)$ 多了一个 a , 直接告诉你哪一动作好。 $\arg \min_a$ 对离散 a 是 table-lookup, 对连续 a 才需要进一步优化——这就是 RL 各种方法分歧的源头。

3.6 Bellman 在四象限中的位置



4. Linear Quadratic Regulator (LQR)

4.1 问题定义

LQR = 线性动力学 + 二次目标:

$$\begin{aligned} \min \quad & s_T^\top Q_T s_T + \sum_{t=0}^{T-1} (s_t^\top Q_t s_t + a_t^\top R_t a_t) \\ \text{s.t.} \quad & s_{t+1} = A_t s_t + B_t a_t, \quad 0 \leq t < T \end{aligned} \quad (4.1)$$

[NOTE] 这里有个隐含设计选择

目标里没有 a_t 的目标值 (隐含目标 = 让 $s_t \rightarrow 0$, $a_t \rightarrow 0$)。换句话说, LQR 是个 **regulator**——把状态压回原点。要跟踪非零参考, 可以做 affine 变换或转写成 error dynamics。

4.2 反向递推: 从末端到起点

第 T 步: 因 a_T 不再有作用, $V(s_T) = s_T^\top Q_T s_T$ 。设 $P_T := Q_T$ 。

第 $T - 1$ 步：把 Bellman 展开

$$Q(s_{T-1}, a_{T-1}) = s_{T-1}^\top Q_{T-1} s_{T-1} + a_{T-1}^\top R_{T-1} a_{T-1} + V(s_T)$$

代入 $s_T = A_{T-1} s_{T-1} + B_{T-1} a_{T-1}$ ：

$$= s_{T-1}^\top \underbrace{[Q_{T-1} + A_{T-1}^\top P_T A_{T-1}]}_{\text{记 } Q_{xx}} s_{T-1} + a_{T-1}^\top \underbrace{[R_{T-1} + B_{T-1}^\top P_T B_{T-1}]}_{\text{记 } Q_{aa}} a_{T-1} + 2 s_{T-1}^\top \underbrace{A_{T-1}^\top P_T B_{T-1}}_{Q_{xa}} a_{T-1}$$

对 a_{T-1} 求导 =0：

$$a_{T-1}^* = - \underbrace{[R_{T-1} + B_{T-1}^\top P_T B_{T-1}]^{-1} B_{T-1}^\top P_T A_{T-1}}_{:= K_{T-1}} s_{T-1}$$

回代：

$$V(s_{T-1}) = s_{T-1}^\top P_{T-1} s_{T-1}, \quad P_{T-1} = (\text{closed-form in } Q, R, A, B, P_T)$$

4.3 LQR 主结论

[TIP] 三句话记住 LQR

1. 最优策略是线性反馈： $a_t^* = -K_t s_t$ 2. 最优值函数是二次型： $V(s_t) = s_t^\top P_t s_t$ 3. 增益矩阵满足 Riccati 反向递推：

$$K_t = (R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t \quad (4.2)$$

$$P_t = Q_t + A_t^\top P_{t+1} A_t - A_t^\top P_{t+1} B_t (R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t \quad (4.3)$$

[WARN] 注意 P 不是任意正定矩阵

Riccati 递推保证：若 $Q_T \succeq 0$, $Q_t \succeq 0$, $R_t \succ 0$, 则所有 $P_t \succeq 0$ 。这是 LQR 闭环稳定性的源头。

4.4 非线性扩展：iLQR / DDP

[NOTE] iLQR (iterative LQR)

在当前 nominal 轨迹 $(\bar{s}_t, \bar{a}_t)$ 处做一阶展开 f + 二次展开 h ：

$$h(s_t, a_t) \approx h(\bar{s}_t, \bar{a}_t) + \nabla h \cdot \delta + \frac{1}{2} \delta^\top \nabla^2 h \delta, \quad f(s_t, a_t) \approx \bar{f} + \nabla f \cdot \delta$$

得到一个局部 LQR 问题，反向递推算 K_t ，更新轨迹；迭代直至收敛。

DDP (Differential Dynamic Programming) = iLQR + 把 f 也展到二阶，更精但更慢。

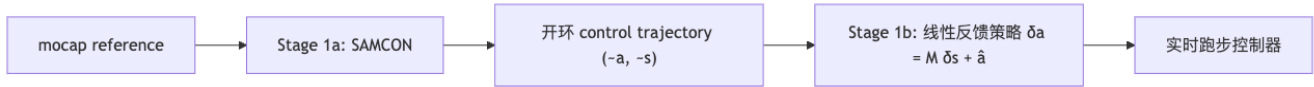
应用：Muico et al. 2011 Composite Control of Physically Simulated Characters。

5. Sampling-based Policy Optimization (Terrain Runner 案例)

5.1 通用框架

迭代寻找参数化策略 $\pi(s; \theta)$ 让 $J(\theta) = \sum_t h(s_t, a_t)$ 最小：1. 围绕 θ 采样候选 θ_i 2. 用 π_{θ_i} 控制角色仿真出 (s_t, a_t) 序列 3. 评估 $J(\theta_i)$ 4. 更新 θ (如 CMA-ES)

5.2 Terrain Runner (Liu et al. 2012) 两阶段管线



Stage 1a —Open-loop policy: 用 SAMCON 把 mocap 段拟合成一组 $(\tilde{s}_t, \tilde{a}_t)$ 。

Stage 1b —线性反馈: 测得当前 deviation $\delta s = s - \tilde{s}$, 输出 $\delta a = M\delta s + \hat{a}$ 。再叠加在 $\tilde{a}$ 上: $a = \tilde{a} + \delta a$ 。

5.3 Reduced-order State

直接学 $M \in \mathbb{R}^{|a| \times |s|}$ 维度爆炸。引入低秩分解:

$$\delta a = \underbrace{M_{ap}}_{|a| \times r} \cdot \underbrace{M_{sp}}_{r \times |s|} \cdot \delta s + \hat{a} \quad (5.1)$$

其中 r 是手选的 reduced-order state 维数。

5.4 手工设计的 state / action

[EX] Terrain Runner 的实际维度

– Running state s : 12 维 = { CoM 横向偏置, CoM 速度, swing-leg phase, support-leg phase, ... } – Action a : 9 维 = { 9 个关键关节 PD 目标偏移 } – 优化变量: 完整 M 是 $12 \times 9 = 108$; 用 reduced-order $r = 3$ 时 $12 \times 3 + 3 \times 9 = 63$ – 训练成本: 12 分钟 $\times$ 24 核 CPU, CMA-ES

[WARN] 这是”手设计状态特征 + 线性策略”→ 后面 RL 节直接用神经网络代替手设计, 但要付出仿真采样代价

6. 从最优控制到 Reinforcement Learning

6.1 区别在哪?

[TIP] 一句话

RL = 最优控制 + 模型未知 / 模型不准 / 系统有噪声 / 系统高度非线性

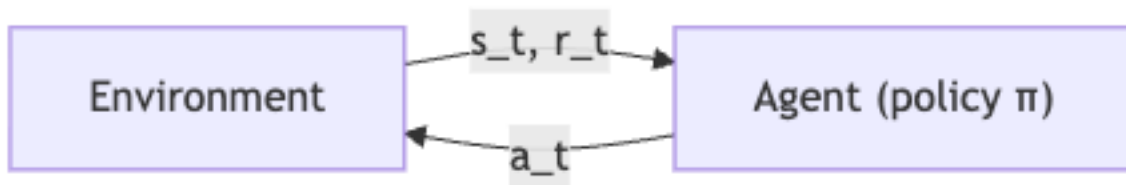
维度	Optimal Control	Reinforcement Learning
动力学 f	已知, 可微	未知, 黑盒
求解工具	KKT, Riccati, iLQR	采样, TD, 策略梯度
目标	$\min \sum h$	$\max \sum r = \max \sum -h$
折扣 γ	通常 1	$\gamma \in (0, 1)$ (无穷视界)
策略	多为确定性	常为随机 $a \sim \pi(\cdot s)$

Model-based RL 介于二者之间: 把 f 也用神经网络学下来 → 仍可走 OC 路线。

6.2 Markov Decision Process (MDP)

[NOTE] MDP 五元组 $M = \{S, A, p, r, \gamma\}$

- S 状态空间, A 动作空间 - $p(s' | s, a)$ 转移概率 (替代确定性 f) - $r(s, a)$ 奖励 (取负就是 cost h)
- $\gamma \in (0, 1]$ 折扣因子, 保证无穷和收敛



轨迹 $\tau = (s_0, a_0, s_1, a_1, \dots)$ 上的回报:

$$R(\tau) = \sum_t \gamma^t r(s_t, a_t) \quad (6.1)$$

目标:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_t \gamma^t r(s_t, a_t) \right] \quad (6.2)$$

6.3 Bellman 方程: RL 版

[WARN] 与最优控制版的差别

RL 里有两组 Bellman 方程: - **Policy 评估** (V^π, Q^π , 对固定 π 的期望回报, 未必最优):

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [r(s, a) + \gamma V^\pi(s')] \quad (6.3)$$

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}[V^\pi(s')] \quad (6.4)$$

- **最优 Bellman** (V^*, Q^* , 对 $\max_\pi$):

$$V^*(s) = \max_a [r(s, a) + \gamma \mathbb{E}[V^*(s')]]$$

把 min 换 max、 h 换 $-r$ 、再加上期望 (因转移随机), 就和最优控制完全对齐了。

7. Value-Based Methods

7.1 Temporal Difference (TD) Learning

核心思想: 用一步真实回报 + 下一状态估计值, 构造自举目标。

$$V(s_t) \leftarrow V(s_t) + \underbrace{\alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]}_{\text{TD error}} \quad (7.1)$$

[NOTE] 为什么叫“Temporal Difference”

TD error = “用 $t + 1$ 时刻信息修正 t 时刻估计”的差。和 supervised learning 不同——目标 $r_t + \gamma V(s_{t+1})$ 自身也是估计，叫 **bootstrapping**。这是 RL 收敛理论的核心。

7.2 Q-Learning

把 TD 用到 Q 上：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (7.2)$$

[WARN] Off-policy

Q-learning 是 off-policy：行为策略可随意，更新目标里强行用 max（即 greedy）。这让它能复用旧数据 → replay buffer。

7.3 Deep Q-Network (DQN)

参数化 $Q(s, a; \theta)$ 为神经网络，最小化 TD 残差平方：

$$\theta^* = \arg \min_{\theta} \sum_{(s, a, r, s') \in D} \left[r + \gamma \max_{a'} Q_{\text{target}}(s', a'; \theta_{\text{target}}) - Q(s, a; \theta) \right]^2 \quad (7.3)$$

[TIP] DQN 两个关键工程 trick

1. **Replay buffer D** ：缓存历史 transitions，随机抽样去除时间相关。2. **Target network θ_{target}** ：用 Polyak 平均缓慢更新 $\theta_{\text{target}} \leftarrow (1 - \beta)\theta_{\text{target}} + \beta\theta$ ，否则 TD 目标随训练剧烈漂移导致发散。

代表：Mnih et al. 2015 Human-level control through deep RL（Atari 49 款达到/超越人类）。

[WARN] DQN 与连续动作

$\max_{a'} Q(s, a')$ 对连续 a 难算 → DQN 主要用于离散动作。连续控制要走 policy gradient 或 DDPG / SAC (DQN 的 actor-critic 推广)。

8. Policy Gradient Methods

8.1 直接对策略求导

参数化 $\pi(a | s; \theta)$ ，目标 $J(\theta) = \mathbb{E}_{\tau \sim \pi}[R(\tau)]$ 。

[TIP] 策略梯度定理 (Sutton et al. 1999)

$$\nabla_{\theta} J(\theta) \propto \sum_s d^{\pi}(s) \sum_a Q^{\pi}(s, a) \nabla_{\theta} \pi(a | s; \theta) \quad (8.1)$$

等价写法 (log-trick)：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim d^{\pi}, a \sim \pi} [Q^{\pi}(s, a) \nabla_{\theta} \log \pi(a | s; \theta)] \quad (8.2)$$

其中 $d^{\pi}(s)$ 是 π 下的状态平稳分布。

8.2 REINFORCE

用 Monte-Carlo return 估计 Q^π : 跑完一整段 τ , 用 $\sum_{t' \geq t} \gamma^{t'-t} r_{t'}$ 当 $Q^\pi(s_t, a_t)$ 的无偏估计。优点: 纯 model-free; 缺点: 方差大。

8.3 Actor-Critic

[NOTE] AC = Value-based + Policy Gradient

– Critic: 用 TD 同时学 V 或 Q – Actor: 用 critic 估计的 Q^π 套入策略梯度更新 θ – 两者交替/同步训练。比 REINFORCE 方差小很多。

伪代码:

循环:

```
采样 (s_t, a_t, r_t, s_{t+1})
critic 更新:  $V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$ 
优势  $A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 
actor 更新:  $\theta \leftarrow \theta + \beta A_t \nabla \log \pi(a_t | s_t; \theta)$ 
```

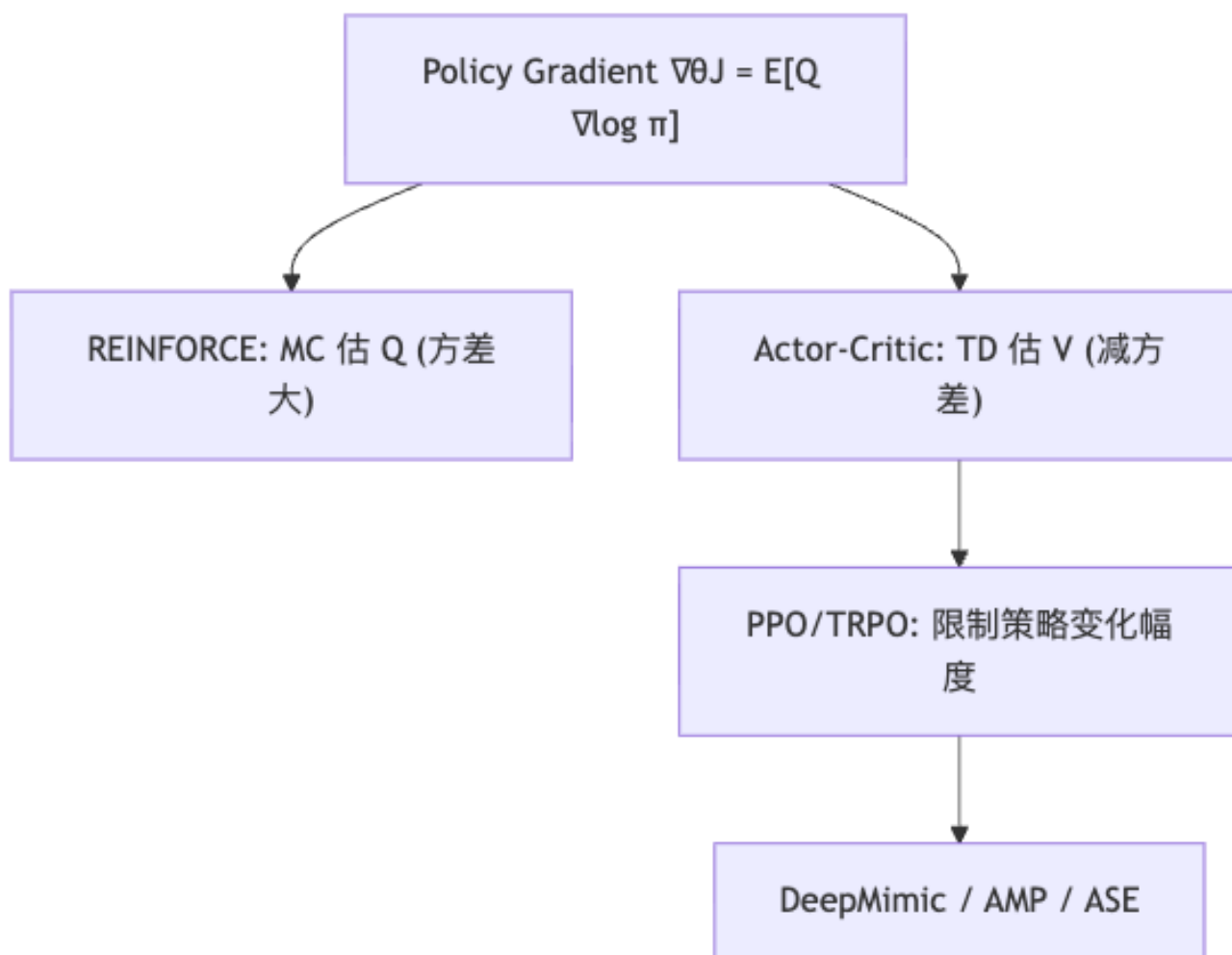
8.4 TRPO / PPO

[WARN] vanilla 策略梯度的痛点

学习率太大 $\rightarrow$ 一步走太远, 新策略效果突然崩; 太小 $\rightarrow$ 慢得令人发指。

TRPO 用 trust region 约束 $D_{KL}(\pi_{old} \parallel \pi_{new}) \leq \delta$; PPO 用 clipped surrogate $\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)$ 近似, 工程上更简洁。
PPO 是当下角色控制 (DeepMimic、AMP、ASE、MoConVQ 等) 默认 RL 算法。

代表: DeepMimic (Peng et al. 2018) 一用 PPO + reference-state init + tracking reward, 实现物理仿真角色拟合 mocap。



9. Model-Based Reinforcement Learning

9.1 动机

[WARN] Model-free 的两大成本

– 采样代价高：每条 (s, a, s', r) 来自一次真实仿真，单条几十秒。– 梯度不能传过仿真器：必须用 score-function trick 估梯度。

如果有可微的模型 $\hat{f}$ ，直接对整段轨迹反向传播：

$$\min_{\theta} h(s_0, \pi_{\theta}(s_0)) + h(\hat{f}(s_0, a_0), \pi_{\theta}(\hat{f}(s_0, a_0))) + \dots$$

需要 $df/ds, df/da, dh/ds, dh/da, d\pi/ds$ ——神经网络模型全部可得。

9.2 Learning a Model

[EX] SuperTrack (Fussell et al. 2021)

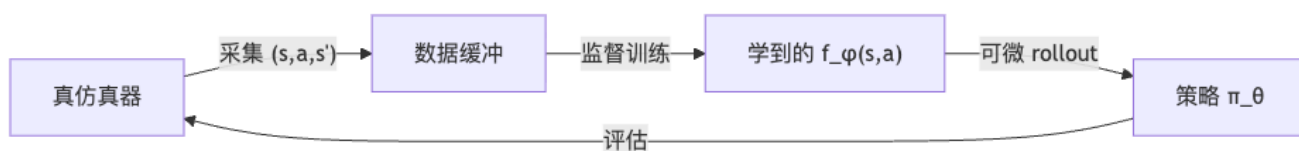
用神经网络 $\hat{f}_\phi(s, a)$ 学习 PD-servo 后的人体动力学。损失 = 多步预测误差：

$$L = \sum_{t=1}^N \|s_t - \bar{s}_t\|^2, \quad \bar{s}_{t+1} = \hat{f}_\phi(\bar{s}_t, a_t), \quad \bar{s}_0 = s_0$$

训出来后 policy 直接走 backprop-through-time, 比 PPO 数据效率高一个数量级。

! [Red: prediction, White: simulation] — 课件 p136 验证了多步预测精度接近真仿真。

9.3 与 OC 的回路



10. 生成式控制：从 VAE 到 LLM

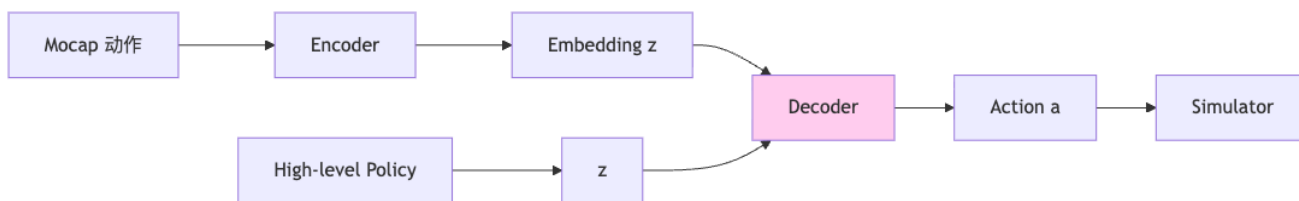
10.1 为什么需要 latent control

[WARN] 高维高频角色控制的固有难点

直接学 $\pi : s \mapsto a$ (a 是 N 关节 PD 目标, 几十维) 会面临：- 动作分布多模：同一状态下“左转/右转”都合理 → 单峰高斯 actor 会平均化 - 时序连贯：一帧一帧采样的动作不一定形成自然动作 - 数据效率：大量动作可用 mocap 监督，但 RL 信号稀疏

解：先用 mocap 学一个低维 latent action 空间，再让控制策略输出 latent code，由 decoder 解码成关节动作。

10.2 VAE 路线—ControlVAE (Yao et al. 2022)



训练损失：

$$L = L_{\text{rec}} + L_{\text{KL}} \quad (10.1)$$

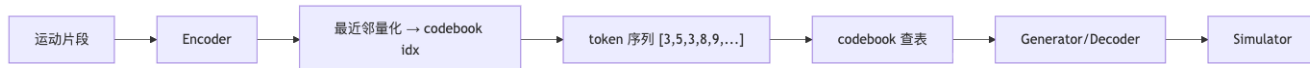
- L_{rec} ：仿真轨迹拟合 mocap - L_{KL} ：让 z 接近标准高斯，便于上层策略采样

两阶段使用：1. 训练阶段：Encoder + Decoder + Control Policy 联合优化 2. 推理阶段：丢弃 Encoder，只保留 latent prior $z \sim \mathcal{N}(0, I)$ 经 Decoder → Action → Sim

10.3 VQ-VAE 路线—Sequential VQ Representation

[TIP] VQ-VAE vs. VAE

– VAE 的 z 连续 $\rightarrow$ 离散下游任务 (“先走再跳再坐”) 不好衔接 LLM – **VQ-VAE** (Oord et al. 2017): 把 z 量化到 K 个 codebook 向量之一 $\rightarrow$ 整段动作 = 离散 token 序列 “..., 3, 5, 3, 8, 9, 10, 1, 0, 0, 2, ...”



10.4 MoConVQ: Text $\rightarrow$ Motion Control

[EX] MoConVQ (Yao et al. 2024)

Codebook 学好后, 把”文本 prompt”喂给 LLM, LLM 输出 codebook token 序列, 物理 decoder 把 token 序列变成实际控制动作。

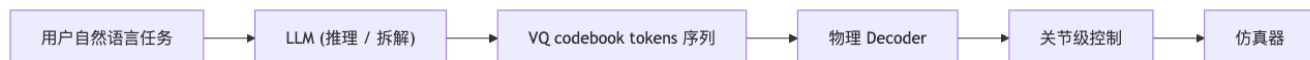
- prompt: “a man walks forward two steps and begins to dance” $\rightarrow$ tokens $\rightarrow$ 物理仿真出实际动作
- prompt: “a person raises his hair and arms and jumps forward very high” $\rightarrow$ 同上

10.5 与 LLM 的对接: Abstract Tasks

[NOTE] LLM 作为高层 planner

Q (user): “How do you make a character walk in a square trajectory?” A (LLM): “I would need the character to take straight walks forward, combined with 90 degree turns at each corner. $\rightarrow$ Forward walk: [297,471,246,463,463]. Sharp right turn: [360,360,360,108,...]”

复杂任务 (开门取钥匙坐椅) 被 LLM 拆解成原子动作 token 序列, 物理 decoder 逐 token 解码 $\rightarrow$ 整套 long-horizon 控制方案。



11. 结论与全局对照

[TIP] 期中复习要点

1. 统一目标: $\min \sum h(s_t, a_t)$ s.t. $s_{t+1} = f(s_t, a_t)$ —决策树的根 2. Lagrange + KKT $\rightarrow$ PMP: 协态 λ_t 从末端反向递推, $a_t = \arg \min_a [h + f^\top \lambda_{t+1}]$ 3. Bellman: $V(s) = \min_a [h(s, a) + V(f(s, a))]$, $\pi(s) = \arg \min_a Q(s, a)$ 4. LQR: 线性 + 二次 $\Rightarrow V$ 是二次型、 π 是线性反馈 $a_t^* = -K_t s_t$, K_t 由 Riccati 反推 5. iLQR/DDP: 在 nominal 轨迹处局部线性化得到 LQR, 再迭代 6. MDP = OC + 模型未知/随机: 奖励替代成本, max 替代 min, 引入期望和折扣 7. Q-Learning / DQN: 用 TD bootstrap, replay buffer + target network 稳定训练; 离散动作友好 8. Policy Gradient / PPO: 用 $\nabla \log \pi$ 估梯度, 连续动作友好, DeepMimic 标配 9. Model-based RL: 学神经网络动力学, 可微 rollout 反传, SuperTrack 代表 10. 生成式控制: ControlVAE (连续 latent) $\rightarrow$ VQ-VAE (离散 codebook) $\rightarrow$ LLM 长程任务拆解 11. 总览决策路径 (必背): 已知 f 且线性二次 $\rightarrow$ LQR (闭式解) 已知 f 且非线性 $\rightarrow$ iLQR / DDP / Trajectory Opt / SAMCON 未知 f 且离散动作 $\rightarrow$ Q-Learning / DQN 未知 f 且连续动作 $\rightarrow$ Policy Gradient / PPO / SAC / Actor-Critic 未知 f 但想用 OC $\rightarrow$ Model-based RL (SuperTrack) LLM-driven 长程 $\rightarrow$ MoConVQ + LLM planner

12. 记号速查

符号	含义
$s_t \in S$	第 t 时刻状态
$a_t \in A$	第 t 时刻动作 (控制)
$f(s, a)$	动力学函数 (已知 / 学得)
$h(s, a)$	阶段成本 (OC)
$r(s, a)$	奖励 (RL, 与 $-h$ 等价)
γ	折扣因子 $\in (0, 1]$
$\pi(s) / \pi(a s)$	确定性 / 随机策略
λ_t	协态 / Lagrange 乘子
$V(s)$	值函数 = 从 s 出发的最优总成本 / 期望回报
$Q(s, a)$	状态-动作值函数
V^π, Q^π	在固定 π 下的 (非必最优) 值/Q
V^*, Q^*	最优值 / 最优 Q
$J(\theta)$	策略 π_θ 的期望回报目标
$d^\pi(s)$	状态平稳分布
A_t, B_t	LQR 的线性动力学矩阵
Q_t, R_t	LQR 的代价加权矩阵
P_t	LQR 值函数 $V = s^\top P_t s$ 的核
K_t	LQR 反馈增益, $a_t^* = -K_t s_t$
D	DQN 经验回放缓冲
TD error	$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$
L_{KL}	VAE 的 KL 正则项
codebook index	VQ-VAE 量化后的离散 token

13. 参考文献

- Witkin & Kass 1988. Spacetime Constraints. SIGGRAPH.
- Pontryagin et al. 1962. The Mathematical Theory of Optimal Processes.
- Bellman 1957. Dynamic Programming. Princeton.
- Hansen 2006. The CMA Evolution Strategy: A Comparing Review.
- Wampler & Popovic 2009. Optimal Gait and Form for Animal Locomotion. SIGGRAPH.
- Liu et al. 2010/2015. Sampling-based Contact-rich Motion Control (SAMCON). SIGGRAPH.
- Liu et al. 2012. Terrain Runner. SIGGRAPH Asia.
- Al Borno et al. 2013. Trajectory Optimization for Full-Body Movements with Complex Contacts. TVCG.
- Hämmäläinen et al. 2020. Visualizing Movement Control Optimization Landscapes.
- Muico et al. 2011. Composite Control of Physically Simulated Characters. TOG.
- Sutton et al. 1999. Policy Gradient Methods for Reinforcement Learning with Function Approximation. NeurIPS.
- Mnih et al. 2015. Human-level Control through Deep Reinforcement Learning. Nature (DQN).
- Schulman et al. 2017. Proximal Policy Optimization Algorithms. (PPO).
- Peng et al. 2018. DeepMimic: Example-Guided Deep RL of Physics-Based Character Skills. SIGGRAPH.
- Fussell et al. 2021. SuperTrack: Motion Tracking for Physically Simulated Characters. SIGGRAPH.
- Oord et al. 2017. Neural Discrete Representation Learning. NeurIPS (VQ-VAE).
- Yao et al. 2022. ControlVAE: Model-Based Learning of Generative Controllers for Physics-Based Characters. TOG.
- Yao et al. 2024. MoConVQ: Unified Physics-Based Motion Control via Scalable Discrete Representations.